A Structural Approach to Interpretable Deep Learning

Technical Supplement

Jakob Zimmermann

Technische Universität Berlin

Contents

List of Figures ii

List of Tables iii

S.1 Extended Game-Theoretic Derivations 1
- S.1.1 Notation and Conventions . . . 1
- S.1.2 Stopping Game . . . 2
- S.1.3 Routing Game: Log-Space Formulation . . . 12
- S.1.4 Analytical Modes . . . 22
- S.1.5 Softplus Variant of the Stopping Game . . . 24

S.2 Extended Attention-Routing Derivations 27
- S.2.1 State space . . . 27
- S.2.2 Legendre–Fenchel transform of the KL divergence . . . 28
- S.2.3 Value-routing equilibrium . . . 29
- S.2.4 $\alpha\beta$-***L**ayer-wise **R**elevance **P**ropagation* (LRP) recovery for the attention block . . . 30
- S.2.5 Risk-Averse Attention via ADF . . . 31

S.3 Extended Trajectory Derivations 33
- S.3.1 Layered Markov Process Foundations . . . 33
- S.3.2 Hellinger Distance from Bhattacharyya Coefficients . . . 35
- S.3.3 Hellinger Identities and Proofs . . . 36
- S.3.4 Proof of the Per-Pixel Decomposition . . . 38
- S.3.5 Conditioned-Survival Trajectory Laws . . . 38
- S.3.6 Randomization Bounds . . . 41

S.4 Extended Experimental Results and Protocol 44
- S.4.1 Full Evaluation Results for the Split Methods . . . 44
- S.4.2 Stabilization Ablations for the Split Methods . . . 50
- S.4.3 Implementation and Evaluation Details . . . 53
- S.4.4 Additional Results . . . 56
- S.4.5 Numerical Diagnostic of the Transformer DC Construction . . . 71
- S.4.6 Formal Verification Details . . . 72

List of References 74

List of Figures

S.1.1 Stopping Game: trajectories and local stopping decisions 5
S.1.2 Routing Game: a local routing subgame 15

S.4.1 Additional VGG-16 saliency map comparisons 45
S.4.2 ViT-B/16 qualitative comparison on six ImageNet-S examples 71

List of Tables

S.1.1 Notation for games and trajectories . . . 1
S.1.2 Modes: default value, effect and cost per image . . . 22

S.4.1 Comprehensive interpretability evaluation: VGG-16 . . . 46
S.4.2 Comprehensive interpretability evaluation: ResNet-18 . . . 47
S.4.3 Randomization sanity check for VGG-16 split methods . . . 48
S.4.4 Mean attribution and model-splitting times . . . 50
S.4.5 Alpha ablation for SplitGrad and SplitCAM . . . 51
S.4.6 Scale versus shift stabilization for SplitLRP . . . 52
S.4.7 Alpha ablation without forward stabilization . . . 52
S.4.8 Alpha ablation with shift normalization . . . 53
S.4.9 Per-method validation grids . . . 55
S.4.10 ViT-B/16: validation-selected configurations . . . 57
S.4.11 VGG-16: validation-selected configurations . . . 58
S.4.12 ResNet-50: validation-selected configurations . . . 59
S.4.13 ViT-B/16: full ImageNet-S results with confidence intervals . . . 59
S.4.14 VGG-16: full ImageNet-S results with confidence intervals . . . 60
S.4.15 ResNet-50: full ImageNet-S results with confidence intervals . . . 61
S.4.16 ViT-B/16 on the Pascal VOC 2012 test pool . . . 63
S.4.17 ViT-B/16: one-factor ablations on ImageNet-S . . . 64
S.4.18 VGG-16: one-factor ablations on ImageNet-S . . . 66
S.4.19 ResNet-50: one-factor ablations on ImageNet-S . . . 67
S.4.20 Pointing game: Quantus tie rule and matched granularity . . . 69
S.4.21 ViT-B/16 Routing Game attention rule ablation . . . 70

S.1 Extended Game-Theoretic Derivations

S.1.1 Notation and Conventions

Throughout, $p, q, r \in \{+, -\}$ label players P^+,P^- or turns, and p' denotes the opponent ($+' = -$, $-' = +$). Table S.1.1 groups the symbols by topic; each has a single meaning. We write $\delta_{p,q} = 1$ if $p = q$ and 0 otherwise, the sign operator $\xi_{p,q} = 2\delta_{p,q} - 1$ ($+1$ if $p = q$, -1 if $p \neq q$), and the XOR operator $p \oplus q = +$ if $p = q$ and $-$ if $p \neq q$; it is commutative and associative, with $p \oplus p = +$, $p \oplus + = p$, $p \oplus - = p'$, and $p' \oplus q = (p \oplus q)'$. **Boundary conventions.** We fix the selected backward value $\mathrm{ReLU}'(0) \coloneqq 0$ (an algorithmic convention, since ReLU has no classical derivative at zero); equivalently, the indicator $\mathbf{1}[z > 0]$ used throughout returns 0 at $z = 0$. For a finite tuple $(x_1, \ldots, x_m)$, $\arg\max_k x_k \coloneqq \min\{k : x_k = \max_l x_l\}$, the smallest index attaining the maximum. We reserve the word *trajectory* for sequences of game states $\zeta = (\zeta_0, \zeta_1, \ldots)$ and use *path* for sequences of neuron indices.

Tab. S.1.1: Notation. Each symbol has one meaning throughout the game and trajectory chapters, except the local attention symbols $Q_t, K_k, e_{tk}, s_{tk}, m, s, c, T_n$ and the token index t, defined in Section 5.5.

Symbol	Meaning	Symbol	Meaning
l, L	layers; input 0 ($a^{(0)} = x$), output L	$W_{ij}^{(l)}$, $b_j^{(l)}$	weight from i (layer $l-1$) to j (layer l); bias
$z_j^{(l)}$, $a_j^{(l)}$	$\sum_i a_i^{(l-1)} W_{ij}^{(l)} + b_j^{(l)}$; $\mathrm{ReLU}(z_j^{(l)})$	$f(x) = a_u^{(L)}$	scalar activation output
$[t]^{\pm}$, $x_k^{\pm}$	$\max\{\pm t, 0\}$, so $t = [t]^+ - [t]^-$; $x_k^{\pm} = [x_k]^{\pm}$	$W^{(l,\pm)}$, $b^{(l,\pm)}$	$[W^{(l)}]^{\pm}$, $[b^{(l)}]^{\pm}$ (sign split)
$z_j^{(l,\pm)}$, $a_{\mathrm{stop},j}^{(l,\pm)}$	sign split of z; Stopping Decomposition	$\nabla^2 f$	Hessian
p, q, r; p'	players / turns in $\{+,-\}$; opponent	$\kappa \in \{+,-\}$	stream sign of a linear routing state
$s_{j,q}^{(l,\mathrm{act})}$, $s_{k,q}^{(0)}$	activation state (q in turn); input terminal	$s_{j,q,\kappa}^{(l,\mathrm{lin})}$	RG linear routing state
$\dagger$	cemetery (absorbing) state	$\perp_{\mathrm{lin}}$	outside action (log-payoff $\log \varepsilon$)
$r_p(s,a)$	payoff of player p	π, $\pi^\star$	policy; equilibrium (SPNE) policy
$\tilde{V}_p(s)$, $\tilde{Q}_p(s,a)$	SG player-specific value, action value	$V_p(s)$	game value (player p minus p')
$Q_r(s,i)$	RG log-gain of routing to predecessor i	$\Sigma_j^{(l,\kappa)}$	$\sum_i [a_i^{(l-1)} W_{ij}^{(l)}]^\kappa + [b_j^{(l)}]^\kappa$
$\gamma_j^{(l)}$	SG discount $\sum_i \lvert W_{ij}^{(l)} \rvert$	γ_{O}	Oracle discount (2 add./branch, 1 max-pool)
$\gamma(s)$	per-state discount: SG $\gamma_j^{(l)}$ or γ_{O}; RG $\in \{2\alpha, 2\beta, 1\}$	ζ	trajectory $(\zeta_0, \zeta_1, \ldots)$
Π_x	equilibrium trajectory law at input x	$d_t(\zeta)$	cumulative trajectory discount
$\Gamma_x(s)$	discounted occupation measure of state s	$\mathscr{H}$, $\mathscr{H}_\theta$	Shannon entropy; binary form
τ	RG temperature (entropy weight)	α, β	$\alpha\beta$-LRP weights, $\alpha - \beta = 1$
ε	LRP stabilizer	$R_i^{(l)}$	LRP relevance
A	attention matrix (held fixed)	v, W_v	Value vectors, projection (sup. "val")
n_h	number of attention heads	N	equivariant-averaging samples
λ	activation-side risk aversion ($\lambda \leq 0$)	λ_{sm}	attention-logit risk aversion
μ, σ, σ^2	ADF mean, std.; input variance	λ_{ent}	fan-in entropy-bias mode
sp_θ, sp'_θ	Softplus $\theta \log(1+e^{z/\theta})$; its gate	Φ, ϕ	normal CDF, density
θ; $\tilde{\Phi}$	Softplus temperature; soft gate $\Phi(\mu/\sigma + \lambda)$	$\mathscr{R}_\Phi$	probit regularizer $\phi(\Phi^{-1}(\pi))$
H, $\mathrm{H}_{\mathrm{surv}}$	Hellinger distance; survival-conditioned	BC	Bhattacharyya coefficient
$T_M^{(l)}$	backward kernel of model M	$m_M^{(l)}(s)$	marginal occupation probability
$g^{(l)}(s)$	overlap mass (backward pass)	$c^{(l)}(s)$	continuation mass (forward pass)
$\mathrm{H}^2(s)$	per-terminal-state disagreement	$\psi_M^{(l)}(s)$	survival probability

S.1.2 Stopping Game

The backward two-player Stopping Game on an extended network graph realizes the Stopping Decomposition (S.1.4) as player-specific values and network gradients as integrals over trajectories.

S.1.2.1 Stopping Decomposition

Throughout we consider a feedforward ReLU network of L layers with input layer 0, $a^{(0)} = x$, pre-activations $z_j^{(l)} = \sum_i a_i^{(l-1)} W_{ij}^{(l)} + b_j^{(l)}$, where $W_{ij}^{(l)}$ connects neuron i in layer $l-1$ to neuron j in layer l, activations $a_j^{(l)} = \mathrm{ReLU}(z_j^{(l)})$, and a *single scalar activation output* $f(x) = a_u^{(L)}$. An ungated affine classifier head is treated separately in Section 4.5.

Canonical sign split. Split the weights and biases elementwise by sign,

$$W^{(l,+)} := [W^{(l)}]^+, \qquad W^{(l,-)} := [W^{(l)}]^-, \qquad b^{(l,+)} := [b^{(l)}]^+, \qquad b^{(l,-)} := [b^{(l)}]^-, \tag{S.1.1}$$

which maps non-negative activation pairs $(a_i^{(l-1,+)}, a_i^{(l-1,-)})$ to the pair of pre-activations at neuron j,

$$z_j^{(l,+)} := b_j^{(l,+)} + \sum_i W_{ij}^{(l,+)} a_i^{(l-1,+)} + \sum_i W_{ij}^{(l,-)} a_i^{(l-1,-)}, \tag{S.1.2}$$

$$z_j^{(l,-)} := b_j^{(l,-)} + \sum_i W_{ij}^{(l,+)} a_i^{(l-1,-)} + \sum_i W_{ij}^{(l,-)} a_i^{(l-1,+)}. \tag{S.1.3}$$

If $a_i^{(l-1,+)} - a_i^{(l-1,-)} = a_i^{(l-1)}$ for all i, the pair recovers the original pre-activation: $z_j^{(l,+)} - z_j^{(l,-)} = b_j^{(l)} + \sum_i (W_{ij}^{(l,+)} - W_{ij}^{(l,-)})(a_i^{(l-1,+)} - a_i^{(l-1,-)}) = z_j^{(l)}$.

The **Stopping decomposition function** $f_{\mathrm{stop}} \colon \mathbb{R}^2_{\geq 0} \to \mathbb{R}^2_{\geq 0}$ is

$$f_{\mathrm{stop}}(z^+, z^-) \;:=\; \big(\mathbf{1}[z^+ - z^- > 0]\, z^+,\; \mathbf{1}[z^+ - z^- > 0]\, z^-\big). \tag{S.1.4}$$

Applied at every neuron and propagated by (S.1.2)–(S.1.3), it defines the **Stopping Decomposition** activations $\big(a_{\mathrm{stop},j}^{(l,+)}, a_{\mathrm{stop},j}^{(l,-)}\big) = f_{\mathrm{stop}}\big(z_j^{(l,+)}, z_j^{(l,-)}\big)$, with the input-layer initialization

$$a_{\mathrm{stop},k}^{(0,\pm)} = x_k^{\pm}. \tag{S.1.5}$$

Theorem S.1.1 (Layer-wise Recovery of the Original Activations, `Lean_55`). *For every layer l and neuron j,*

$$a_{\mathrm{stop},j}^{(l,+)} - a_{\mathrm{stop},j}^{(l,-)} = a_j^{(l)} = \mathit{ReLU}\big(z_j^{(l)}\big). \tag{S.1.6}$$

Proof. Induction on l. At $l = 0$, $x_k^+ - x_k^- = x_k = a_k^{(0)}$ by (S.1.5). At layer $l \geq 1$ the identity above gives $z_j^{(l,+)} - z_j^{(l,-)} = z_j^{(l)}$, hence $a_{\mathrm{stop},j}^{(l,+)} - a_{\mathrm{stop},j}^{(l,-)} = \mathbf{1}\{z_j^{(l)} > 0\}\big(z_j^{(l,+)} - z_j^{(l,-)}\big) = \mathrm{ReLU}(z_j^{(l)}) = a_j^{(l)}$. □

ReLU networks compute continuous piecewise linear functions (Arora et al., 2018), which admit a ***D**ifference of **C**onvex* (DC) decomposition (Zimmermann and Loho, 2026); it uses the same sign split (S.1.2)–(S.1.3) with the pairwise activation $(\max\{z^+, z^-\}, z^-)$ in place of f_{stop} and yields components that are convex in x. The Stopping Decomposition instead gates both streams by the same

indicator, so its components are not convex, but on each linear region they are non-negative affine, hence monotone, in (x^+, x^-).

Parity path decomposition of the split streams. Each non-negative Stopping-Decomposition activation sums contributions from active backward paths to the input. Each contribution is the path's absolute weight product times $x^+_{P_0}$ or $x^-_{P_0}$, selected by the parity of its negative weights.

Lemma S.1.2 (Path view on the Stopping Decomposition, Lean_56). *Fix a feedforward ReLU network with weights $W^{(l)}$ (biases ignored; see remark at end of proof). For neuron j in layer $l \geq 1$, call a* backward path *from j to the input layer a list of neuron indices*

$$P = (P_l, P_{l-1}, \ldots, P_0), \qquad P_l = j, \tag{S.1.7}$$

with P_t indexing a neuron in layer t. The edges of P use the weights $W^{(t)}_{P_{t-1},P_t}$ for $t = 1, \ldots, l$. Define

$$w(P) \coloneqq \prod_{t=1}^{l} W^{(t)}_{P_{t-1},P_t}, \qquad |w|(P) \coloneqq \prod_{t=1}^{l} \big|W^{(t)}_{P_{t-1},P_t}\big|, \tag{S.1.8}$$

the parity sign

$$\operatorname{sgn}(P) \in \{+,-\}, \qquad \operatorname{sgn}(P) = + \iff \#\big\{t : W^{(t)}_{P_{t-1},P_t} < 0\big\} \text{ is even}, \tag{S.1.9}$$

and the gate factor *(1 iff every gate on P at intermediate layers $1, \ldots, l-1$ is open)*

$$G_{<l}(P) \coloneqq \prod_{t=1}^{l-1} \mathbf{1}\big[z^{(t)}_{P_t} > 0\big]. \tag{S.1.10}$$

Then the decomposed pre-activations (S.1.2)–(S.1.3) admit the path-sum representation

$$z^{(l,+)}_j = \sum_{P:\, P_l = j} G_{<l}(P)\, |w|(P)\, x^{\operatorname{sgn}(P)}_{P_0}, \tag{S.1.11}$$

$$z^{(l,-)}_j = \sum_{P:\, P_l = j} G_{<l}(P)\, |w|(P)\, x^{\operatorname{sgn}(P)'}_{P_0}. \tag{S.1.12}$$

Proof. Induct on l in (S.1.11)–(S.1.12).

Base case ($l = 1$): By the input convention (S.1.5), $a^{(0,+)}_{\text{stop},k} = x^+_k$ and $a^{(0,-)}_{\text{stop},k} = x^-_k$. Using (S.1.2) with biases omitted,

$$z^{(1,+)}_j = \sum_k \big(W^{(1,+)}_{kj} x^+_k + W^{(1,-)}_{kj} x^-_k\big). \tag{S.1.13}$$

Paths (j,k) correspond to predecessors k, with $|w|(P) = |W^{(1)}_{kj}|$ and $G_{<1}(P) = 1$ (empty product). For nonzero weights, parity is positive iff $W^{(1)}_{kj} > 0$; zero weights contribute nothing:

- $W^{(1)}_{kj} > 0$: $W^{(1,+)}_{kj} = |W^{(1)}_{kj}|$ contributes $|W^{(1)}_{kj}| x^+_k$ with $\operatorname{sgn}(P) = +$, matching $|w|(P)\, x^{\operatorname{sgn}(P)}_{P_0}$.
- $W^{(1)}_{kj} < 0$: $W^{(1,-)}_{kj} = |W^{(1)}_{kj}|$ contributes $|W^{(1)}_{kj}| x^-_k$ with $\operatorname{sgn}(P) = -$, again matching.

Swapping parity gives $z_j^{(1,-)}$ and (S.1.12).

Inductive step ($l \geq 2$): By (S.1.2),

$$z_j^{(l,+)} = \sum_i \big(W_{ij}^{(l,+)}\, a_{\text{stop},i}^{(l-1,+)} + W_{ij}^{(l,-)}\, a_{\text{stop},i}^{(l-1,-)}\big), \tag{S.1.14}$$

where $a_{\text{stop},i}^{(l-1,\pm)} = \mathbf{1}[z_i^{(l-1)} > 0]\, z_i^{(l-1,\pm)}$. Multiplying the inductive path sum by this gate changes $G_{<l-1}(P')$ to $G_{\leq l-1}(P')$ for each sub-path P' ending at i.

Prepending j to $P' = (i, P'_{l-2}, \ldots, P'_0)$ uniquely gives each path with $P_l = j$ and $P_{l-1} = i$. The new edge multiplies $|w|(P')$ by $|W_{ij}^{(l)}|$ and preserves or flips parity according to its sign. Since $G_{<l}$ excludes the layer-l gate, $G_{<l}(P) = G_{\leq l-1}(P')$.

Case $W_{ij}^{(l)} > 0$. Then $W_{ij}^{(l,+)} = |W_{ij}^{(l)}|$ and $W_{ij}^{(l,-)} = 0$. The surviving term preserves parity, $\operatorname{sgn}(P) = \operatorname{sgn}(P')$; the induction hypothesis gives

$$\begin{aligned} W_{ij}^{(l,+)}\, a_{\text{stop},i}^{(l-1,+)} &= |W_{ij}^{(l)}| \sum_{P' :\, P'_{l-1}=i} G_{\leq l-1}(P')\, |w|(P')\, x_{P'_0}^{\operatorname{sgn}(P')} \\ &= \sum_{P :\, P_l=j,\, P_{l-1}=i} G_{<l}(P)\, |w|(P)\, x_{P_0}^{\operatorname{sgn}(P)}. \end{aligned} \tag{S.1.15}$$

Case $W_{ij}^{(l)} < 0$. Then $W_{ij}^{(l,+)} = 0$ and $W_{ij}^{(l,-)} = |W_{ij}^{(l)}|$. The surviving term $W_{ij}^{(l,-)} a_{\text{stop},i}^{(l-1,-)}$ flips parity: $\operatorname{sgn}(P) = \operatorname{sgn}(P')'$. Thus $x_{P'_0}^{\operatorname{sgn}(P')'} = x_{P_0}^{\operatorname{sgn}(P)}$, and the induction hypothesis for $z_i^{(l-1,-)}$ gives the same path sum. *Case* $W_{ij}^{(l)} = 0$. Both terms vanish and no path through i is prepended.

Summing over i yields (S.1.11). The same argument with positive/negative decomposed activations swapped gives (S.1.12).

Signed recovery. Using $x_k^+ - x_k^- = x_k$ and $\operatorname{sgn}(P) \in \{+,-\}$,

$$z_j^{(l,+)} - z_j^{(l,-)} = \sum_P G_{<l}(P)\, |w|(P)\, \big(x_{P_0}^{\operatorname{sgn}(P)} - x_{P_0}^{\operatorname{sgn}(P)'}\big) = \sum_P G_{<l}(P)\, w(P)\, x_{P_0}, \tag{S.1.16}$$

the standard path sum for $z_j^{(l)}$, with inactive intermediate gates contributing zero.

Biases. Adjoining a bias "stub path" at each intermediate layer l'—starting from (l,j), terminating at the bias node of neuron k' at layer l', with weight product taken over the $l - l'$ edges above and an appropriate gate factor $G_{<l}$ restricted to those edges—handles the bias terms by the same parity rule; the argument above goes through verbatim after absorbing these stub contributions into the path sums. □

S.1.2.2 Game Definition

Relation to the network games of Gaubert & Vlassopoulos (2025). The Stopping Game adapts Gaubert and Vlassopoulos' game to the Stopping Decomposition, retaining per-neuron ± states, bias continuation and input terminal payoffs, sign-split weight transitions, a cemetery state, and stop/continue actions. For attribution, (i) we express their game value $\pm a_j^{(l)}$ as the difference of non-negative player-specific payoffs; (ii) we replace signed terminal values $\pm x_k$ by x_k^+, x_k^-, preserving non-negative payoffs and exposing the parity decomposition of Lemma S.1.2.

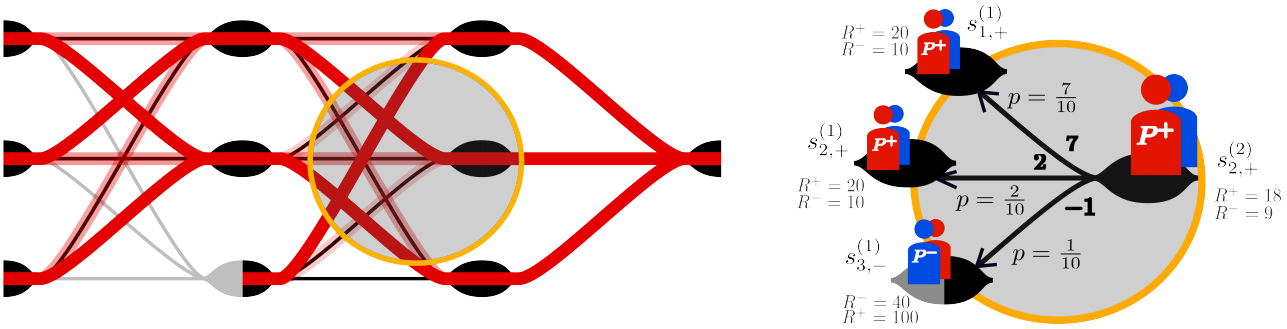


Fig. S.1.1: Stopping Game: trajectory distribution and local stopping decisions on a toy subnetwork. *Left:* trajectory distribution visualized on the network graph. *Right:* local game around $s^{(2)}_{2,+}$. Player P^+ decides whether to stop or continue. Under continuation, transitions are to $s^{(1)}_{1,+}$ with probability 7/10, to $s^{(1)}_{2,+}$ with probability 2/10 (both turn unchanged), and to $s^{(1)}_{3,-}$ with probability 1/10 (turn switches to P^-, reflecting the negative sign of the weight). The labels $R^{\pm}$ are the players' expected payoffs $\tilde{Q}_{\pm}(\cdot, \text{cont})$ (at $s^{(2)}_{2,+}$ without the discount $\gamma^{(2)}_2 = 10$). At $s^{(1)}_{3,-}$, continuing yields expected payoffs R^-=40 and R^+=100, giving continuation advantage $R^- - R^+ = -60 < 0$; therefore P^- stops (so $V_-(s^{(1)}_{3,-}) = 0$), shown by the grayed-out half-neuron. Correspondingly, in the left panel the trajectories terminate at that state (the cemetery state is omitted). Since P^- stops in $s^{(1)}_{3,-}$, traversals to this state contribute zero continuation payoff to both players. Hence, at $s^{(2,\text{act})}_{2,+}$ the positive player has larger expected payoff under continuation than his opponent and chooses to continue.

Definition S.1.1 (Stopping Game State Space). The game is played on the following states:

- **Activation States:** $s^{(l,\text{act})}_{j,q}$ where layer $l \in \{1, \dots, L\}$ and player $q \in \{+,-\}$ is in turn at neuron j.
- **Terminal States:** $s^{(0)}_{k,q}$ where k is an input dimension and $q \in \{+,-\}$ is the final turn label.
- **Cemetery State:** †, an absorbing state reached upon stopping.

In kernels and recursions, $s^{(l-1,\text{act})}_{i,r}$ denotes the predecessor activation state for $l - 1 \geq 1$ and the terminal state $s^{(0)}_{i,r}$ for $l = 1$.

Definition S.1.2 (Payoffs, Transitions, and Value Recursion). Fix an input x.

Payoffs. Player p receives r_p according to the chosen action at activation states, and unconditionally at terminal and cemetery states:

- Cemetery: $r_p(\dagger) := 0$.
- Active state: $r_p(s^{(l,\text{act})}_{j,q}, \text{stop}) := 0$ (stopping pays nothing) and $r_p(s^{(l,\text{act})}_{j,q}, \text{cont}) := b^{(l,p\oplus q)}_j$ (a positive bias pays player q; a negative bias pays its magnitude to q', according to $p \oplus q$).
- Terminal: $r_p(s^{(0)}_{k,q}) := x^{p\oplus q}_k$ (the game ends with a state-only payoff: positive input pays q, negative input pays its magnitude to q').

Transition kernel. Stopping at $s^{(l,\text{act})}_{j,p}$ sends the game to the cemetery. Continuation moves backward one layer with probabilities proportional to absolute incoming weights; positive weights preserve player p's turn and negative weights switch it to p'.

$$\mathbb{P}\left(\dagger \,\middle|\, s^{(l,\text{act})}_{j,q}, \text{stop}\right) := 1, \tag{S.1.17}$$

$$\mathbb{P}\big(s_{i,r}^{(l-1,\mathrm{act})} \,\big|\, s_{j,q}^{(l,\mathrm{act})}, \mathrm{cont}\big) \coloneqq \frac{W_{ij}^{(l,q\oplus r)}}{\gamma_j^{(l)}}, \qquad \gamma_j^{(l)} \coloneqq \sum_i \big(W_{ij}^{(l,+)} + W_{ij}^{(l,-)}\big) = \sum_i \big|W_{ij}^{(l)}\big|, \tag{S.1.18}$$

Here $\gamma_j^{(l)}$ normalizes the kernel and weights trajectories in the occupation measure; if $\gamma_j^{(l)} = 0$, continuation leads to †.

Policy. A pure (deterministic) policy $\pi = (\pi_+, \pi_-)$ assigns the player in turn a stop/continue decision at each active state,

$$\pi_q\big(s_{j,q}^{(l,\mathrm{act})}\big) \in \{0,1\}, \qquad 1 = \mathrm{cont},\ 0 = \mathrm{stop}. \tag{S.1.19}$$

Player-specific action and game values. For each policy π, the player-specific **action value** $\tilde{Q}_p^\pi(s,a)$ is the expected discounted payoff for player p starting at s with action a and following π thereafter, defined by the standard Bellman equation

$$\tilde{Q}_p^\pi(s,a) \coloneqq r_p(s,a) + \gamma(s) \sum_{s'} \mathbb{P}\big(s' \,\big|\, s,a\big)\, \tilde{V}_p^\pi(s'), \tag{S.1.20}$$

where $\tilde{V}_p^\pi$ is the player-specific value defined below; V_p^π denotes its advantage over p'. At active states, (S.1.17)–(S.1.18) specialize (S.1.20) to

$$\tilde{Q}_p^\pi\big(s_{j,q}^{(l,\mathrm{act})}, \mathrm{stop}\big) = 0 + 1 \cdot \tilde{V}_p^\pi(\dagger) = 0 \tag{S.1.21}$$

$$\tilde{Q}_p^\pi\big(s_{j,q}^{(l,\mathrm{act})}, \mathrm{cont}\big) = b_j^{(l,p\oplus q)} + \gamma_j^{(l)} \sum_{s'} \mathbb{P}\big(s' \,\big|\, s_{j,q}^{(l,\mathrm{act})}, \mathrm{cont}\big)\, \tilde{V}_p^\pi(s') \tag{S.1.22}$$

$$= b_j^{(l,p\oplus q)} + \sum_{i,r} W_{ij}^{(l,q\oplus r)}\, \tilde{V}_p^\pi\big(s_{i,r}^{(l-1,\mathrm{act})}\big). \tag{S.1.23}$$

The **player-specific game value** $\tilde{V}_p^\pi(s)$ is the action value averaged by the policy. At terminal and cemetery states it equals the local payoff there:

$$\tilde{V}_p^\pi\big(s_{k,q}^{(0)}\big) \coloneqq x_k^{p\oplus q}, \qquad \tilde{V}_p^\pi(\dagger) \coloneqq 0. \tag{S.1.24}$$

At active states,

$$\tilde{V}_p^\pi\big(s_{j,q}^{(l,\mathrm{act})}\big) \coloneqq \pi_q\big(s_{j,q}^{(l,\mathrm{act})}\big)\, \tilde{Q}_p^\pi\big(s_{j,q}^{(l,\mathrm{act})}, \mathrm{cont}\big) + \big(1 - \pi_q\big(s_{j,q}^{(l,\mathrm{act})}\big)\big)\, \tilde{Q}_p^\pi\big(s_{j,q}^{(l,\mathrm{act})}, \mathrm{stop}\big). \tag{S.1.25}$$

Since π_q is binary, (S.1.25) selects either $\tilde{Q}_p^\pi(\cdot, \mathrm{cont})$ or $\tilde{Q}_p^\pi(\cdot, \mathrm{stop}) = 0$.

Game value. The quantity the player in turn maximizes at each active state is the advantage of player p at s,

$$V_p^\pi(s) \coloneqq \tilde{V}_p^\pi(s) - \tilde{V}_{p'}^\pi(s). \tag{S.1.26}$$

Figure S.1.1 illustrates the trajectory distribution and local stop/continue decisions.

Theorem S.1.3 (Forward Equivalence for Stopping Game, `Lean_57`). *The game admits a Subgame Perfect Nash Equilibrium (SPNE) where both players follow the same pure strategy:*

$$\pi_q^\star\big(s_{j,q}^{(l,\mathrm{act})}\big) = \mathbf{1}\big[z_j^{(l)} > 0\big]. \tag{S.1.27}$$

Under this equilibrium, the player-specific game values and the game values satisfy

$$\tilde{V}_p^{\pi^\star}\big(s_{j,q}^{(l,\mathrm{act})}\big) = a_{\mathrm{stop},j}^{(l,p\oplus q)}, \qquad V_p^{\pi^\star}\big(s_{j,q}^{(l,\mathrm{act})}\big) = \xi_{p,q}\, a_j^{(l)}. \tag{S.1.28}$$

Remark S.1.1 (Uniqueness of the SPNE up to tie-breaking). The proof shows that (S.1.27) is unique except at zero-pre-activation ties, where continuation has zero advantage: $\tilde{Q}_q^{\pi^\star}(\cdot,\mathrm{cont}) - \tilde{Q}_{q'}^{\pi^\star}(\cdot,\mathrm{cont}) = z_j^{(l)} = 0$. The pinned Stopping-Game attribution code selects continuation at exact zero in its hard backward mask, while the Routing Game and Hellinger diagnostic use strict positivity. These choices give the same original forward activation at a tie but can give different selected backward quantities there. The theorem uses the **stop-at-tie-breaks** equilibrium (S.1.27).

Proof. We proceed by backward induction on the layer index l.

Base Case ($l = 0$): By (S.1.24), $\tilde{V}_p^{\pi^\star}(s_{k,q}^{(0)}) = x_k^{p\oplus q} = a_{\mathrm{stop},k}^{(0,p\oplus q)}$ (input convention (S.1.5)). The game value is

$$V_p^{\pi^\star}\big(s_{k,q}^{(0)}\big) = x_k^{p\oplus q} - x_k^{p'\oplus q} = \xi_{p,q}\, x_k, \tag{S.1.29}$$

using $x_k^+ - x_k^- = x_k$ and $x_k^- - x_k^+ = -x_k$.

Inductive Step: Assume the subgames starting at states up to layer $l-1$ admit the claimed SPNE and (S.1.28) holds at layer $l-1$. Consider an active state $s_{j,q}^{(l,\mathrm{act})}$. The active player q chooses $a \in \{\mathrm{stop},\mathrm{cont}\}$ to maximize the advantage $\tilde{Q}_q^{\pi^\star}(s_{j,q}^{(l,\mathrm{act})}, a) - \tilde{Q}_{q'}^{\pi^\star}(s_{j,q}^{(l,\mathrm{act})}, a)$. For $a = \mathrm{stop}$ this advantage is 0 by (S.1.21); for $a = \mathrm{cont}$, using (S.1.22),

$$\tilde{Q}_q^{\pi^\star}\big(s_{j,q}^{(l,\mathrm{act})},\mathrm{cont}\big) - \tilde{Q}_{q'}^{\pi^\star}\big(s_{j,q}^{(l,\mathrm{act})},\mathrm{cont}\big) = \big(b_j^{(l,q\oplus q)} - b_j^{(l,q'\oplus q)}\big) + \sum_{i,r} W_{ij}^{(l,q\oplus r)}\, V_q^{\pi^\star}\big(s_{i,r}^{(l-1,\mathrm{act})}\big) \tag{S.1.30}$$

$$= \big(b_j^{(l,+)} - b_j^{(l,-)}\big) + \sum_{i,r} W_{ij}^{(l,q\oplus r)}\, V_q^{\pi^\star}\big(s_{i,r}^{(l-1,\mathrm{act})}\big). \tag{S.1.31}$$

By the inductive hypothesis $V_q^{\pi^\star}(s_{i,r}^{(l-1,\mathrm{act})}) = \xi_{q,r}\, a_i^{(l-1)}$, with $\xi_{q,q} = +1$ and $\xi_{q,q'} = -1$,

$$\tilde{Q}_q^{\pi^\star}\big(s_{j,q}^{(l,\mathrm{act})},\mathrm{cont}\big) - \tilde{Q}_{q'}^{\pi^\star}\big(s_{j,q}^{(l,\mathrm{act})},\mathrm{cont}\big) = b_j^{(l)} + \sum_i \big(W_{ij}^{(l,q\oplus q)}\,\xi_{q,q} + W_{ij}^{(l,q\oplus q')}\,\xi_{q,q'}\big) a_i^{(l-1)} \tag{S.1.32}$$

$$= b_j^{(l)} + \sum_i W_{ij}^{(l)}\, a_i^{(l-1)} = z_j^{(l)}. \tag{S.1.33}$$

Thus q prefers continuation for $z_j^{(l)} > 0$, stopping for $z_j^{(l)} < 0$, and is indifferent at zero, where we select stopping. Hence $\pi_q^\star(s_{j,q}^{(l,\mathrm{act})}) = \mathbf{1}[z_j^{(l)} > 0]$ extends the lower-layer equilibrium to an SPNE rooted at $s_{j,q}^{(l,\mathrm{act})}$.

To verify (S.1.28), substitute $\pi_q^\star$ into (S.1.25) and use the inductive hypothesis $\tilde{V}_p^{\pi^\star}(s_{i,r}^{(l-1,\mathrm{act})}) = a_{\mathrm{stop},i}^{(l-1,p\oplus r)}$ in (S.1.22):

$$\tilde{V}_p^{\pi^\star}\big(s_{j,q}^{(l,\mathrm{act})}\big) = \mathbf{1}\big[z_j^{(l)} > 0\big]\, \tilde{Q}_p^{\pi^\star}\big(s_{j,q}^{(l,\mathrm{act})}, \mathrm{cont}\big) \tag{S.1.34}$$

$$= \mathbf{1}\big[z_j^{(l)} > 0\big] \left[b_j^{(l,p\oplus q)} + \sum_i \big(W_{ij}^{(l,+)}\, a_{\mathrm{stop},i}^{(l-1,p\oplus q)} + W_{ij}^{(l,-)}\, a_{\mathrm{stop},i}^{(l-1,p\oplus q')} \big) \right]. \tag{S.1.35}$$

If $p = q$, the bracket is $b_j^{(l,+)} + \sum_i \big(W_{ij}^{(l,+)}\, a_{\mathrm{stop},i}^{(l-1,+)} + W_{ij}^{(l,-)}\, a_{\mathrm{stop},i}^{(l-1,-)} \big) = z_j^{(l,+)}$. If $p = q'$, the bracket is $b_j^{(l,-)} + \sum_i \big(W_{ij}^{(l,+)}\, a_{\mathrm{stop},i}^{(l-1,-)} + W_{ij}^{(l,-)}\, a_{\mathrm{stop},i}^{(l-1,+)} \big) = z_j^{(l,-)}$. In both cases, $\tilde{V}_p^{\pi^\star}(s_{j,q}^{(l,\mathrm{act})}) = \mathbf{1}[z_j^{(l)} > 0]\, z_j^{(l,p\oplus q)} = a_{\mathrm{stop},j}^{(l,p\oplus q)}$. Finally,

$$V_p^{\pi^\star}\big(s_{j,q}^{(l,\mathrm{act})}\big) = a_{\mathrm{stop},j}^{(l,p\oplus q)} - a_{\mathrm{stop},j}^{(l,(p\oplus q)')} = \xi_{p,q}\, a_j^{(l)}. \quad \square \tag{S.1.36}$$

□

S.1.2.3 Skip Connections and Max Pooling

For ResNet skip connections and CNN pooling, we extend the Stopping Game with an **Addition Oracle** and deterministic **Max-Pooling Decision State**, paralleling the Routing Game (Section S.1.3.1).

Definition S.1.3 (Addition Oracle). For a residual node $z = x + y$, the *addition state* $s_{z,q}^{\mathrm{add}}$ has active player $q \in \{+,-\}$ and operand states $s_{x,q}, s_{y,q}$ at the two summands. The transition kernel is the uniform Oracle

$$\mathbb{P}\big(s_{x,q} \,\big|\, s_{z,q}^{\mathrm{add}}\big) := \tfrac{1}{2}, \qquad \mathbb{P}\big(s_{y,q} \,\big|\, s_{z,q}^{\mathrm{add}}\big) := \tfrac{1}{2}, \tag{S.1.37}$$

with player q retained at the chosen operand. The immediate payoff is $r_p(s_{z,q}^{\mathrm{add}}) := 0$ for both $p \in \{+,-\}$, and the structural discount is $\gamma_{\mathrm{O}} := 2$.

Proposition S.1.4 (Forward Equivalence for Addition, `Lean_58`). *The player-specific state value at the addition state equals the sum of the operand state values,*

$$\tilde{V}_p^{\pi}\big(s_{z,q}^{\mathrm{add}}\big) \;=\; \tilde{V}_p^{\pi}(s_{x,q}) + \tilde{V}_p^{\pi}(s_{y,q}). \tag{S.1.38}$$

Consequently, if the operand states recover the Stopping-Decomposition activations—$\tilde{V}_p^{\pi^\star}(s_{x,q}) = x^{(p\oplus q)}$ and $\tilde{V}_p^{\pi^\star}(s_{y,q}) = y^{(p\oplus q)}$—then $\tilde{V}_p^{\pi^\star}(s_{z,q}^{\mathrm{add}}) = z^{(p\oplus q)}$, preserving the forward equivalence of Theorem S.1.3 across the residual sum.

Proof. Apply the Bellman recursion (S.1.20)–(S.1.25) at $s_{z,q}^{\mathrm{add}}$ with $r_p = 0$, the uniform Oracle kernel, and $\gamma_{\mathrm{O}} = 2$ from Definition S.1.3:

$$\tilde{V}_p^{\pi}\big(s_{z,q}^{\mathrm{add}}\big) \;=\; 0 + \gamma_{\mathrm{O}} \cdot \Big(\tfrac{1}{2}\, \tilde{V}_p^{\pi}(s_{x,q}) + \tfrac{1}{2}\, \tilde{V}_p^{\pi}(s_{y,q}) \Big) \;=\; \tilde{V}_p^{\pi}(s_{x,q}) + \tilde{V}_p^{\pi}(s_{y,q}), \tag{S.1.39}$$

Substituting the operand assumption and the additive convention $z^{(p\oplus q)} = x^{(p\oplus q)} + y^{(p\oplus q)}$ from (S.1.2)–(S.1.3) (Section S.1.2.1) gives the second identity. □

On the uniform $1/2$ split. The $(\frac{1}{2}, \frac{1}{2})$ Oracle with $\gamma_{\mathrm{O}} = 2$ is not forced by the architecture; it is a convention, the simplest choice inside a one-parameter family of conservation-preserving Oracles $(\rho, 1-\rho)$ with matching discounts $(1/\rho, 1/(1-\rho))$ attached to the branch taken (with the single state discount γ_{O} of (S.1.20) and d_t, $\rho\gamma_{\mathrm{O}} = (1-\rho)\gamma_{\mathrm{O}} = 1$ forces $\rho = \frac{1}{2}$). The uniform split is the maximum-entropy choice—the one that imposes *no prior* on which summand carries more responsibility. In the Stopping Game, Proposition S.1.4 and Theorems S.1.3 and S.1.6 hold for any such split with branch discounts, since the proofs only use that routing probability times Oracle discount equals 1 on each branch; only the trajectory law Π_x (and with it the Hellinger distance of Section 8.3) depends on the split. This invariance does not extend to the Routing Game (Section S.1.3.1), where the split changes the relevance assigned to each summand.

Max pooling is simpler, because the forward operator already selects a single predecessor.

Definition S.1.4 (Max-Pooling Decision State). For a pooled scalar output $z = \max\{x_1, \dots, x_m\}$, the *pooling state* $s^{\max}_{z,q}$ has active player $q \in \{+,-\}$ and admissible action set $\mathscr{A}(s^{\max}_{z,q}) = \{1, \dots, m\}$ indexing the operand states $s_{x_1,q}, \dots, s_{x_m,q}$. Under action k, the transition is deterministic,

$$\mathbb{P}\big(s_{x_k,q} \,\big|\, s^{\max}_{z,q}, k\big) \coloneqq 1, \tag{S.1.40}$$

with player q retained at the chosen operand. The immediate payoff is $r_p(s^{\max}_{z,q}, k) \coloneqq 0$ for both $p \in \{+,-\}$, and the structural discount is $\gamma_{\mathrm{O}} \coloneqq 1$.

Tie-breaking at the pooling state. Tied maxima give indifferent pure best responses with the same forward value and different policies and occupation measures. The *stop-at-tie-breaks equilibrium* of Section S.1.1 selects $k^\star \coloneqq \min\{k : x_k = \max_l x_l\}$. All subsequent gradient and occupation-measure statements use this smallest-index convention.

Proposition S.1.5 (Forward Equivalence for Max Pooling, `Lean_59`). *Assume the operand states recover the Stopping-Decomposition activations, $\tilde{V}_p^{\pi^\star}(s_{x_k,q}) = x_k^{(p\oplus q)}$ for all k, with non-negative operand activations $x_k \geq 0$. Then at the SPNE of the subgame rooted at $s^{\max}_{z,q}$,*

$$\tilde{V}_p^{\pi^\star}\big(s^{\max}_{z,q}\big) \;=\; z^{(p\oplus q)}, \tag{S.1.41}$$

so the pooling state inherits the same Stopping-Decomposition identity as its winning operand and the forward equivalence of Theorem S.1.3 extends across the pooling node.

Proof. The active player q chooses the operand that maximizes its game value $V_q^{\pi^\star} = \tilde{V}_q^{\pi^\star} - \tilde{V}_{q'}^{\pi^\star}$. By the operand assumption, $V_q^{\pi^\star}(s_{x_k,q}) = \xi_{q,q}\, x_k = x_k$ for every k, so the maximizer is $k^\star \in \arg\max_k x_k$ (ties resolved by the smallest index). The Bellman recursion (S.1.20)–(S.1.25) with $r_p = 0$, the deterministic transition, and $\gamma_{\mathrm{O}} = 1$ from Definition S.1.4 gives

$$\tilde{V}_p^{\pi^\star}\big(s^{\max}_{z,q}\big) \;=\; 0 + 1 \cdot \tilde{V}_p^{\pi^\star}(s_{x_{k^\star},q}) \;=\; x_{k^\star}^{(p\oplus q)}. \tag{S.1.42}$$

Since $z = x_{k^\star}$, the right-hand side equals $z^{(p\oplus q)}$. □

S.1.2.4 Discounted Occupation Measures and Gradient Representations

The **discounted occupation measure** connects trajectories to network gradients by weighting visits to each neuron with accumulated discounts, including paths through skips and max pooling.

Definition S.1.5 (Game trajectory and trajectory law, Lean_60). Fix an input x and a root state s. A *game trajectory* rooted at s is a finite sequence of game states

$$\zeta \;=\; (\zeta_0, \zeta_1, \ldots, \zeta_T), \qquad \zeta_0 = s, \tag{S.1.43}$$

following admissible transitions (Definition S.1.2 and the addition/max-pooling Oracles of Definitions S.1.3–S.1.4) until reaching an input terminal $\zeta_T = s^{(0)}_{k,r}$ or the cemetery $\zeta_T = \dagger$ on stopping or zero-weight continuation. These trajectories form the *ground set* $\mathscr{T}_s$. Combining (S.1.17)–(S.1.18), the Oracles, and a possibly stochastic policy π defines a Markov kernel; starting at s induces $\Pi^{\pi}_x(\cdot \mid s)$ on $\mathscr{T}_s$. The ground set includes trajectories continuing past closed gates ($\mathbf{1}[z^{(l)}_j > 0] = 0$), which have zero probability under $\pi^\star$ because it stops there. We write $\Pi_x(\cdot \mid s) \coloneqq \Pi^{\pi^\star}_x(\cdot \mid s)$ for the equilibrium *trajectory law* and $\Pi_x \coloneqq \Pi_x(\cdot \mid s_0)$ for the law rooted at the output state $s_0 \coloneqq s^{(L,\mathrm{act})}_{u,+}$.

Definition S.1.6 (Discounted Occupation Measure, Lean_61). Let $\Pi_x(\cdot \mid s)$ be the equilibrium trajectory law of Definition S.1.5. Define the cumulative discount

$$d_t(\zeta) \coloneqq \prod_{m=0}^{t-1} \gamma(\zeta_m), \qquad d_0(\zeta) = 1, \tag{S.1.44}$$

where $\gamma(\zeta_m) = \gamma_{\mathrm{O}} = 2$ for addition Oracle states, $\gamma(\zeta_m) = \gamma_{\mathrm{O}} = 1$ for max-pooling states, and $\gamma(\zeta_m) = \gamma^{(l)}_j$ for continuation at activation states $\zeta_m = s^{(l,\mathrm{act})}_{j,q}$ (per-neuron weight-magnitude discount). The *discounted occupation measure* of state v for trajectories rooted at s is

$$\Gamma_x(v \mid s) \coloneqq \mathbb{E}_{\zeta \sim \Pi_x(\cdot \mid s)} \left[\sum_t d_t(\zeta)\, \mathbf{1}\{\zeta_t = v\} \right], \qquad \Gamma_x(v) \coloneqq \Gamma_x(v \mid s_0). \tag{S.1.45}$$

Theorem S.1.6 (Activation–Occupation Correspondence, Lean_62). *Fix a neuron state $s = s^{(m,\mathrm{act})}_{j,q}$ and a layer $l \le m$ (for $l = 0$, $s^{(0,\mathrm{act})}_{i,\cdot}$ denotes the terminal state $s^{(0)}_{i,\cdot}$). Under the stop-at-tie-breaks equilibrium fixed by the boundary conventions of Section S.1.1 (uniquely fixing the SPNE at ReLU gates with $z = 0$ and at max-pooling states with multiple maximizers), the discounted occupation measures exactly represent, in a multi-trajectory ReLU network (e.g., a ResNet or a CNN with max pooling), the local Stopping-Decomposition and scalar activation derivatives on open cells with nonzero ReLU pre-activations and unique max-pooling winners. At ties, the same formulas represent the backward quantities computed using the selected gates and winners; they do not assert classical differentiability of the individual stopping streams, which can be discontinuous there:*

1) ***Stopping-Decomposition Gradient:***

$$\frac{\partial a^{(m,p\oplus q)}_{\mathrm{stop},j}}{\partial a^{(l,k)}_{\mathrm{stop},i}} \;=\; \Gamma_x\big(s^{(l,\mathrm{act})}_{i,p\oplus k} \;\big|\; s\big). \tag{S.1.46}$$

2) ***Network Activation Gradient.*** *The ordinary-network gradient of $a^{(m)}_j$ with respect to the scalar activation $a^{(l)}_i$ is*

$$\frac{\partial a^{(m)}_j}{\partial a^{(l)}_i} \;=\; \xi_{q,+}\big(\Gamma_x(s^{(l,\mathrm{act})}_{i,+} \mid s) - \Gamma_x(s^{(l,\mathrm{act})}_{i,-} \mid s)\big). \tag{S.1.47}$$

Proof. We prove Part 1 by induction on $m - l$ and derive Part 2 from it. Fixing x, write $\Gamma(v \mid s) \coloneqq \Gamma_x(v \mid s)$.

Part 2. By Theorem S.1.3, $a_i^{(l)} = a_{\mathrm{stop},i}^{(l,+)} - a_{\mathrm{stop},i}^{(l,-)}$ and $a_j^{(m)} = a_{\mathrm{stop},j}^{(m,+)} - a_{\mathrm{stop},j}^{(m,-)}$. Differentiating $a_j^{(m)}$ along the $+$ direction (perturb $a_{\mathrm{stop},i}^{(l,+)}$, keep $a_{\mathrm{stop},i}^{(l,-)}$ fixed) and applying Part 1 at $k = +$,

$$\frac{\partial a_j^{(m)}}{\partial a_i^{(l)}} = \frac{\partial a_{\mathrm{stop},j}^{(m,+)}}{\partial a_{\mathrm{stop},i}^{(l,+)}} - \frac{\partial a_{\mathrm{stop},j}^{(m,-)}}{\partial a_{\mathrm{stop},i}^{(l,+)}} = \Gamma\big(s_{i,q}^{(l,\mathrm{act})} \mid s\big) - \Gamma\big(s_{i,q'}^{(l,\mathrm{act})} \mid s\big) = \xi_{q,+}\big(\Gamma(s_{i,+}^{(l,\mathrm{act})} \mid s) - \Gamma(s_{i,-}^{(l,\mathrm{act})} \mid s)\big), \tag{S.1.48}$$

using $q \oplus + = q$ and $q' \oplus + = q'$ in the second equality.

Part 1. The decomposition recursion of Theorem S.1.3 writes the upper-layer Stopping-Decomposition activation as

$$a_{\mathrm{stop},j}^{(m,p\oplus q)} = \mathbf{1}\big[z_j^{(m)} > 0\big]\left[b_j^{(m,p\oplus q)} + \sum_h \Big(W_{hj}^{(m,+)}\, a_{\mathrm{stop},h}^{(m-1,p\oplus q)} + W_{hj}^{(m,-)}\, a_{\mathrm{stop},h}^{(m-1,p\oplus q')}\Big)\right]. \tag{S.1.49}$$

Base case ($m = l$). If $s = s_{i,r}^{(l,\mathrm{act})}$ already lies at the target neuron and decomposed activation, then

$$\frac{\partial a_{\mathrm{stop},i}^{(l,p\oplus r)}}{\partial a_{\mathrm{stop},i}^{(l,k)}} = \delta_{p\oplus r,\,k} = \mathbf{1}\{r = p \oplus k\} = \Gamma\big(s_{i,p\oplus k}^{(l,\mathrm{act})} \mid s_{i,r}^{(l,\mathrm{act})}\big). \tag{S.1.50}$$

Inductive step (linear routing, $m > l$). Differentiating (S.1.49) with respect to $a_{\mathrm{stop},i}^{(l,k)}$ (the bias $b_j^{(m,p\oplus q)}$ does not depend on lower-layer activations) and substituting the inductive hypothesis at layer $m-1$,

$$\frac{\partial a_{\mathrm{stop},j}^{(m,p\oplus q)}}{\partial a_{\mathrm{stop},i}^{(l,k)}} = \mathbf{1}\big[z_j^{(m)} > 0\big] \sum_h \Big(W_{hj}^{(m,+)}\, \Gamma\big(s_{i,p\oplus k}^{(l,\mathrm{act})} \mid s_{h,q}^{(m-1,\mathrm{act})}\big) + W_{hj}^{(m,-)}\, \Gamma\big(s_{i,p\oplus k}^{(l,\mathrm{act})} \mid s_{h,q'}^{(m-1,\mathrm{act})}\big)\Big). \tag{S.1.51}$$

On the game side, the one-step occupation recursion at $s = s_{j,q}^{(m,\mathrm{act})}$ uses transition probabilities $W_{hj}^{(m,q\oplus r)}/\gamma_j^{(m)}$ together with the matching cumulative-discount factor $\gamma_j^{(m)}$ and the equilibrium gate $\mathbf{1}[z_j^{(m)} > 0]$:

$$\Gamma\big(s_{i,p\oplus k}^{(l,\mathrm{act})} \mid s\big) = \mathbf{1}\big[z_j^{(m)} > 0\big] \sum_{h,r} W_{hj}^{(m,q\oplus r)}\, \Gamma\big(s_{i,p\oplus k}^{(l,\mathrm{act})} \mid s_{h,r}^{(m-1,\mathrm{act})}\big). \tag{S.1.52}$$

The right-hand sides agree, proving the linear-routing step.

Addition-node step. For an addition node $z = x + y$, the additivity of the Stopping Decomposition from Section S.1.2.1 gives $a_{\mathrm{stop},z}^{(p\oplus q)} = a_{\mathrm{stop},x}^{(p\oplus q)} + a_{\mathrm{stop},y}^{(p\oplus q)}$, so by the inductive hypothesis,

$$\frac{\partial a_{\mathrm{stop},z}^{(p\oplus q)}}{\partial a_{\mathrm{stop},i}^{(l,k)}} = \Gamma\big(s_{i,p\oplus k}^{(l,\mathrm{act})} \mid s_{x,q}\big) + \Gamma\big(s_{i,p\oplus k}^{(l,\mathrm{act})} \mid s_{y,q}\big). \tag{S.1.53}$$

The Oracle split of Definition S.1.3 sends mass $\frac{1}{2}$ to each branch and applies the discount $\gamma_{\mathrm{O}} = 2$, so the occupation measure satisfies the matching identity

$$\Gamma\big(s_{i,p\oplus k}^{(l,\mathrm{act})} \mid s_{z,q}^{\mathrm{add}}\big) = 2 \cdot \tfrac{1}{2}\,\Gamma\big(s_{i,p\oplus k}^{(l,\mathrm{act})} \mid s_{x,q}\big) + 2 \cdot \tfrac{1}{2}\,\Gamma\big(s_{i,p\oplus k}^{(l,\mathrm{act})} \mid s_{y,q}\big). \tag{S.1.54}$$

Max-pooling step. For a max-pooling node $z = \max\{x_1, \dots, x_m\}$ with winner $k^\star$, Proposition S.1.5 gives $a_{\mathrm{stop},z}^{(p\oplus q)} = a_{\mathrm{stop},x_{k^\star}}^{(p\oplus q)}$, and the inductive hypothesis yields

$$\frac{\partial a_{\mathrm{stop},z}^{(p\oplus q)}}{\partial a_{\mathrm{stop},i}^{(l,k)}} = \Gamma\big(s_{i,p\oplus k}^{(l,\mathrm{act})} \mid s_{x_{k^\star},q}\big). \tag{S.1.55}$$

The deterministic transition with unit discount in Definition S.1.4 gives the matching identity $\Gamma(s_{i,p\oplus k}^{(l,\mathrm{act})} \mid s_{z,q}^{\max}) = \Gamma(s_{i,p\oplus k}^{(l,\mathrm{act})} \mid s_{x_{k^\star},q})$. This proves the max-pooling step and completes the induction. □

S.1.3 Routing Game: Log-Space Formulation

The Routing Game recursion of Section 4.3.2 represents an entropy-regularized two-player game in log space, with a Gibbs subgame-perfect equilibrium for every $\tau > 0$. Its global network interpretation requires non-negative activation outputs; the later GELU discussion concerns local gates conditioned on a fixed forward pass.

S.1.3.1 Exact Routing Kernel Behind $\alpha\beta$-*Layer-wise Relevance Propagation* (LRP)

The Legendre–Fenchel identity for Shannon entropy $\mathscr{H}(\pi) \coloneqq -\sum_i \pi(i) \log \pi(i)$ (Boyd and Vandenberghe, 2004) solves each one-step linear routing subgame: for every $\tau > 0$, $n \in \mathbb{N}$, and payoff vector $r \in \mathbb{R}^n$,

$$\max_{\pi \in \Delta_n} \Big\{ \textstyle\sum_i \pi(i)\, r_i + \tau\, \mathscr{H}(\pi) \Big\} = \tau \log \textstyle\sum_i \exp(r_i/\tau), \qquad \pi^\star(i) \propto \exp(r_i/\tau). \tag{S.1.56}$$

We write

$$\ell(z) \coloneqq \begin{cases} \log z, & z > 0, \\ -\infty, & z \le 0, \end{cases} \qquad \exp(-\infty) \coloneqq 0, \tag{S.1.57}$$

Thus $\exp(\ell(x)) = \mathrm{ReLU}(x)$ for every $x \in \mathbb{R}$, and nonpositive routed mass receives the stopping value $-\infty$. Activation policies implement ReLU gating as in the Stopping Game: *stop at every log-space tie*, and continue exactly when the log value exceeds $-\infty$, equivalently when the signed routed mass is positive.

State Space and Game Mechanics. The state space distinguishes player turns from stream signs:

- **Activation States:** $\mathscr{S}_{act}^{(l)} = \{s_{j,p}^{(l,act)}\}$, where $p \in \{+,-\}$ is the player in turn.
- **Linear Routing States:** $\mathscr{S}_{lin}^{(l)} = \{s_{j,q,\kappa}^{(l,lin)}\}$, where q is the player turn and $\kappa \in \{+,-\}$ is the stream sign.

At $s_{j,p}^{(l,act)}$, the active player p chooses between *stop* and *continue*. Under the *continue* action, the environment's transition kernel is uniform over the two sign branches,

$$\mathbb{P}\big(s_{j,p,+}^{(l,lin)} \mid s_{j,p}^{(l,act)}, \text{continue}\big) = \tfrac{1}{2}, \qquad \mathbb{P}\big(s_{j,p',-}^{(l,lin)} \mid s_{j,p}^{(l,act)}, \text{continue}\big) = \tfrac{1}{2}, \tag{S.1.58}$$

The positive branch retains player p; the negative branch switches to p'. An Oracle structural discount $\gamma_{\mathrm{O}} = 2$ compensates for the $1/2$ branching; it differs from the Stopping Game's weight-magnitude discount $\gamma_j^{(l)}$.

Definition S.1.7 (Entropy-Regularized Two-Player Routing Game). Fix an input x and stabilizer $\varepsilon \geq 0$. At each activation state $s_{j,p}^{(l,act)}$, player p chooses *stop* or *continue*, modeling the ReLU gate as in the Stopping Game (Section S.1.2). Stopping sends the game to † with immediate payoff $\ell(0) = -\infty$; continuation uses the uniform sign-branch kernel above and discount $\gamma = 2$. Opponent gains at the negative-stream state $s_{j,p',-}^{(l,\mathrm{lin})}$ count as losses for p. At each linear routing state $s_{j,q,\kappa}^{(l,lin)}$, the player in turn instead chooses a mixed action $\nu \in \Delta(\mathrm{Pred}_j^{(l)} \cup \{\mathrm{bias}, \bot_{\mathrm{lin}}\})$ over the predecessors of neuron j augmented by a terminal *bias node* with immediate payoff $\ell([b_j^{(l)}]^\kappa)$ and a fictitious *outside option* $\bot_{\mathrm{lin}}$ which routes the game's mass to the cemetery with immediate payoff $\ell(\varepsilon)$; trajectories entering either are absorbed. Let $\mathscr{H}(\nu) \coloneqq -\sum_a \nu(a) \log \nu(a)$ be Shannon entropy over this augmented action set. At $\varepsilon = 0$ the outside option carries $\ell(0) = -\infty$ and is dominated by every positive-mass predecessor, so the augmented game reduces to the Routing Game over $\mathrm{Pred}_j^{(l)} \cup \{\mathrm{bias}\}$.

An admissible profile chooses a pure activation action and a probability vector at each linear routing state. Its recursions use these choices in place of maxima, with expected log score plus entropy at routing states. Zero-probability terms contribute zero even at score $-\infty$; positive probability on such a score makes the routing value $-\infty$.

The player-specific log values $\mathbf{V}(\cdot) = (V_p(\cdot), V_{p'}(\cdot))^\top$ are defined by backward induction through (S.1.63) and (S.1.66). The finite DAG makes this well-posed: at $l = 1$, (S.1.59) needs no lower-layer value; subsequent layers use only strictly lower-layer values.

At a **linear routing state** with $l \geq 2$, routing to predecessor i gives the active player the log weight plus the next state's log value; at $l = 1$ it gives the terminal input payoff. The inactive player's payoff is $\ell(0) = -\infty$. Write $Q_r(s,a)$ for player r's action gain and $\bar{Q}_r$ for the entropy-regularized mixture of opponent-payoff differences under the chosen strategy.

$$Q_r\big(s_{j,q,\kappa}^{(l,lin)}, i\big) \coloneqq \begin{cases} \ell\big(W_{ij}^{(l,\kappa)}\big) + V_r\big(s_{i,q}^{(l-1,act)}\big), & r = q,\ l \geq 2, \\ \ell\big([x_i W_{ij}^{(1)}]^\kappa\big), & r = q,\ l = 1, \\ -\infty, & r = q'. \end{cases} \tag{S.1.59}$$

The outside option $\bot_{\mathrm{lin}}$ and the bias node have log-scores $Q_r(s_{j,q,\kappa}^{(l,\mathrm{lin})}, \bot_{\mathrm{lin}}) \coloneqq \ell(\varepsilon)$ and $Q_r(s_{j,q,\kappa}^{(l,\mathrm{lin})}, \mathrm{bias}) \coloneqq \ell([b_j^{(l)}]^\kappa)$ for $r = q$ and $-\infty$ for $r = q'$; they route the ε-fraction and the bias share of the mass to absorbing terminals instead of to any predecessor. If player q uses mixed action ν at this state, the induced *individual routed masses* are

$$\begin{aligned} \bar{Q}_r\big(s_{j,q,\kappa}^{(l,lin)}; \nu\big) \coloneqq \sum_a \nu(a)\, \ell\left(\exp\left(Q_r\big(s_{j,q,\kappa}^{(l,lin)}, a\big)\right) - \exp\left(Q_{r'}\big(s_{j,q,\kappa}^{(l,lin)}, a\big)\right)\right) \\ + \tau \mathscr{H}(\nu), \qquad r \in \{q, q'\}, \end{aligned} \tag{S.1.60}$$

which, as $\exp(-\infty) = 0$, evaluates to

$$\bar{Q}_q\big(s^{(l,lin)}_{j,q,\kappa};\nu\big) = \sum_a \nu(a)\, Q_q\big(s^{(l,lin)}_{j,q,\kappa}, a\big) + \tau\mathscr{H}(\nu) \tag{S.1.61}$$

$$\bar{Q}_{q'}\big(s^{(l,lin)}_{j,q,\kappa};\nu\big) = -\infty. \tag{S.1.62}$$

If $W^{(l,\kappa)}_{ij} = 0$ for every predecessor i and $[b^{(l)}_j]^\kappa = 0$ (i.e. the stream-κ part of the fan-in to neuron j is identically zero), then $Q_q(s^{(l,\mathrm{lin})}_{j,q,\kappa}, i) = -\infty$ for all i as well and the value (S.1.63) equals $\ell(\varepsilon)$. If $\varepsilon = 0$ and every action scores $-\infty$ (e.g. $\Sigma^{(l,\kappa)}_j = 0$ at $\tau = 1$), every ν is a best response; we fix $\nu = \delta_{\perp_{\mathrm{lin}}}$ (absorption), matching the convention $0/0 := 0$ in (S.1.85). We define the corresponding equilibrium linear-game values by

$$\begin{aligned} V^\star_q\big(s^{(l,lin)}_{j,q,\kappa}\big) &:= \max_{\nu\in\Delta(\mathrm{Pred}^{(l)}_j\cup\{\mathrm{bias},\perp_{\mathrm{lin}}\})} \bar{Q}_q\big(s^{(l,lin)}_{j,q,\kappa};\nu\big) \\ &= \max_{\nu\in\Delta(\mathrm{Pred}^{(l)}_j\cup\{\mathrm{bias},\perp_{\mathrm{lin}}\})} \left\{\sum_a \nu(a)\, Q_q\big(s^{(l,lin)}_{j,q,\kappa}, a\big) + \tau\mathscr{H}(\nu)\right\} \\ &= \tau\cdot\ell\left(\varepsilon^{\frac{1}{\tau}} + \big([b^{(l)}_j]^\kappa\big)^{\frac{1}{\tau}} + \sum_i \exp\Big(Q_q(s^{(l,lin)}_{j,q,\kappa}, i)\Big)^{\frac{1}{\tau}}\right), \end{aligned} \tag{S.1.63}$$

$$V^\star_{q'}\big(s^{(l,lin)}_{j,q,\kappa}\big) := -\infty, \tag{S.1.64}$$

by (S.1.56). At $\tau = 1$, the outside-option probability is $\varepsilon/(\varepsilon + \sum_{a\neq\perp_{\mathrm{lin}}} \exp Q_q(\cdots, a))$; the remaining mass is distributed among predecessors and the bias node proportionally to $\exp Q_q$.

At an **activation state**, write $s_+ := s^{(l,\mathrm{lin})}_{j,p,+}$ and $s_- := s^{(l,\mathrm{lin})}_{j,p',-}$. Continuing selects either branch with probability $1/2$; stopping gives $V_r(\dagger) = \ell(0) = -\infty$. Combining the uniform kernel with the log-space discount $\gamma_O = 2$ gives

$$\begin{aligned} Q_r\big(s^{(l,act)}_{j,p}, \mathrm{continue}\big) &:= \ell\left(\sum_{s'\in\{s_+,s_-\}} \mathbb{P}(s' \mid s^{(l,act)}_{j,p})\cdot\gamma_{\mathrm{O}}\cdot\Big(\exp V_r(s') - \exp V_{r'}(s')\Big)\right) \\ &= \ell\left(\xi_{r,p}\Big(\exp\left(V_p\left(s_+\right)\right) - \exp\left(V_{p'}\left(s_-\right)\right)\Big)\right). \end{aligned} \tag{S.1.65}$$

Stopping yields $-\infty$ for both players. We select *stop at every activation-state tie in log space*, so the selected policy and active-player value are

$$\pi^\star_p(\mathrm{continue} \mid s^{(l,\mathrm{act})}_{j,p}) := \mathbf{1}\Big[Q_p(s^{(l,\mathrm{act})}_{j,p},\mathrm{continue}) > -\infty\Big], \tag{S.1.66}$$

$$V_p(s^{(l,\mathrm{act})}_{j,p}) := \max\{-\infty, Q_p(s^{(l,\mathrm{act})}_{j,p},\mathrm{continue})\}. \tag{S.1.67}$$

For $D_p = \exp V_p(s_+) - \exp V_{p'}(s_-)$, continuation is selected exactly when $D_p > 0$; otherwise both log values equal $-\infty$ and stopping is selected. When continuation is selected, the opponent receives $-\infty$; both players do under stopping. Hence

$$V_{p'}(s^{(l,\mathrm{act})}_{j,p}) = -\infty. \tag{S.1.68}$$

Figure S.1.2 shows a worked example of the local routing subgame.

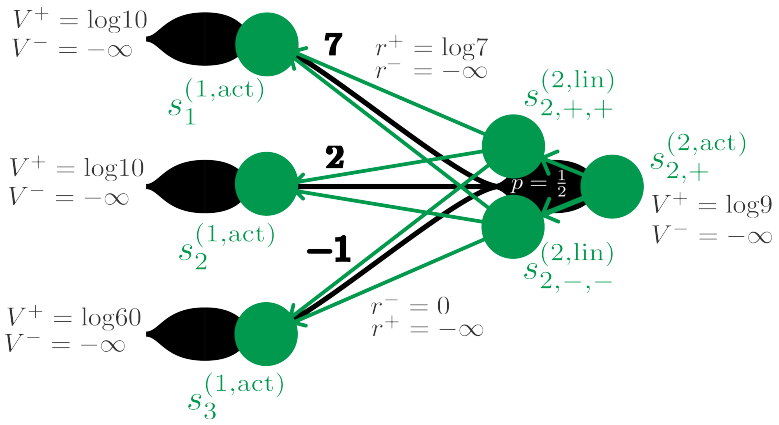


Fig. S.1.2: Routing Game: local routing subgame around $s_{2,+}^{(2,\mathrm{act})}$. This uses the same toy subnetwork and numerical values as Figure S.1.1. At this activation state, P^+ is in turn and continues because the routed mass is positive ($V^+ = \log 9 > -\infty$ in the depicted example). The game then branches with probability 1/2 to $s_{2,+,+}^{(2,\mathrm{lin})}$ (turn stays with P^+) and with probability 1/2 to $s_{2,-,-}^{(2,\mathrm{lin})}$ (turn switches to P^-). In each linear state, the active player samples a predecessor according to the Gibbs policy from the immediate log-payoff and game value. Example: along $s_{2,+,+}^{(2,\mathrm{lin})} \to s_{1,+}^{(1,\mathrm{act})}$, the player on the positive stream state gets immediate payoff $\log W_{12}^{(2,+)} = \log 7$ and P^- gets $-\infty$. Edges with zero routed mass have payoff $\log W_{12}^{(2,-)} = \log 0 = -\infty$. For readability, the $\pm$ activation copies in layer 1 are collapsed, and the intermediate payoff $\log 2$ on the transition from position 2 to 2 is omitted.

In the following let us denote, with $a^{(0)} = x$,

$$\Sigma_j^{(l,\kappa)} := \textstyle\sum_i [a_i^{(l-1)} W_{ij}^{(l)}]^\kappa + [b_j^{(l)}]^\kappa, \tag{S.1.69}$$

so that $z_j^{(l)} = \Sigma_j^{(l,+)} - \Sigma_j^{(l,-)}$; for $l \geq 2$, $a^{(l-1)} \geq 0$ gives $[a_i^{(l-1)} W_{ij}^{(l)}]^\kappa = a_i^{(l-1)} W_{ij}^{(l,\kappa)}$.

Theorem S.1.7 (SPNE and Forward Equivalence of the Routing Game, `Lean_63`).

(i) *SPNE existence for every $\tau > 0$ and $\varepsilon \geq 0$. For every $\tau > 0$ and every $\varepsilon \geq 0$, the entropy-regularized two-player Routing Gameof Definition S.1.7 (with linear-state entropy weight $\tau\mathscr{H}$ and outside-option stabilizer ε) admits a subgame-perfect Nash equilibrium (see (S.1.74)–(S.1.75) in the proof for the explicit form of the equilibrium policy). We write $\cdot^\star$ for equilibrium quantities throughout.*

(ii) *Forward equivalence at the anchor $\tau = 1$, $\varepsilon \geq 0$. At $\tau = 1$ and $\varepsilon \geq 0$, the equilibrium quantities recover the network forward pass: for every layer l, neuron j, player labels p,q,r, and stream sign $\kappa \in \{+,-\}$,*

$$V_r\big(s_{j,q,\kappa}^{(l,lin)}\big) = \ell\big(\delta_{r,q}\,(\Sigma_j^{(l,\kappa)} + \varepsilon)\big), \tag{S.1.70}$$

$$Q_r\left(s_{j,p}^{(l,act)}, \mathit{continue}\right) = \ell\left(\xi_{r,p} \cdot z_j^{(l)}\right), \tag{S.1.71}$$

$$V_r\big(s_{j,p}^{(l,act)}\big) = \ell\big(\delta_{r,p} \cdot a_j^{(l)}\big), \tag{S.1.72}$$

$$\pi_p^\star\big(s_{j,p}^{(l,act)}\big) = \mathbf{1}[z_j^{(l)} > 0]. \tag{S.1.73}$$

Remark S.1.2 (Uniqueness). When some routing action has a finite score, strict concavity makes the Gibbs best response (S.1.74) unique; otherwise Definition S.1.7 selects the absorbing $\perp_{\text{lin}}$. At activation states, nonpositive signed continuation mass $\exp V_p(s_+) - \exp V_{p'}(s_-)$ ($z_j^{(l)} \le 0$ at $\tau = 1$) makes both log values $-\infty$, permitting multiple best responses. The theorem selects $\pi_p^\star = \mathbf{1}[Q_p(\cdot,\text{continue}) > -\infty]$, reducing to $\mathbf{1}[z_j^{(l)} > 0]$ at $\tau = 1$. As in the ***Stopping Game*** (SG) (Section S.1.2), this is the **stop-at-tie-breaks** equilibrium. Its conventions resolve both activation log ties and empty routing streams.

Proof. **SPNE existence for every $\tau > 0$.** Use backward induction on the finite DAG: layer-l linear states lead to layer-$(l-1)$ activations (input terminals at $l = 1$), and layer-l activation states lead to same-layer linear states or †.

At layer $l = 1$ the linear routing subgames have no unresolved game values, so the action values Q_q in (S.1.59) are fully specified by the input x. At each linear state the active player solves $\max_{\nu \in \Delta(\text{Pred}_j^{(1)})} \{\sum_i \nu(i)\, Q_q(s_{j,q,\kappa}^{(1,lin)}, i) + \tau\, \mathscr{H}(\nu)\}$, which is strictly concave on the simplex face supported on finite-score actions whenever that face is nonempty. If every score is $-\infty$, every distribution is optimal and we select the absorbing outside action as above. Identity (S.1.56) yields the unique Gibbs best response in the nonempty finite-score case, and simplifying $\exp \circ \ell$ in (S.1.59) gives the explicit form

$$\pi_q^\star\big(i \,\big|\, s_{j,q,\kappa}^{(l,lin)}\big) \propto \begin{cases} \big(W_{ij}^{(l,\kappa)}\big)^{1/\tau} \exp\big(V_q(s_{i,q}^{(l-1,act)})/\tau\big), & l \ge 2, \\ \big([x_i W_{ij}^{(1)}]^\kappa\big)^{1/\tau}, & l = 1, \end{cases} \tag{S.1.74}$$

with zero mass on routes of zero or wrong sign and closed-form value

$$V_q(s_{j,q,\kappa}^{(1,lin)}) = \tau \log \sum_i \exp(Q_q(s_{j,q,\kappa}^{(1,lin)}, i)/\tau).$$

At $l = 1$, compare stopping's log-payoff $\ell(0) = -\infty$ with continuation (S.1.65). Stopping at every tie gives the pure best response

$$\pi_p^\star\big(s_{j,p}^{(l,act)}\big) = \mathbf{1}\big[Q_p(s_{j,p}^{(l,act)}, \text{continue}) > -\infty\big]. \tag{S.1.75}$$

Given equilibrium values $V_r(s_{i,q}^{(l-1,act)})$ through layer $l-1$, (S.1.59) specifies the layer-l action values. The same argument gives the Gibbs best response (S.1.74), its log-sum-exp value, and the activation best response (S.1.75). Each subgame optimizes against lower-layer values, so their composition is subgame-perfect. Adding the bias and outside actions of Definition S.1.7 preserves the argument: their constant scores $\ell([b_j^{(l)}]^\kappa)$ and $\log \varepsilon$ may be finite or $-\infty$, and the same strict-concavity/Legendre–Fenchel step applies to the augmented simplex. The log-sum-exp gains $([b_j^{(l)}]^\kappa)^{1/\tau}$ and $\varepsilon^{1/\tau}$, proving existence for every $(\tau,\varepsilon) \in \mathbb{R}_{>0} \times \mathbb{R}_{\ge 0}$.

Forward equivalence at $\tau = 1$. Induct on l.

Base case ($l = 1$): Identical to the inductive step below, with the fused-input action values $Q_q(s_{j,q,\kappa}^{(1,lin)}, i) = \ell([x_i W_{ij}^{(1)}]^\kappa)$ of (S.1.59) in place of $\ell(a_i^{(l-1)} W_{ij}^{(l,\kappa)})$.

Inductive step ($l \ge 2$): Assume $V_q(s_{i,q}^{(l-1,act)}) = \ell(a_i^{(l-1)})$ for all predecessors i and both player labels q. Since the assumed non-negative activation functions satisfy $a_i^{(l-1)} \ge 0$, the linear routing-state

action values become

$$Q_q\big(s^{(l,lin)}_{j,q,\kappa}, i\big) = \ell\big(W^{(l,\kappa)}_{ij}\big) + \ell\big(a^{(l-1)}_i\big) = \ell\big(a^{(l-1)}_i W^{(l,\kappa)}_{ij}\big). \tag{S.1.76}$$

Applying (S.1.56) to the linear routing subgame gives the Gibbs best response (S.1.74) and

$$V_q\big(s^{(l,lin)}_{j,q,\kappa}\big) = \ell\left(\sum_a \exp\big(Q_q(s^{(l,lin)}_{j,q,\kappa}, a)\big)\right) = \ell\left(\sum_i a^{(l-1)}_i W^{(l,\kappa)}_{ij} + [b^{(l)}_j]^\kappa + \varepsilon\right) = \ell(\Sigma^{(l,\kappa)}_j + \varepsilon), \tag{S.1.77}$$

At the activation state the active player's continuation advantage is again $z^{(l)}_j$

$$Q_p\big(s^{(l,act)}_{j,p}, \text{continue}\big) = \ell\left(\Sigma^{(l,+)}_j - \Sigma^{(l,-)}_j\right) = \ell(z^{(l)}_j), \tag{S.1.78}$$

whereas for the opponent

$$Q_{p'}\left(s^{(l,\text{act})}_{j,p}, \text{continue}\right) = \ell\left(\Sigma^{(l,-)}_j - \Sigma^{(l,+)}_j\right) = \ell(-z^{(l)}_j), \tag{S.1.79}$$

again confirming (S.1.71). Thus, the best response is (S.1.75). Finally,

$$V_r\big(s^{(l,act)}_{j,p}\big) = \ell\big(\delta_{r,p}\,\mathbf{1}[z^{(l)}_j > 0]\, z^{(l)}_j\big) = \ell\big(\delta_{r,p}\, a^{(l)}_j\big), \tag{S.1.80}$$

This proves (S.1.72) and completes the induction; the player in turn chooses optimally in every subgame, so the profile is subgame-perfect. □

Log-Space Addition Subgame, Skip Connections, and Max Pooling

Residual additions $z = x + y$ and max-pooling nodes $z = \max_k x_k$ enter the ***R**outing **G**ame* (RG) through the addition state $s^{\text{add}}_{z,p}$ (uniform Oracle to the operand states, player p retained, $\gamma_{\text{O}} = 2$) and the pooling state $s^{\max}_{z,p}$ (deterministic choice of an operand, player p retained, $\gamma_{\text{O}} = 1$, ties broken by the smallest index) of Section S.1.2.3, with the values written in log space:

$$V_r(s^{\text{add}}_{z,p}) = \ell\big(\exp V_r(s_{x,p}) + \exp V_r(s_{y,p})\big), \qquad V_p(s^{\max}_{z,p}) = \max_k V_p(s_{x_k,p}). \tag{S.1.81}$$

If the operand states recover their (non-negative) activations, $V_p(s_{x,p}) = \ell(a_x)$ etc., then $V_p(s^{\text{add}}_{z,p}) = \ell(a_x + a_y) = \ell(a_z)$ and $V_p(s^{\max}_{z,p}) = \ell(a_{x_{k^\star}}) = \ell(a_z)$, so forward equivalence extends across both nodes. Two things differ from the SG. First, $\gamma_{\text{O}} = 2$ enters only the values: the trajectory discount (S.1.83) of the addition transition is 1, so each operand receives half of the addition-state occupation (Theorem S.1.8, item 2). Second, the relevance split therefore equals the Oracle probabilities: a $(\rho, 1-\rho)$ Oracle with value discounts $(1/\rho, 1/(1-\rho))$ leaves all forward values unchanged but splits the relevance as $(\rho, 1-\rho)$, so the uniform split is a convention of the RG rule. The implementation passes the full relevance to both operands (trajectory discount 2); since every backward trajectory of a ResNet or ***Vi**sion **T**ransformer* (ViT) that reaches the input crosses each of its B addition nodes exactly once, this rescales the input occupation by the constant 2^B and leaves normalized maps unchanged.

S.1.3.2 Backward Path Integral and $\alpha\beta$-LRP Recovery

The SPNE's projected discounted occupation measure recovers $\alpha\beta$-LRP-ε relevance of (4.1). We distinguish the two objects:

– the *game-side* object is the discounted occupation measure $\Gamma_x(\,\cdot\,)$ on the states of the routing graph (Definition S.1.6), now driven by the SPNE trajectory law Π_x rooted at $s_{u,+}^{(L,\mathrm{act})}$ with unit initial mass: $\Gamma_x(s_{u,+}^{(L,\mathrm{act})}) = 1$ and $\Gamma_x(s_{j,p}^{(L,\mathrm{act})}) = 0$ for every other output activation state. With non-negative branch discounts (cf. (S.1.83)) Γ_x is itself non-negative; we then form the per-neuron difference of the two player copies,

$$\Gamma_j^{(l)} \;\coloneqq\; \Gamma_x\big(s_{j,+}^{(l,\mathrm{act})}\big) \;-\; \Gamma_x\big(s_{j,-}^{(l,\mathrm{act})}\big). \tag{S.1.82}$$

– the *attribution-side* object is the $\alpha\beta$-LRP-ε relevance vector $R^{(l)}$ (the $\alpha\beta$ rule of Bach et al. (2015) with the stabilization convention adopted here, see below), a deterministic backward DP on the network's linear layers seeded by a chosen output mass $R_u^{(L)}$ and recursing via (S.1.85) below.

Under the SPNE and discount (S.1.83), Theorem S.1.8 identifies the recursions of $\Gamma_j^{(l)}$ and $R_j^{(l)}$, giving $\Gamma_j^{(l)} = R_j^{(l)}/R_u^{(L)}$.

Trajectory discount. We track sign-dependent branch discounts governed by the stream-balance mode (α,β) $(\alpha-\beta=1)$. Let $\zeta = (\zeta_0,\zeta_1,\dots)$ be a backward trajectory on linear routing, activation, addition or max-pooling states. Define the per-state discount γ as a non-negative function of the entered state,

$$\gamma(s) \coloneqq \begin{cases} 2\alpha, & s = s_{j,q,+}^{(l,lin)} \text{ for some } j,q,l, \\ 2\beta, & s = s_{j,q,-}^{(l,lin)} \text{ for some } j,q,l, \\ 1, & \text{otherwise (activation, addition Oracle, max-pool).} \end{cases} \tag{S.1.83}$$

The cumulative discount along the trajectory up to time t is the product

$$d_t(\zeta) \coloneqq \prod_{m=1}^{t} \gamma(\zeta_m), \qquad d_0(\zeta) = 1 \text{ (empty product).} \tag{S.1.84}$$

The signed minus of the $\alpha\beta$-LRP recursion is produced by the difference (S.1.82) between the two player copies.

Remark S.1.3 (Choice of starting player). Rooting Π_x at $s_{u,-}^{(L,\mathrm{act})}$ instead of $s_{u,+}^{(L,\mathrm{act})}$ swaps the two output-layer occupations and therefore negates $\Gamma_j^{(l)}$ at every layer. We fix the canonical convention that the game starts at the +-state.

Remark S.1.4 (Equivalent signed-discount sum formulation). The same recovery theorem can be phrased with the signed trajectory discount $\gamma(s_{j,q,+}^{(l,\mathrm{lin})}) = 2\alpha$, $\gamma(s_{j,q,-}^{(l,\mathrm{lin})}) = -2\beta$ and the per-neuron *sum* $\Gamma_j^{(l)} = \Gamma_x(s_{j,+}^{(l,\mathrm{act})}) + \Gamma_x(s_{j,-}^{(l,\mathrm{act})})$ in place of (S.1.82). Our non-negative discounts keep Γ_x a non-negative occupation measure and express the LRP sign through the explicit difference rather than the trajectory weights.

Attribution-side: $\alpha\beta$-LRP-ε relevance. Independently of the game, the $\alpha\beta$-LRP-ε relevance vector $R^{(l)}$ is defined recursively from $R_j^{(L)} = R_u^{(L)}\,\delta_{j,u}$ at the output by

$$R_i^{(l-1)} \;=\; \sum_j \left(\alpha \frac{[a_i^{(l-1)} W_{ij}^{(l)}]^+}{\varepsilon + \Sigma_j^{(l,+)}} - \beta \frac{[a_i^{(l-1)} W_{ij}^{(l)}]^-}{\varepsilon + \Sigma_j^{(l,-)}} \right) R_j^{(l)}, \tag{S.1.85}$$

where $\Sigma_j^{(l,\pm)}$ is (S.1.69): the bias enters the denominators as a terminal bias node, and the relevance routed to it (and to the outside option) is absorbed, as in the implementation. Recursion (S.1.85) is the $\alpha\beta$ rule of Bach et al. (2015) with the stabilization convention adopted throughout this paper: the stabilizer ε of their LRP-ε rule is added separately to each sign denominator. Bach et al. (2015) present the two rules as alternative constructions and do not state this combined form; it is the convention of common LRP implementations (Anders et al., 2026), and Theorem S.1.8 is exact for this form. At $\varepsilon = 0$ it is the original $\alpha\beta$ rule.

Theorem S.1.8 (Recovery of Multi-Path $\alpha\beta$-LRP, `Lean_64`). *For every $\varepsilon \geq 0$ and every input with $f(x) = a_u^{(L)} > 0$ (if $f(x) = 0$ the root player stops and $R^{(l)} = f(x)\Gamma^{(l)} = 0$ trivially), under the stop-at-tie-breaks equilibrium (Remark S.1.2; ties at max-pooling states broken by the smallest index) at the anchor temperature $\tau = 1$ and stabilizer ε, with the cumulative trajectory discount (S.1.84) and the occupation measure $\Gamma_j^{(l)}$ from (S.1.82):*

1) ***Linear Map.*** *The per-neuron difference $\Gamma_j^{(l)}$ of (S.1.82) satisfies*

$$\Gamma_i^{(l-1)} \;=\; \sum_j \left(\alpha \frac{[a_i^{(l-1)} W_{ij}^{(l)}]^+}{\varepsilon + \Sigma_j^{(l,+)}} - \beta \frac{[a_i^{(l-1)} W_{ij}^{(l)}]^-}{\varepsilon + \Sigma_j^{(l,-)}} \right) \Gamma_j^{(l)}. \tag{S.1.86}$$

   Because (S.1.86) and (S.1.85) share their computational step, $\Gamma_j^{(l)} \cdot R_u^{(L)} = R_j^{(l)}$ at every layer; in particular $R_j^{(l)} = f(x)\,\Gamma_j^{(l)}$ for the seed $R_u^{(L)} = f(x)$. The same formula covers the fused input layer with $a^{(0)} = x$. At $\varepsilon = 0$ this reduces to the unstabilized $\alpha\beta$-LRP redistribution (not conservative: bias nodes absorb relevance); at $\varepsilon > 0$ it is the $\alpha\beta$ rule with the stabilization convention adopted here, i.e. (4.1) with ε added to each sign denominator separately.

2) ***Residual Addition.*** *Let $z = x_{\mathrm{op}} + y_{\mathrm{op}}$ be an addition node with addition state $s_{z,p}^{\mathrm{add}}$ and operand states $s_{x_{\mathrm{op}},p}, s_{y_{\mathrm{op}},p}$ as in Section S.1.3.1. For every player label $p \in \{+,-\}$, with the edge occupation $\Gamma_x(s \to s') := \mathbb{E}_{\zeta\sim\Pi_x}\big[\sum_t d_{t+1}(\zeta)\mathbf{1}\{\zeta_t = s, \zeta_{t+1} = s'\}\big]$ (an operand's total occupation adds the edges from its other parents),*

$$\Gamma_x\big(s_{z,p}^{\mathrm{add}} \to s_{x_{\mathrm{op}},p}\big) \;=\; \tfrac{1}{2}\,\Gamma_x\big(s_{z,p}^{\mathrm{add}}\big), \qquad \Gamma_x\big(s_{z,p}^{\mathrm{add}} \to s_{y_{\mathrm{op}},p}\big) \;=\; \tfrac{1}{2}\,\Gamma_x\big(s_{z,p}^{\mathrm{add}}\big), \tag{S.1.87}$$

   so the operand pair carries the full addition-state mass with no duplication.

3) ***Max Pooling.*** *For a pooled output $z = \max\{x_1, \ldots, x_m\}$ with winner $k^\star$ and pooling state $s_{z,p}^{\max}$ (Section S.1.3.1), the deterministic value-maximizing transition concentrates all mass on the winner: for every player label $p \in \{+,-\}$,*

$$\Gamma_x\big(s_{z,p}^{\max} \to s_{x_{k^\star},p}\big) \;=\; \Gamma_x\big(s_{z,p}^{\max}\big), \qquad \Gamma_x\big(s_{z,p}^{\max} \to s_{x_k,p}\big) \;=\; 0 \text{ for } k \neq k^\star. \tag{S.1.88}$$

Proof. Induct backward on l to prove $R_u^{(L)} \cdot \Gamma_j^{(l)} = R_j^{(l)}$.

Base case ($l = L$): The trajectory law Π_x has unit mass at $s_{u,+}^{(L,\mathrm{act})}$ by definition, so $\Gamma_x(s_{u,+}^{(L,\mathrm{act})}) = 1$ and $\Gamma_x(s_{j,p}^{(L,\mathrm{act})}) = 0$ for every $(j,p) \neq (u,+)$. By (S.1.82),

$$\Gamma_j^{(L)} \;=\; \Gamma_x\big(s_{j,+}^{(L,\mathrm{act})}\big) - \Gamma_x\big(s_{j,-}^{(L,\mathrm{act})}\big) \;=\; \delta_{j,u}, \tag{S.1.89}$$

Multiplying by the output mass gives $R_u^{(L)} \cdot \Gamma_j^{(L)} = R_u^{(L)} \delta_{j,u} = R_j^{(L)}$, the root specification of (S.1.85).

Inductive step ($l \Rightarrow l-1$): Assume $R_u^{(L)} \cdot \Gamma_j^{(l)} = R_j^{(l)}$ for every neuron j in layer l. From the SPNE construction in Theorem S.1.7 together with the activation-game value reduction $V_p(s_{j,p}^{(l,\mathrm{act})}) = \ell(a_j^{(l)})$ (which holds for every $\varepsilon \geq 0$ because the additive ε-shifts on the two sign branches cancel in the continuation advantage), the linear-state Gibbs best response on the alive support is

$$\pi^\star_{j,\kappa}(\perp_{\mathrm{lin}}) = \frac{\varepsilon}{\varepsilon + \Sigma_j^{(l,\kappa)}}, \qquad \pi^\star_{j,\kappa}(\mathrm{bias}) = \frac{[b_j^{(l)}]^\kappa}{\varepsilon + \Sigma_j^{(l,\kappa)}}, \qquad \pi^\star_{j,\kappa}(i) = \frac{a_i^{(l-1)} W_{ij}^{(l,\kappa)}}{\varepsilon + \Sigma_j^{(l,\kappa)}}. \tag{S.1.90}$$

A trajectory reaches the layer-$(l-1)$ activation state $s_{i,r}^{(l-1,\mathrm{act})}$ from a layer-l activation state through one of two routes. By the activation-state continuation kernel of Definition S.1.7, departing from $s_{j,p}^{(l,\mathrm{act})}$ (the player continues: at $l = L$ since $f(x) > 0$; at $l < L$, $\Gamma_x(s_{j,\cdot}^{(l,\mathrm{act})}) = 0$ unless $a_j^{(l)} > 0$ by (S.1.90)) the trajectory takes the $+$ branch to $s_{j,p,+}^{(l,\mathrm{lin})}$ with probability $\frac{1}{2}$ (player p retains the turn), and the $-$ branch to $s_{j,p',-}^{(l,\mathrm{lin})}$ with probability $\frac{1}{2}$ (turn switches to opponent p'); from each linear state the policy (S.1.90) routes to predecessor i with the player label of that linear state. Combining the two branch discount factors $2\alpha, 2\beta$ from (S.1.83) with these $\frac{1}{2}$-probabilities, the trajectory contributions to $\Gamma_x(s_{i,r}^{(l-1,\mathrm{act})})$ are

$$\begin{aligned}
\Gamma_x\big(s_{i,r}^{(l-1,\mathrm{act})}\big) &= \sum_j \Big(\tfrac{1}{2}(2\alpha)\, \pi^\star_{j,+}(i)\, \Gamma_x\big(s_{j,r}^{(l,\mathrm{act})}\big) \;+\; \tfrac{1}{2}(2\beta)\, \pi^\star_{j,-}(i)\, \Gamma_x\big(s_{j,r'}^{(l,\mathrm{act})}\big) \Big) \\
&= \sum_j \Big(\alpha\, \pi^\star_{j,+}(i)\, \Gamma_x\big(s_{j,r}^{(l,\mathrm{act})}\big) \;+\; \beta\, \pi^\star_{j,-}(i)\, \Gamma_x\big(s_{j,r'}^{(l,\mathrm{act})}\big) \Big),
\end{aligned} \tag{S.1.91}$$

where $r' \in \{+,-\}$ denotes the opposite player label and the absorbing actions bias and $\perp_{\mathrm{lin}}$ contribute no mass to any predecessor.

Subtracting the $r = -$ case of (S.1.91) from the $r = +$ case produces $+\Gamma_j^{(l)}$ in the positive branch and $-\Gamma_j^{(l)}$ in the negative branch:

$$\begin{aligned}
\Gamma_i^{(l-1)} &= \Gamma_x\big(s_{i,+}^{(l-1,\mathrm{act})}\big) - \Gamma_x\big(s_{i,-}^{(l-1,\mathrm{act})}\big) \\
&= \sum_j \Big(\alpha\, \pi^\star_{j,+}(i) \left[\Gamma_x(s_{j,+}^{(l,\mathrm{act})}) - \Gamma_x(s_{j,-}^{(l,\mathrm{act})}) \right] \;+\; \beta\, \pi^\star_{j,-}(i) \left[\Gamma_x(s_{j,-}^{(l,\mathrm{act})}) - \Gamma_x(s_{j,+}^{(l,\mathrm{act})}) \right] \Big) \\
&= \sum_j \big(\alpha\, \pi^\star_{j,+}(i) - \beta\, \pi^\star_{j,-}(i) \big)\, \Gamma_j^{(l)} \\
&= \sum_j \left(\alpha \frac{[a_i^{(l-1)} W_{ij}^{(l)}]^+}{\varepsilon + \Sigma_j^{(l,+)}} - \beta \frac{[a_i^{(l-1)} W_{ij}^{(l)}]^-}{\varepsilon + \Sigma_j^{(l,-)}} \right) \Gamma_j^{(l)},
\end{aligned} \tag{S.1.92}$$

where the third equality regroups the two square brackets as $+\Gamma_j^{(l)}$ and $-\Gamma_j^{(l)}$ respectively, and the fourth substitutes (S.1.90) together with $a_i^{(l-1)}W_{ij}^{(l,\kappa)} = [a_i^{(l-1)}W_{ij}^{(l)}]^\kappa$ for non-negative $a_i^{(l-1)}$. Multiplying both sides by $R_u^{(L)}$ and using the inductive hypothesis $R_u^{(L)} \cdot \Gamma_j^{(l)} = R_j^{(l)}$,

$$R_u^{(L)} \cdot \Gamma_i^{(l-1)} \;=\; \sum_j \left(\alpha \frac{[a_i^{(l-1)}W_{ij}^{(l)}]^+}{\varepsilon + \Sigma_j^{(l,+)}} - \beta \frac{[a_i^{(l-1)}W_{ij}^{(l)}]^-}{\varepsilon + \Sigma_j^{(l,-)}} \right) R_j^{(l)} \;=\; R_i^{(l-1)}. \tag{S.1.93}$$

This closes the induction for every layer and neuron.

Input layer ($l = 1$): The same calculation applies with the fused first-layer masses $[x_i W_{ij}^{(1)}]^\kappa$ in place of $a_i^{(l-1)}W_{ij}^{(l,\kappa)}$, yielding the identical formula with $a^{(0)} = x$.

Addition node: Let $z = x_{\text{op}} + y_{\text{op}}$ be an addition node. By Section S.1.3.1, from $s_{z,p}^{\text{add}}$ the Oracle transitions to $s_{x_{\text{op}},p}$ and $s_{y_{\text{op}},p}$ with probability $\frac{1}{2}$ each, retaining player label p. Under the trajectory-discount convention (S.1.83)–(S.1.84), addition Oracle transitions carry trajectory-discount factor 1. Hence

$$\Gamma_x\big(s_{z,p}^{\text{add}} \to s_{x_{\text{op}},p}\big) \;=\; \tfrac{1}{2}\,\Gamma_x\big(s_{z,p}^{\text{add}}\big), \qquad \Gamma_x\big(s_{z,p}^{\text{add}} \to s_{y_{\text{op}},p}\big) \;=\; \tfrac{1}{2}\,\Gamma_x\big(s_{z,p}^{\text{add}}\big), \tag{S.1.94}$$

proving Item 2 of Theorem S.1.8 for both player labels.

Max-pooling node: Let $z = \max\{x_1, \ldots, x_m\}$ with winner $k^\star$ and pooling state $s_{z,p}^{\max}$. By Section S.1.3.1, the active player has the unique value-maximizing action $x_{k^\star}$ at $s_{z,p}^{\max}$, so the transition to $s_{x_{k^\star},p}$ is deterministic, retains player label p, and (S.1.83) assigns unit trajectory factor. Therefore $\Gamma_x(s_{x_{k^\star},p}) = \Gamma_x(s_{z,p}^{\max})$ and $\Gamma_x(s_{x_k,p}) = 0$ for $k \neq k^\star$, proving Item 3. □

S.1.3.3 Temperature τ as a Controlled Policy Deformation

Using $\tau > 0$ throughout the backward recursion gives an SPNE (Theorem S.1.7(i)); exact $\alpha\beta$-LRP recovery requires $\tau = 1$ (Theorem S.1.8). In practice, we deform only each linear subgame's *local* Gibbs readout,

$$\pi_{j,\kappa}^{(\tau)\star}(i) \propto \exp\big(Q_q(s_{j,q,\kappa}^{(l,lin)}, i)/\tau\big), \tag{S.1.95}$$

keeping $V(s_{i,q}^{(l-1,act)})$ at the $\tau = 1$ equilibrium, the network activations of Theorem S.1.7(ii). This *myopic* policy leaves the players' Bellman values at temperature 1 and changes only local routing. The players remain unaware of subsequent temperature changes when evaluating continuation values for their current decisions. Under non-negative predecessor activations this gives $\pi_{j,\kappa}^{(\tau)\star}(i) \propto (a_i^{(l-1)}W_{ij}^{(l,\kappa)})^{1/\tau}$, so $\tau < 1$ sharpens and $\tau > 1$ flattens the route distribution. This controlled deformation is used in the experiments; it carries no exactness claim at $\tau \neq 1$. The implementation raises activations, weights and bias-node masses to the power $1/\tau$ but keeps the outside-option weight at ε rather than $\varepsilon^{1/\tau}$. On ResNet-50 it passes relevance through the global average pool uniformly (not in proportion to the activations) and merges batch normalization into the preceding convolution with an ε-stabilized rule, so the ResNet-50 anchor is $\alpha\beta$-LRP up to these two operations; VGG-16 (whose average pool is the identity at 224×224) and ViT-B/16 are not affected.

S.1.4 Analytical Modes

We derive four analytical modes beyond the $\alpha\beta$-LRP anchor: temperature τ in the linear-state Gibbs readout (Section S.1.3.3), activation-side risk aversion λ via *Assumed Density Filtering* (ADF) variance propagation, the noisy-observation stop/continue gate $\tilde{\Phi}$, and fan-in entropy bias λ_{ent}. Each translates a game concept (focus, risk aversion) into a backward-rule modification, without an exactness claim, and recovers the anchor at its default ($\tau = 1$, $\lambda = 0$, $\tilde{\Phi}$ off, $\lambda_{\text{ent}} = 0$). The mode-by-mode empirical sweeps appear in Section S.4.4.3.

S.1.4.1 Defaults, costs and order of use

Table S.1.2 lists each mode's default, effect and measured cost (median seconds per image, one A100 GPU, batch size 1; Section S.4.3.3). At the defaults, SG is the gradient and RG is $\alpha\beta$-LRP. A practical order of use: (i) start at the anchor; (ii) vary τ first, sharpening or diffusing routing at negligible cost (Table S.4.17); (iii) add risk aversion (λ on ***C**onvolutional **N**eural **N**etwork**s*** (CNNs), λ_{sm} on ViTs, with the input variance σ^2) when the maps should discount high-variance units, costing one ADF pass; (iv) use equivariant averaging on ViTs for token-grid artifacts, multiplying cost by N. Section S.4.4.1 reports the selected combination per model and criterion.

Tab. S.1.2: Modes: default (anchor) value, effect and measured cost per image.

Mode	Default	Effect	Cost (s/image)
(α,β), ε	(2,1), 0.5	LRP stream weights and stabilizer (exact)	none
τ	1	sharper ($\tau < 1$) or more diffuse ($\tau > 1$) routing	none (VGG-16 0.035 $\rightarrow$ 0.037, ResNet-50 0.089 $\rightarrow$ 0.094)
λ, σ^2	λ=0	attenuates routes through high-variance units	one ADF pass (ResNet-50 0.089 $\rightarrow$ 0.140)
λ_{sm}, σ^2	0	downweights high-variance keys in attention routing	one ADF pass (ViT-B/16 0.097 $\rightarrow$ 0.172)
$\tilde{\Phi}$	off	smooth stop/continue gate	needs the ADF σ (one ADF pass)
N	off	averages over input transformations (token-grid artifacts)	$\times N$ (ViT-B/16, N=50: 4.93; with λ_{sm}: 8.66)

S.1.4.2 Activation-Side Risk Aversion λ via ADF Variance Propagation

Activation-side risk aversion replaces the observed activation $a_i^{(l)} = \text{ReLU}(z_i^{(l)})$ by the mean–variance confidence bound $V_\lambda = \mu + \lambda\sigma$ ($\lambda \leq 0$), computed by an ADF variance propagation pass (Gast and Roth, 2018). We propagate the exact mean $\mathbb{E}[\text{ReLU}(X)]$ for $X \sim \mathcal{N}(\mu,\sigma^2)$ (Gast and Roth, 2018); our implementation approximates $\text{Var}[\text{ReLU}(X)]$ by $\sigma^2\Phi(\mu/\sigma)$.

Every input neuron starts with variance σ^2. Negative λ lowers the observed game value at stopping layers only, attenuating routes through high-variance neurons, analogously to confidence-bound exploration in bandit / RL settings (Auer et al., 2002; Fei et al., 2020). The softmax-shift variant λ_{sm} on the attention-logit oracle (ViT) is treated separately in Section S.2.5.

S.1.4.3 Noisy-Observation Gate $\tilde{\Phi}$

Noisy observation replaces each activation state's pure stop/continue policy by a *mixed* policy. At $s_{i,q}^{(l,\mathrm{act})}$, continuation has probability

$$\tilde{\Phi}(z_i^{(l)}) \;\coloneqq\; \Phi\big(\mu_i^{(l)}/\sigma_i^{(l)} + \lambda\big) \tag{S.1.96}$$

and stopping has probability $1 - \tilde{\Phi}(z_i^{(l)})$, where Φ is the standard normal *Cumulative Distribution Function* (CDF) and $\mu_i^{(l)}$, $\sigma_i^{(l)}$ are the ADF mean and standard deviation of $z_i^{(l)}$ (Section S.1.4.2). The analysis below takes $\mu_i^{(l)} = z_i^{(l)}$, $\sigma_i^{(l)} = 1$ and $\lambda = 0$; the general case follows with $z \mapsto \mu + \lambda\sigma$ and $\mathscr{R}_\Phi \mapsto \sigma\mathscr{R}_\Phi$. The mixed policy (S.1.96) uniquely solves a regularized maximization with probit regularizer

$$\mathscr{R}_\Phi(\pi) \;\coloneqq\; \phi\big(\Phi^{-1}(\pi)\big), \qquad \pi \in [0,1], \tag{S.1.97}$$

where $\phi(y) = (2\pi)^{-1/2}\, e^{-y^2/2}$ is the standard normal probability density and Φ^{-1} is the standard normal quantile function.

Lemma S.1.9 (Probit dual of the soft gate). *For every $z \in \mathbb{R}$, the regularized maximization*

$$\max_{\pi\in[0,1]} \big\{\, \pi z \;+\; \mathscr{R}_\Phi(\pi) \,\big\} \tag{S.1.98}$$

admits the unique maximizer $\pi^\star = \Phi(z)$ and optimum value

$$z\,\Phi(z) + \phi(z) \;=\; \mathbb{E}_{\eta\sim\mathcal{N}(0,1)}\big[\max(z+\eta, 0)\big]. \tag{S.1.99}$$

Proof. Using $\phi'(y) = -y\,\phi(y)$ and the inverse-function derivative $(\Phi^{-1})'(\pi) = 1/\phi(\Phi^{-1}(\pi))$,

$$\frac{\mathrm{d}\mathscr{R}_\Phi}{\mathrm{d}\pi} \;=\; \phi'\big(\Phi^{-1}(\pi)\big)\cdot\frac{1}{\phi(\Phi^{-1}(\pi))} \;=\; -\Phi^{-1}(\pi). \tag{S.1.100}$$

The first-order condition $z - \Phi^{-1}(\pi) = 0$ yields $\pi^\star = \Phi(z)$; substitution with $\phi(\Phi^{-1}(\Phi(z))) = \phi(z)$ gives the left-hand side of (S.1.99). The right-hand side follows from Gaussian smoothing: $\mathbb{E}[\max(z+\eta,0)] = z\Phi(z) + \phi(z)$. □

Remark S.1.5 (Optimal policy under perceived payoff noise). Equivalently, $\pi^\star = \Phi(z_i^{(l)})$ marginalizes an optimal pure rule: the player perceives continuation payoff $z_i^{(l)} + \eta$ at $s_{i,q}^{(l)}$, with $\eta \sim \mathcal{N}(0,1)$ independent across states, and follows *"continue iff the perceived payoff is strictly positive"*. Marginalizing over η at fixed $z_i^{(l)}$ gives

$$\mathbb{P}_{\eta\sim\mathcal{N}(0,1)}\big(z_i^{(l)} + \eta > 0\big) \;=\; \Phi(z_i^{(l)}). \tag{S.1.101}$$

Only perception is noisy: continuation pays the deterministic forward value $z_i^{(l)}$. Thus the expected backward Bellman comparison at $s_{i,q}^{(l)}$ uses $\Phi(z_i^{(l)}) \cdot z_i^{(l)}$ in place of $\mathbf{1}[z_i^{(l)} > 0] \cdot z_i^{(l)}$.

The regularizer $\mathscr{R}_\Phi$ is symmetric around $\pi = \frac{1}{2}$ and peaks there at $\phi(0) = 1/\sqrt{2\pi}$ (where $\Phi^{-1}(\frac{1}{2}) = 0$), tapering to 0 at the boundary $\pi \in \{0,1\}$. It rewards *indecisiveness*, as does Shannon entropy $\mathscr{H}(\pi)$ in the Softplus variant of §S.1.5, where the entropy bonus rewards the active player's mixing. The

optimum $\pi^\star = \Phi(z) = \mathbb{E}_{\eta\sim\mathcal{N}(0,1)}[\,\mathbf{1}(z+\eta>0)\,]$ is the hard ReLU gate averaged over a Gaussian shift of its threshold, aligning with the original stochastic-gate motivation for GELU (Hendrycks and Gimpel, 2016). The optimal value (S.1.99) is the Gaussian-smoothed ReLU $z\Phi(z)+\phi(z)$, whose derivative is Φ; it is not $\mathrm{GELU}(z) = z\Phi(z)$, which is not convex and hence not the conjugate of any regularizer.

On ReLU networks, the exact forward activations $\mathrm{ReLU}(z_i^{(l)})$ and downstream game values of Theorem S.1.3 remain: $\tilde{\Phi}$ applies to the *current* layer, using original activations below. This optional smoothing mode, used in the VGG-16 SG row, carries no exactness claim.

GELU networks. On a GELU network, $\tilde{\Phi}$ multiplies the occupation-measure recursion by $\Phi(z_i^{(l)})$ at every layer, which is the chain rule of the gate-conditioned map $a_i^{(l)} = \Phi(\bar{z}_i^{(l)})\, z_i^{(l)}$ with the gate argument $\bar{z}_i^{(l)}$ held at its forward value, the conditioning used for ViTs (Section 5.5.3). It is not the GELU gradient, which needs $\mathrm{GELU}'(z) = \Phi(z) + z\phi(z)$; the substitution $\mathbf{1}[z>0] \to \Phi(z)$ is exact only for the Gaussian-smoothed-ReLU network, as the entropy-regularized gate is for Softplus (Section S.1.5).

S.1.4.4 Fan-in entropy bias λ_{ent}

Fan-in entropy lowers the bias—the continuation payoff (SG) or bias-node payoff (RG)—of neurons with diffuse fan-in: $b_j^{(l)} \mapsto b_j^{(l)} - \lambda_{\mathrm{ent}}\tilde{\mathscr{H}}_j^{(l)}$ with $\tilde{\mathscr{H}}_j^{(l)} = -\sum_i p_{ij}\log p_{ij}/\log n_j$, $p_{ij} = |W_{ij}^{(l)}|/\gamma_j^{(l)}$ and n_j the fan-in of neuron j. It depends on the weights only (negligible cost) and changes the forward pass of the explained network.

S.1.5 Softplus Variant of the Stopping Game

We adapt the activation-state entropy regularization of Gaubert & Vlassopoulos (2025) for Softplus networks to the Stopping Decomposition (Section S.1.2.1): player-specific values equal Softplus-Decomposition activations, and a linear combination of occupation measures recovers the network gradient, as for ReLU.

Modified rule: entropy-regularized activation-state policy. We use the Legendre–Fenchel identity for Shannon entropy, also used in the Routing Game (Section S.1.3) and attention (Section 5.5): for every $\theta>0$, $n\in\mathbb{N}$, and payoff vector $r\in\mathbb{R}^n$,

$$\max_{\pi\in\Delta_n}\left\{\sum_{i=1}^n \pi(i)\,r_i + \theta\,\mathscr{H}(\pi)\right\} = \theta\,\log\sum_{i=1}^n \exp\!\left(\frac{r_i}{\theta}\right), \qquad \pi^\star(i) \propto \exp\!\left(\frac{r_i}{\theta}\right), \tag{S.1.102}$$

with $\mathscr{H}(\pi) \coloneqq -\sum_i \pi(i)\log\pi(i)$; the unique maximizer is the Gibbs measure at inverse temperature $1/\theta$. Identity (S.1.102) is the Fenchel conjugate of the (scaled) negative entropy (Boyd and Vandenberghe, 2004).

Only the active player's stop/continue choice changes. At every activation state $s = s_{j,q}^{(l,\mathrm{act})}$, replace the binary maximization (S.1.25) by an entropy-regularized maximization over mixed policies $\pi_q(s) \in$

$\Delta(\{\text{stop,cont}\})$ at temperature θ. With $\tilde{Q}_q^\pi(s,\text{stop}) = \tilde{Q}_{q'}^\pi(s,\text{stop}) = 0$ (cf. (S.1.21)) and $\tilde{Q}_p^\pi(s,\text{cont})$ as in (S.1.22), the soft Bellman recursion for the active-player advantage is

$$V_q^\pi(s) \coloneqq \max_{\nu\in\Delta(\{\text{stop,cont}\})} \Big\{ \textstyle\sum_a \nu(a)\big(\tilde{Q}_q^\pi(s,a) - \tilde{Q}_{q'}^\pi(s,a)\big) + \theta\,\mathscr{H}(\nu) \Big\}. \tag{S.1.103}$$

Applying (S.1.102) to the binary action set {stop,cont} with payoffs 0 and $\tilde{Q}_q^\pi(s,\text{cont}) - \tilde{Q}_{q'}^\pi(s,\text{cont})$ yields the unique Gibbs optimizer at inverse temperature $1/\theta$ and soft advantage,

$$\pi_q^\star\big(\text{cont} \,\big|\, s\big) = \mathrm{sp}'_\theta\big(\tilde{Q}_q^\pi(s,\text{cont}) - \tilde{Q}_{q'}^\pi(s,\text{cont})\big), \qquad V_q^\pi(s) = \mathrm{sp}_\theta\big(\tilde{Q}_q^\pi(s,\text{cont}) - \tilde{Q}_{q'}^\pi(s,\text{cont})\big), \tag{S.1.104}$$

with $\mathrm{sp}_\theta(z) = \theta\log(1+e^{z/\theta})$ and $\mathrm{sp}'_\theta(z) = 1/(1+e^{-z/\theta})$. The transition kernel (S.1.17)–(S.1.18), the payoffs r_p, the Bellman equations (S.1.20)–(S.1.22) for $\tilde{Q}_p^\pi$ and the value recursion (S.1.25) for $\tilde{V}_p^\pi$ are unchanged, except that π is no longer pure and the entropy bonus $\theta\mathscr{H}(\pi_q(s))$ is credited to the active player q.

Theorem S.1.10 (Softplus forward equivalence, `Lean_65`). *The entropy-regularized Stopping Game of (S.1.103) admits an SPNE whose activation-state policy is the Gibbs policy (S.1.104); it is unique on networks without max-pooling nodes, since (S.1.103) is strictly concave (ties at max-pooling states are resolved as in Section S.1.1). Under this equilibrium, at every activation state $s_{j,q}^{(l,\text{act})}$,*

$$\tilde{V}_p^{\pi^\star}\big(s_{j,q}^{(l,\text{act})}\big) \;=\; a_{\text{soft},j}^{(l,p\oplus q,\,\theta)}, \qquad V_q^{\pi^\star}\big(s_{j,q}^{(l,\text{act})}\big) \;=\; \mathrm{sp}_\theta\big(z_j^{(l)}\big), \tag{S.1.105}$$

where $a_{\text{soft},j}^{(l,\pm,\theta)}$ are the Softplus-Decomposition activations of the Softplus network with activation sp_θ, propagated by (S.1.2)–(S.1.3) from $a_{\text{soft},k}^{(0,\pm,\theta)} = x_k^\pm$ with the decomposition function

$$f_{\text{soft}}^\theta(z^+,z^-) \;\coloneqq\; \big(\mathrm{sp}'_\theta(z^+-z^-)\,z^+ + \theta\,\mathscr{H}_\theta(z^+-z^-),\; \mathrm{sp}'_\theta(z^+-z^-)\,z^-\big), \tag{S.1.106}$$

and $\mathscr{H}_\theta(z) \coloneqq \mathscr{H}(\mathrm{sp}'_\theta(z))$ the binary entropy of the Gibbs weight.

Proof. Induct backward in l as in Theorem S.1.3, with same base case $\tilde{V}_p^{\pi^\star}(s_{k,q}^{(0)}) = x_k^{p\oplus q} = a_{\text{soft},k}^{(0,p\oplus q,\theta)}$. For $l \ge 1$, assume (S.1.105) holds at layer $l-1$, so the decomposed activations obey $a_{\text{soft},i}^{(l-1,+,\theta)} - a_{\text{soft},i}^{(l-1,-,\theta)} = a_i^{(l-1)}$. Substituting into (S.1.22), the Q-difference of (S.1.104) computes to $b_j^{(l)} + \sum_{i,r} W_{ij}^{(l,q\oplus r)}\,\xi_{q,r}\,a_i^{(l-1)} = z_j^{(l)}$, using $\sum_r W_{ij}^{(l,q\oplus r)}\xi_{q,r} = W_{ij}^{(l)}$ exactly as in the proof of Theorem S.1.3. Hence, the Gibbs policy is $\pi_q^\star(\text{cont} \mid \cdot) = \mathrm{sp}'_\theta(z_j^{(l)})$, and (S.1.104) gives $V_q^{\pi^\star}(s_{j,q}^{(l,\text{act})}) = \mathrm{sp}_\theta(z_j^{(l)})$, the second identity of (S.1.105).

For the first identity, the active player ($p = q$, so $p\oplus q = +$) receives both the routed expectation and entropy bonus in (S.1.103): $\tilde{V}_q^{\pi^\star}(s_{j,q}^{(l,\text{act})}) = \mathrm{sp}'_\theta(z_j^{(l)})\,z_j^{(l,+)} + \theta\,\mathscr{H}_\theta(z_j^{(l)}) = a_{\text{soft},j}^{(l,+,\theta)}$, where $\tilde{Q}_q^{\pi^\star}(\cdot,\text{cont}) = z_j^{(l,+)}$ follows from the inductive hypothesis via (S.1.2). The opponent ($p = q'$, so $p\oplus q = -$) receives only the routed expectation: $\tilde{V}_{q'}^{\pi^\star}(s_{j,q}^{(l,\text{act})}) = \mathrm{sp}'_\theta(z_j^{(l)})\,z_j^{(l,-)} = a_{\text{soft},j}^{(l,-,\theta)}$ by (S.1.3), closing the induction. □

Gradient recovery on Softplus networks. Definition S.1.6 now uses the trajectory law Π_x^θ generated by Gibbs policies (S.1.104), replacing deterministic stop/continue decisions.

Theorem S.1.11 (Softplus gradient recovery, Lean_66). *Under the SPNE of Theorem S.1.10 on the Softplus network with activation* sp_θ*, the discounted occupation difference recovers the Softplus-network gradient exactly as in Theorem S.1.6:*

$$\frac{\partial a_j^{(m)}}{\partial a_i^{(l)}} \;=\; \xi_{q,+}\Big(\Gamma_x\big(s_{i,+}^{(l,\mathrm{act})} \mid s\big) - \Gamma_x\big(s_{i,-}^{(l,\mathrm{act})} \mid s\big)\Big), \qquad s = s_{j,q}^{(m,\mathrm{act})}, \tag{S.1.107}$$

with Γ_x *now driven by* Π_x^θ.

Proof. Write $D(s_{h,r}^{(m',\mathrm{act})}) \coloneqq \Gamma_x(s_{i,r}^{(l,\mathrm{act})} \mid s_{h,r}^{(m',\mathrm{act})}) - \Gamma_x(s_{i,r'}^{(l,\mathrm{act})} \mid s_{h,r}^{(m',\mathrm{act})})$, the right-hand side above. The one-step occupation recursion in the proof of Theorem S.1.6 holds with the gate $\mathbf{1}[z_j^{(m)} > 0]$ replaced by the Gibbs continuation probability $\mathrm{sp}'_\theta(z_j^{(m)})$; taking its difference over the two targets gives $D(s_{j,q}^{(m,\mathrm{act})}) = \mathrm{sp}'_\theta(z_j^{(m)}) \sum_h \big(W_{hj}^{(m,+)} D(s_{h,q}^{(m-1,\mathrm{act})}) - W_{hj}^{(m,-)} D(s_{h,q'}^{(m-1,\mathrm{act})})\big)$. By induction on $m-l$, $D(s_{h,r}^{(m-1,\mathrm{act})}) = \partial a_h^{(m-1)}/\partial a_i^{(l)}$ for both r, so the right-hand side is $\mathrm{sp}'_\theta(z_j^{(m)}) \sum_h W_{hj}^{(m)}\, \partial a_h^{(m-1)}/\partial a_i^{(l)}$, the chain rule at a Softplus gate; the base case $D(s_{i,r}^{(l,\mathrm{act})}) = 1$ and the addition and max-pooling steps are unchanged. Unlike Part 1 of Theorem S.1.6, the individual occupation measures are not derivatives of the decomposed activations (S.1.106), because the soft gate depends on them; only their difference is. □

Probit analog and GELU. Replacing $\theta\mathscr{H}$ in (S.1.103) by the probit regularizer $\mathscr{R}_\Phi(\pi) = \phi(\Phi^{-1}(\pi))$ gives, by Lemma S.1.9, the continuation probability $\Phi(z)$ and the value

$$z\Phi(z) + \phi(z) = \mathbb{E}_{\eta\sim\mathcal{N}(0,1)}[\mathrm{ReLU}(z+\eta)],$$

so the proofs above carry over verbatim with sp_θ replaced by this Gaussian-smoothed ReLU, sp'_θ by its derivative Φ and $\theta\mathscr{H}_\theta(z)$ by $\phi(z)$: the probit-regularized game recovers Gaussian-smoothed-ReLU networks and their exact gradient. GELU, $z\Phi(z)$, is *not* covered: it is not convex, so it is not the value $\max_\pi\{\pi z + \mathscr{R}(\pi)\}$ (a Fenchel conjugate) of any regularized stop/continue choice. Substituting the gate $\Phi(z)$ into Theorem S.1.6 on a GELU network yields the gradient of the gate-conditioned map $a = \Phi(z^{\mathrm{fwd}})\, z$, not the GELU gradient $\Phi(z) + z\phi(z)$ (Section S.1.4.3).

Relation to the curvature-based defense of Dombrowski et al. Dombrowski et al. (2022) train Softplus networks from scratch as a defense against gradient-explanation manipulation. Their argument bounds the Frobenius norm of the network Hessian in terms of the weight norms and activation derivatives: the bound diverges in the ReLU limit, whereas $|\mathrm{sp}''_\theta(z)| \le 1/(4\theta)$ gives a finite uniform bound for fixed weights, and the gradient theorem then bounds changes in the gradient explanation under input perturbations. Theorems S.1.10–S.1.11 realize this replacement by entropy-regularizing the activation-state stop/continue policy at temperature θ. Whether the Hellinger response of the resulting trajectory law (Section 8.3) yields bounds sharper than, or differently targeted from, the Lipschitz bounds of Dombrowski et al. (2022) remains open.

S.2 Extended Attention-Routing Derivations

We use the target rule, stream masses, zero-mass convention and notation of Section 5.5.1 of the thesis.

S.2.1 State space

We embed attention into the routing graph of §S.1.3, writing $s_{\bullet}^{(\mathrm{att},\,\mathrm{Side},\,\mathrm{type})}$, where $\mathrm{Side} \in \{\mathrm{O}, \mathrm{val}, \mathrm{X}\}$ identifies the activation slot and $\mathrm{type} \in \{\mathrm{act}, \mathrm{lin}\}$ the state type of §S.1.3. For AV mixing and the Value projection, $t,k \in \{1,\dots,S\}$ index output/key tokens, $d \in \{1,\dots,d_h\}$ indexes features within a fixed head h, and $e \in \{1,\dots,D\}$ indexes per-token input embedding dimensions.

States for the composite $Av{+}W_v$ block. The explanatory walk adds three state types to the surrounding ReLU-network's, all indexed by the player in turn.

- **Output activation states** $s_{t,d,p}^{(\mathrm{att},\,\mathrm{O},\,\mathrm{act})}$ at the head's output entry $O_{t,d}$, indexed by output token $t \in \{1,\dots,S\}$, within-head feature dim $d \in \{1,\dots,d_h\}$, and player $p \in \{+,-\}$. The surrounding linear-layer recursion of §S.1.3.2 delivers occupation mass to these entry states $\{s_{t,d,p}^{(\mathrm{att},\,\mathrm{O},\,\mathrm{act})}\}_{t,d,p}$ from the layer behind the head.
- **Output sign-branch (linear) states** $s_{(t,d),p,\kappa}^{(\mathrm{att},\,\mathrm{O},\,\mathrm{lin})}$, indexed by (t,d,p,κ). These mirror the linear routing states $s_{j,q,\kappa}^{(l,\mathrm{lin})}$ of Definition S.1.7 for AV mixing at (t,d). QK logits enter through the reference $\bar{\pi}_t = A_{t,\cdot}$: the active player picks key k using a mixed action $\pi^{\star}_{t,d,\kappa} \in \Delta(\{1,\dots,S\})$ that combines the AV reference $\bar{\pi}_t$ with the κ-stream Value mass $\tilde{v}^{\kappa}_{k,d}$.
- **Value-projection linear states** $s_{(k,d),p,\kappa}^{(\mathrm{att},\,\mathrm{val},\,\mathrm{lin})}$, indexed by key token k, within-head feature dim d, player p, and sign κ. Here the player routes through the per-token Value projection to input dimension e (or the bias node), with game value $V_p\big(s_{(k,d),p,\kappa}^{(\mathrm{att},\,\mathrm{val},\,\mathrm{lin})}\big) = \ell(\tilde{v}^{\kappa}_{k,d})$ with $\tilde{v}^{\kappa}_{k,d}$ from (5.17).
- **Input activation states** $s_{(k,e),p}^{(\mathrm{att},\,\mathrm{X},\,\mathrm{act})}$ at the input embedding entry $X_{k,e}$, indexed by token–embedding-dim pair (k,e) and player p. These exit states lie upstream of the Value projection, indexed by embedding dimension e; the head-feature axis d enters at its output. The surrounding recursion resumes from their per-player occupation on $\{s_{(k,e),p}^{(\mathrm{att},\,\mathrm{X},\,\mathrm{act})}\}_{(k,e),p}$.

Transitions. The three-step trajectory has discount 1 except at the sign-oracle split, which uses $\{2\alpha, 2\beta\}$ as at ReLU activation states in Definition S.1.7. Each output (t,d) has *one* sign-oracle split: $A_{t,k} \geq 0$ absorbs AV mixing into the W_v sign split.

1) *Sign-oracle split (output activation → output sign-branch, fixed (t,d)).* At $s_{t,d,p}^{(\mathrm{att},\,\mathrm{O},\,\mathrm{act})}$ an unobserved Oracle transitions uniformly to $s_{(t,d),p,+}^{(\mathrm{att},\,\mathrm{O},\,\mathrm{lin})}$ (player p retains the turn, trajectory discount 2α) or to $s_{(t,d),p',-}^{(\mathrm{att},\,\mathrm{O},\,\mathrm{lin})}$ (turn switches to opponent p', discount 2β), each with probability $\frac{1}{2}$. The feature index d is preserved.
2) *Value-routing policy (output sign-branch → Value-projection linear).* At $s_{(t,d),p,\kappa}^{(\mathrm{att},\,\mathrm{O},\,\mathrm{lin})}$ the active player picks a key token k by the mixed action

$$\pi^{\star}_{t,d,\kappa}(k) \;=\; \frac{A_{t,k}\,\tilde{v}^{\kappa}_{k,d}}{Z^{\kappa}_{t,d}}, \tag{S.2.1}$$

derived in §S.2.3 as the equilibrium of a KL-regularized log-payoff problem against the reference $\bar{\pi}_t = A_{t,\cdot}$. The transition to $s^{(\mathrm{att},\,\mathrm{val},\,\mathrm{lin})}_{(k,d),p,\kappa}$ preserves player, sign κ and feature d, changing only the token index t to k, with discount 1.

3) *Value-projection routing (Value-projection linear → input activation).* At $s^{(\mathrm{att},\,\mathrm{val},\,\mathrm{lin})}_{(k,d),p,\kappa}$ the active player picks an input dim e by the standard linear-state Gibbs policy of Definition S.1.7 on the κ-stream masses $[X_{k,e}W_{v,e,d}]^\kappa$ (the bias node receives $[b_{v,d}]^\kappa/\tilde{v}^\kappa_{k,d}$ and is absorbed):

$$\pi^\star_{\mathrm{val},k,d,\kappa}(e) \;=\; \frac{[X_{k,e}W_{v,e,d}]^\kappa}{\tilde{v}^\kappa_{k,d}}. \tag{S.2.2}$$

The trajectory transitions to $s^{(\mathrm{att},\,\mathrm{X},\,\mathrm{act})}_{(k,e),p}$ with the player label preserved and trajectory-discount factor 1.

We evaluate the discounted occupation measure $\Gamma_x(\,\cdot\,)$ of §S.1.3.2 at all four state types. The recovery theorem of §S.2.4 compares its input-neuron player-differences with the target rule (5.21).

S.2.2 Legendre–Fenchel transform of the KL divergence

The Value-routing equilibrium follows from the KL convex-conjugate identity:

Lemma S.2.1 (Legendre–Fenchel transform of KL divergence, `Lean_78`**).** *Let $\mathscr{X}$ be a finite set, $\bar{\pi} \in \Delta(\mathscr{X})$ a reference distribution with full support on $\mathscr{X}$, and $r : \mathscr{X} \to \mathbb{R}$ a real-valued payoff. The KL-regularized linear maximization*

$$F(r;\bar{\pi}) \;\coloneqq\; \max_{\pi\in\Delta(\mathscr{X})} \Big\{ \mathbb{E}_\pi[r] \;-\; \mathrm{KL}(\pi \,\|\, \bar{\pi}) \Big\} \tag{S.2.3}$$

has optimizer and optimum:

$$\pi^\star_r(x) \;=\; \frac{\bar{\pi}(x)\,\exp\big(r(x)\big)}{Z(r;\bar{\pi})}, \qquad F(r;\bar{\pi}) \;=\; \log Z(r;\bar{\pi}) \;=\; \log \sum_{x\in\mathscr{X}} \bar{\pi}(x)\,\exp\big(r(x)\big). \tag{S.2.4}$$

Two specializations are used below. For $r(x) = \ell(u(x))$, with $u : \mathscr{X} \to \mathbb{R}_{\geq 0}$ and $\exp(-\infty) \coloneqq 0$, we have $\exp(r(x)) = u(x)$. The value is the logarithm of the $\bar{\pi}$-weighted sum, and the optimizer its tilted normalization:

$$F\big(\ell(u);\,\bar{\pi}\big) \;=\; \ell\Big(\sum_x \bar{\pi}(x)\,u(x)\Big), \qquad \pi^\star_{\ell(u)}(x) \;=\; \frac{\bar{\pi}(x)\,u(x)}{\sum_{x'} \bar{\pi}(x')\,u(x')}. \tag{S.2.5}$$

When $\bar{\pi}$ is uniform on $\mathscr{X}$, $\mathrm{KL}(\pi \,\|\, \mathrm{Unif}_{\mathscr{X}}) = \log|\mathscr{X}| - \mathscr{H}(\pi)$, so the KL penalty in (S.2.3) becomes an entropy bonus:

$$\mathbb{E}_\pi[r] \;-\; \mathrm{KL}\big(\pi \,\|\, \mathrm{Unif}_{\mathscr{X}}\big) \;=\; \mathbb{E}_\pi[r] \;+\; \mathscr{H}(\pi) \;-\; \log|\mathscr{X}|. \tag{S.2.6}$$

Up to the additive constant $-\log|\mathscr{X}|$, (S.2.3) therefore reduces to the entropy-regularized maximization of (S.1.56), with value $\log\sum_x \exp(r(x))$ and optimizer $\pi^\star(x) \propto \exp(r(x))$.

Proof. This is the Gibbs variational principle (Boyd and Vandenberghe, 2004): for every π, $\mathbb{E}_\pi[r] - \mathrm{KL}(\pi \,\|\, \bar{\pi}) = \log Z(r;\bar{\pi}) - \mathrm{KL}(\pi \,\|\, \pi^\star_r)$, which is maximal, with value $\log Z(r;\bar{\pi})$, exactly at $\pi = \pi^\star_r$. □

S.2.3 Value-routing equilibrium

Fix an output entry (t,d), a sign $\kappa \in \{+,-\}$, and the reference $\bar{\pi}_t = A_{t,\cdot}$ from §S.2.1 (shared across feature dimensions d by AV mixing). At the output sign-branch state $s^{(\mathrm{att},\,\mathrm{O},\,\mathrm{lin})}_{(t,d),p,\kappa}$ the active player p solves the KL-regularized log-payoff maximization

$$V_p\big(s^{(\mathrm{att},\,\mathrm{O},\,\mathrm{lin})}_{(t,d),p,\kappa}\big) \;:=\; \max_{\pi\in\Delta(\{1,\dots,S\})} \left\{ \sum_{k=1}^{S} \pi(k)\,\ell\big(\tilde{v}^{\kappa}_{k,d}\big) \;-\; \mathrm{KL}\big(\pi \,\|\, \bar{\pi}_t\big) \right\}, \tag{S.2.7}$$

where $\tilde{v}^{\kappa}_{k,d} \geq 0$ is the κ-stream split of the Value entry $v_{k,d}$ from (5.17) and ℓ is the extended logarithm of §S.1.3. The payoff $\ell(\tilde{v}^{\kappa}_{k,d})$ on action k is the active-player value of the predecessor Value-projection linear state, $V_p(s^{(\mathrm{att},\,\mathrm{val},\,\mathrm{lin})}_{(k,d),p,\kappa}) = \ell(\tilde{v}^{\kappa}_{k,d})$, so combining with the AV log-weight $\ell(A_{t,k})$ gives composite payoff $\ell(A_{t,k}\,\tilde{v}^{\kappa}_{k,d})$ at the (t,d)-output. Define the $\bar{\pi}_t$-weighted Value mass at each output:

$$Z^{\kappa}_{t,d} \;:=\; \sum_{m=1}^{S} \bar{\pi}_t(m)\,\tilde{v}^{\kappa}_{m,d}. \tag{S.2.8}$$

Only $\tilde{v}^{\kappa}_{m,d}$ depends on d.

Proposition S.2.2 (Value-side optimal policy, `Lean_79`). *If $Z^{\kappa}_{t,d} > 0$, the unique optimizer of (S.2.7) is*

$$\pi^{\star}_{t,d,\kappa}(k) \;=\; \frac{\bar{\pi}_t(k)\,\tilde{v}^{\kappa}_{k,d}}{Z^{\kappa}_{t,d}}, \tag{S.2.9}$$

with active-player value $V_p(s^{(\mathrm{att},\,\mathrm{O},\,\mathrm{lin})}_{(t,d),p,\kappa}) = \ell(Z^{\kappa}_{t,d})$ and inactive-player value $V_{p'}(s^{(\mathrm{att},\,\mathrm{O},\,\mathrm{lin})}_{(t,d),p,\kappa}) = -\infty$ as in Definition S.1.7. If $Z^{\kappa}_{t,d} = 0$, no key contributes mass on stream κ and $V_p(s^{(\mathrm{att},\,\mathrm{O},\,\mathrm{lin})}_{(t,d),p,\kappa}) = -\infty$. All key policies then have value $-\infty$; the maximizer is nonunique. We use the cemetery completion above.

Proof. Apply (S.2.5) of Lemma S.2.1 with $\mathscr{X} = \{1,\dots,S\}$, $\bar{\pi} = \bar{\pi}_t$ and $u(k) = \tilde{v}^{\kappa}_{k,d}$. Then $r(k) = \ell(\tilde{v}^{\kappa}_{k,d})$ gives $\exp r(k) = \tilde{v}^{\kappa}_{k,d}$, yielding (S.2.9) and value $\ell(Z^{\kappa}_{t,d})$ for $Z^{\kappa}_{t,d} > 0$. The inactive-player value $-\infty$ is the Routing-Game convention of Definition S.1.7, inherited from the $r = q'$ branch of (S.1.59). If $Z^{\kappa}_{t,d} = 0$ then $\bar{\pi}_t(k)\,\tilde{v}^{\kappa}_{k,d} = 0$ for every k and the supremum of (S.2.7) over the simplex is $-\infty$, consistent with $\ell(0) = -\infty$. □

Specialization to the softmax reference. Setting $\bar{\pi}_t = A_{t,\cdot}$ in (S.2.9) yields the closed-form per-(t,d,κ) policies used in the RG's backward walk through the attention block,

$$\pi^{\star}_{t,d,+}(k) \;=\; \frac{A_{t,k}\,\tilde{v}^{+}_{k,d}}{Z^{+}_{t,d}}, \qquad \pi^{\star}_{t,d,-}(k) \;=\; \frac{A_{t,k}\,\tilde{v}^{-}_{k,d}}{Z^{-}_{t,d}}, \tag{S.2.10}$$

when $Z^{\kappa}_{t,d} > 0$. When $Z^{\kappa}_{t,d} = 0$, set all these key probabilities to zero and the cemetery probability to one. Here $Z^{\kappa}_{t,d} = \sum_m A_{t,m}\,\tilde{v}^{\kappa}_{m,d}$ is defined in (S.2.8). Using the cached softmax row $\bar{\pi}_t = A_{t,\cdot}$ as a fixed input to (S.2.7) suffices; the oracle interpretation of §5.5.2 is optional.

S.2.4 $\alpha\beta$-LRP recovery for the attention block

For the four-state trajectory of §S.2.1, discounted by (S.1.83), the player-difference of Γ_x at the *input activation states* recovers the target rule (5.21) of §5.5.1. These states precede the Value-projection linear states, where W_v acts. The two objects are:

- the *game-side* object is the discounted occupation measure $\Gamma_x(\,\cdot\,)$ of §S.1.3.2 evaluated at the four attention-block state types of §S.2.1. Following (S.1.82), we form the output difference $\Gamma^O_{t,d}$ at entry (defined in (S.2.13)) and the input-neuron difference at exit

$$\Gamma^X_{k,e} \;\coloneqq\; \Gamma_x\big(s^{(\mathrm{att},\,\mathrm{X},\,\mathrm{act})}_{(k,e),+}\big) - \Gamma_x\big(s^{(\mathrm{att},\,\mathrm{X},\,\mathrm{act})}_{(k,e),-}\big). \tag{S.2.11}$$

- the *attribution-side* object is the standard $\alpha\beta$-LRP relevance vector $R^X_{k,e}$ given by the input-level target rule (5.21), seeded by $R^O_{t,d}$ from the surrounding linear-layer recursion above the head.

Theorem S.2.3 ($\alpha\beta$-LRP recovery for attention, `Lean_80`). *For signed inputs X, under the SPNE of §S.2.1 with reference $\bar\pi_t = A_{t,\cdot}$ and the trajectory discount of (S.1.83) restricted to the (sole) sign-oracle split at $s^{(\mathrm{att},\,\mathrm{O},\,\mathrm{act})}$, the per-input-neuron player-difference $\Gamma^X_{k,e}$ from (S.2.11) satisfies*

$$\Gamma^X_{k,e} \;=\; \sum_{t,d}\left(\alpha\,\frac{A_{t,k}\,[X_{k,e}W_{v,e,d}]^+}{Z^+_{t,d}} \;-\; \beta\,\frac{A_{t,k}\,[X_{k,e}W_{v,e,d}]^-}{Z^-_{t,d}}\right)\Gamma^O_{t,d}, \tag{S.2.12}$$

where the per-output-entry input difference is

$$\Gamma^O_{t,d} \;\coloneqq\; \Gamma_x\big(s^{(\mathrm{att},\,\mathrm{O},\,\mathrm{act})}_{t,d,+}\big) \;-\; \Gamma_x\big(s^{(\mathrm{att},\,\mathrm{O},\,\mathrm{act})}_{t,d,-}\big), \tag{S.2.13}$$

with $\Gamma^O_{t,d} = R^O_{t,d}$ by inductive hypothesis from the surrounding recursion, for the unit seed used by the implementation. Any real seed r_u multiplies the occupation difference by r_u, including the signed affine-logit seed of Section 4.5. The right-hand side is exactly the target rule (5.21), hence $\Gamma^X_{k,e} = R^X_{k,e}$. Aggregating over e and the bias node at fixed (k,d) collects the relevance routed through the Value entry $v_{k,d}$:

$$\Gamma^{\mathrm{val}}_{k,d} \;=\; \sum_{t}\left(\alpha\,\frac{A_{t,k}\,\tilde v^+_{k,d}}{Z^+_{t,d}} \;-\; \beta\,\frac{A_{t,k}\,\tilde v^-_{k,d}}{Z^-_{t,d}}\right)\Gamma^O_{t,d} \;=\; R^{\mathrm{val}}_{k,d}, \tag{S.2.14}$$

recovering the per-Value-entry formula (5.19).

Proof. By hypothesis, the surrounding recursion supplies $\Gamma^O_{t,d} = R^O_{t,d}$ at every output (t,d); the proof is block-local.

Per-state recursion at the input activation states. Fix an input embedding (k,e) and a player label $r \in \{+,-\}$. A trajectory reaches $s^{(\mathrm{att},\,\mathrm{X},\,\mathrm{act})}_{(k,e),r}$ from an output activation $s^{(\mathrm{att},\,\mathrm{O},\,\mathrm{act})}_{t,d,p}$ in three stages, with the discounts and player switches of §S.2.1. Each path fixes $d \in \{1,\dots,d_h\}$: the oracle acts at (t,d), $\pi^\star_{t,d,\kappa}$ depends on d through $\tilde v^\kappa_{k,d}$, and $\pi^\star_{\mathrm{val},k,d,\kappa}$ also depends on d. The final transition drops d, reaching $X_{k,e}$ indexed by (k,e); the head-feature axis occurs downstream at the Value projection's output.

- *$\kappa = +$ branch (per (t,d)).* Sign-oracle takes $s^{(\mathrm{O,act})}_{t,d,p} \to s^{(\mathrm{O,lin})}_{(t,d),p,+}$ with probability $\frac{1}{2}$ and trajectory discount 2α (player retained). The Value-routing policy $\pi^\star_{t,d,+}(k)$ takes the trajectory to $s^{(\mathrm{val,lin})}_{(k,d),p,+}$

(player, sign, d preserved, discount 1). The Value-projection routing $\pi^{\star}_{\mathrm{val},k,d,+}(e)$ takes the trajectory to $s^{(\mathrm{X,act})}_{(k,e),p}$ (player preserved, d dropped, discount 1). The resulting contribution from (t,d) to $\Gamma_x(s^{(\mathrm{X,act})}_{(k,e),r})$ is

$$\tfrac{1}{2}(2\alpha)\,\pi^{\star}_{t,d,+}(k)\,\pi^{\star}_{\mathrm{val},k,d,+}(e)\,\Gamma_x\big(s^{(\mathrm{O,act})}_{t,d,r}\big) = \alpha\,\frac{A_{t,k}\,\tilde{v}^{+}_{k,d}}{Z^{+}_{t,d}}\,\frac{[X_{k,e}W_{v,e,d}]^{+}}{\tilde{v}^{+}_{k,d}}\,\Gamma_x\big(s^{(\mathrm{O,act})}_{t,d,r}\big) \tag{S.2.15}$$

$$= \alpha\,\frac{A_{t,k}\,[X_{k,e}W_{v,e,d}]^{+}}{Z^{+}_{t,d}}\,\Gamma_x\big(s^{(\mathrm{O,act})}_{t,d,r}\big), \tag{S.2.16}$$

where cancellation applies when $\tilde{v}^{+}_{k,d} > 0$. If this mass or $Z^{+}_{t,d}$ is zero, the ordinary-successor contribution on both sides is zero by (5.18).

– $\kappa = -$ *branch (per (t,d)).* Sign-oracle takes $s^{(\mathrm{O,act})}_{t,d,p} \to s^{(\mathrm{O,lin})}_{(t,d),p',-}$ with probability $\frac{1}{2}$, discount 2β, turn switched to opponent p'; the rest of the path preserves p', ending at $s^{(\mathrm{X,act})}_{(k,e),p'}$. The contribution to $\Gamma_x(s^{(\mathrm{X,act})}_{(k,e),r})$ comes from sources with player $p = r'$ (so the switch yields r):

$$\tfrac{1}{2}(2\beta)\,\pi^{\star}_{t,d,-}(k)\,\pi^{\star}_{\mathrm{val},k,d,-}(e)\,\Gamma_x\big(s^{(\mathrm{O,act})}_{t,d,r'}\big) = \beta\,\frac{A_{t,k}\,[X_{k,e}W_{v,e,d}]^{-}}{Z^{-}_{t,d}}\,\Gamma_x\big(s^{(\mathrm{O,act})}_{t,d,r'}\big). \tag{S.2.17}$$

Summing over (t,d) and over the two branches,

$$\Gamma_x\big(s^{(\mathrm{att,\,X,\,act})}_{(k,e),r}\big) = \sum_{t,d}\Big[\alpha\,\frac{A_{t,k}\,[X_{k,e}W_{v,e,d}]^{+}}{Z^{+}_{t,d}}\,\Gamma_x\big(s^{(\mathrm{O,act})}_{t,d,r}\big) + \beta\,\frac{A_{t,k}\,[X_{k,e}W_{v,e,d}]^{-}}{Z^{-}_{t,d}}\,\Gamma_x\big(s^{(\mathrm{O,act})}_{t,d,r'}\big)\Big], \tag{S.2.18}$$

where the $\frac{1}{2}$ from the sign-oracle splits cancels against the 2 in $2\alpha, 2\beta$.

Player difference. Differencing (S.2.18) for $r{=}+$ minus $r{=}-$ uses $\Gamma_x(s^{(\mathrm{O,act})}_{t,d,+}) - \Gamma_x(s^{(\mathrm{O,act})}_{t,d,-}) = \Gamma^{O}_{t,d}$ in the $+$ branch (player kept) and the opposite $-\Gamma^{O}_{t,d}$ in the $-$ branch (player switched):

$$\Gamma^{X}_{k,e} = \alpha\sum_{t,d}\frac{A_{t,k}\,[X_{k,e}W_{v,e,d}]^{+}}{Z^{+}_{t,d}}\,\Gamma^{O}_{t,d} - \beta\sum_{t,d}\frac{A_{t,k}\,[X_{k,e}W_{v,e,d}]^{-}}{Z^{-}_{t,d}}\,\Gamma^{O}_{t,d}, \tag{S.2.19}$$

which is (S.2.12).

Target-rule identification. With $\Gamma^{O}_{t,d} = R^{O}_{t,d}$, the preceding identity matches (5.21), giving $\Gamma^{X}_{k,e} = R^{X}_{k,e}$. Aggregating over e and the bias node at fixed (k,d) uses $\sum_e [X_{k,e}W_{v,e,d}]^{\kappa} + [b_{v,d}]^{\kappa} = \tilde{v}^{\kappa}_{k,d}$, giving (S.2.14), which matches (5.19). □

S.2.5 Risk-Averse Attention via ADF

When ADF variance propagation is active, let s_{tk} denote the uncertainty score attached to attending from query token t to key token k (in the current implementation, $s_{tk} = \hat{\sigma}_k$, the square root of the feature-averaged ADF variance of token k's query projection; it depends only on the key token and is a proxy for, not the standard deviation of, e_{tk}). Risk-averse attention shifts the QK-logits in the direction of reduced uncertainty,

$$\tilde{e}_{tk} = e_{tk} + \lambda_{\mathrm{sm}}\,s_{tk}, \qquad \lambda_{\mathrm{sm}} \le 0, \tag{S.2.20}$$

yielding the risk-modified policy

$$\tilde{\pi}_t(k) = \frac{\exp(\tilde{e}_{tk})}{\sum_{k'} \exp(\tilde{e}_{tk'})}. \tag{S.2.21}$$

The sign convention $\lambda_{\mathrm{sm}} \leq 0$ is used throughout the paper and in all tables: more-negative values penalize high-uncertainty keys more strongly, so the shift always redirects relevance toward tokens whose QK-interactions are stable under input noise. The shift is computed in the AV-backward hook from cached pre-softmax logits, so the forward pass and model predictions remain unchanged.

The naming *softmax-shift risk aversion* is a deliberate analog of the activation-side risk aversion λ but acts in a different direction: λ is risk-averse on the value side (it routes through stable activations by lowering mean$-|\lambda|\sigma$), whereas $\lambda_{\mathrm{sm}} < 0$ exponentially tilts the routing policy, $\tilde{\pi}_t(k) \propto A_{t,k} \exp(\lambda_{\mathrm{sm}} s_{tk})$, toward *low*-uncertainty keys: $\sum_k \tilde{\pi}_t(k)\, s_{tk}$ is non-decreasing in λ_{sm}, while the entropy of $\tilde{\pi}_t$ may rise or fall. We retain the "risk-averse" label because what is being avoided is uncertainty in the routed pathway, but the reader should note that the shift acts on the routing policy, not on the player's utility curvature.

S.3 Extended Trajectory Derivations

This chapter gives the foundations and proofs for Chapter 8.

S.3.1 Layered Markov Process Foundations

Every Hellinger statement below uses a finite layered graph with cemetery †, instantiated by the two games' act-state *Markov Process*s (MPs). The natural RG adds a linear-routing state per neuron between activation states. Full-law diagnostics retain its gate, branch, and routing substeps; exact act-only elimination applies only to ordinary surviving paths (Remark S.3.1). The dynamic program indexes these refined substeps as layers and reads diagnostics at activation boundaries.

S.3.1.1 Layered Markov Process with Cemetery

State graph. Fix an integer $L \geq 1$. For each $l \in \{0, 1, \ldots, L\}$ let $\mathscr{S}^{(l)}$ be a finite set of *ordinary states* at layer l. Adjoin a single absorbing *cemetery state* † shared across all layers and write $\mathscr{S}_+^{(l)} := \mathscr{S}^{(l)} \cup \{\dagger\}$. The state graph is the disjoint union $\bigsqcup_l \mathscr{S}_+^{(l)}$; transitions move from layer l to $l-1$, with permanent absorption upon entering the cemetery. We call input-layer states *pixels*; the per-pixel decomposition sums over $\mathscr{S}_+^{(0)}$.

Transition kernel. For each layer $l \geq 1$, a *backward transition kernel* $T^{(l)} : \mathscr{S}_+^{(l)} \to \Delta\big(\mathscr{S}_+^{(l-1)}\big)$ assigns to each state $s \in \mathscr{S}_+^{(l)}$ a probability distribution $T^{(l)}(s \to \cdot)$ over $\mathscr{S}_+^{(l-1)}$, with the constraint that the cemetery is absorbing:

$$T^{(l)}(\dagger \to \dagger) = 1, \qquad T^{(l)}(\dagger \to s') = 0 \ \text{ for } \ s' \in \mathscr{S}^{(l-1)}. \tag{S.3.1}$$

An ordinary state moves to a layer-$(l-1)$ ordinary state or terminates early, with cemetery probability $T^{(l)}(s \to \dagger)$.

Initial distribution and trajectories. Fix a deterministic starting state $s_0 \in \mathscr{S}^{(L)}$. A *trajectory* (indexed by layer, $\zeta_l \in \mathscr{S}_+^{(l)}$) is a sequence

$$\zeta = (\zeta_L, \zeta_{L-1}, \ldots, \zeta_0) \ \in \ \mathscr{S}_+^{(L)} \times \mathscr{S}_+^{(L-1)} \times \cdots \times \mathscr{S}_+^{(0)}, \tag{S.3.2}$$

with trajectory distribution induced by $T^{(\cdot)}$ and δ_{s_0} at layer L:

$$\Pi(\zeta_L, \zeta_{L-1}, \ldots, \zeta_0) \ = \ \mathbf{1}\{\zeta_L = s_0\} \prod_{k=1}^{L} T^{(k)}(\zeta_k \to \zeta_{k-1}). \tag{S.3.3}$$

For any $l \in \{0, \ldots, L\}$ the prefix distribution $\Pi^{(l)}$ over $(\zeta_L, \ldots, \zeta_l)$ is the marginal of Π over the trailing coordinates.

Two parallel processes. We compare processes $M = A, B$ on the same graph and start state s_0, with kernels $T_A^{(l)}, T_B^{(l)}$ and laws $\Pi_A^{(l)}, \Pi_B^{(l)}$. Kernel differences may reflect different weights (e.g. randomization), activations (e.g. different inputs), or both.

S.3.1.2 How the Two Games Fit This Abstraction

We instantiate Section S.3.1.1's layered MP for both games. State labels $s_{i,p,\kappa}^{(l,\mathrm{type})}$ follow Appendices S.1.2 and S.1.3. We evaluate the SPNE policies of Theorems S.1.3 (SG) and S.1.7 (RG) at $\tau = 1$ (so activation-state decisions are deterministic, continue iff $z_j^{(l)} > 0$); for the RG we further fix $\varepsilon = 0$.

SG MP. Ordinary states at layers $l \in \{1, \dots, L\}$ are activation states $s_{i,p}^{(l,\mathrm{act})}$, and layer-0 ordinary states are pixel-player terminal pairs $s_{j,q}^{(0)}$ (two terminals per pixel, tagged by the arriving player). The start state is $s_0 = s_{u,+}^{(L,\mathrm{act})}$. The SPNE backward kernel keeps the player on positive weights and flips it on negative weights:

$$
\begin{aligned}
T_M^{(l)}\big(s_{j,p}^{(l,\mathrm{act})} \to s_{i,p}^{(l-1,\mathrm{act})}\big) &= \frac{W_M^{(l,+)}[i,j]}{\gamma_{M,j}^{(l)}}\,\mathbf{1}\{z_j^{(l)} > 0\},\\
T_M^{(l)}\big(s_{j,p}^{(l,\mathrm{act})} \to s_{i,p'}^{(l-1,\mathrm{act})}\big) &= \frac{W_M^{(l,-)}[i,j]}{\gamma_{M,j}^{(l)}}\,\mathbf{1}\{z_j^{(l)} > 0\}, \qquad 2 \le l \le L,
\end{aligned}
\tag{S.3.4}
$$

with $\gamma_{M,j}^{(l)} = \sum_i |W_M^{(l)}[i,j]|$, $p' = -p$, cemetery row $T_M^{(l)}(s_{j,p}^{(l,\mathrm{act})} \to \dagger) = 1 - \mathbf{1}\{z_j^{(l)} > 0\}\mathbf{1}\{\gamma_{M,j}^{(l)} > 0\}$, with $0/0 \coloneqq 0$ in the ordinary transitions, and the analogous form at $l = 1$ landing on the pixel-player terminals $s_{i,p}^{(0)}, s_{i,p'}^{(0)}$. This is the natural SG MP of Section S.1.2 verbatim.

RG MP. Ordinary states at layers $l \in \{1, \dots, L\}$ are activation states $s_{i,p}^{(l,\mathrm{act})}$, and layer-0 ordinary states are pixel-stream terminal pairs $s_{j,\kappa}^{(0)}$ (two terminals per pixel, tagged by the layer-1 routing-edge stream). Starting at $s_0 = s_{u,+}^{(L,\mathrm{act})}$, the kernel composes the activation gate and κ-branched Gibbs routing into an act-to-act transition:

$$
\begin{aligned}
T_M^{(l)}\big(s_{j,p}^{(l,\mathrm{act})} \to s_{i,p}^{(l-1,\mathrm{act})}\big) &= \tfrac{1}{2}\,\pi_{M,+}^{(l)}(i \mid j)\,\mathbf{1}\{z_j^{(l)} > 0\},\\
T_M^{(l)}\big(s_{j,p}^{(l,\mathrm{act})} \to s_{i,p'}^{(l-1,\mathrm{act})}\big) &= \tfrac{1}{2}\,\pi_{M,-}^{(l)}(i \mid j)\,\mathbf{1}\{z_j^{(l)} > 0\}, \qquad 2 \le l \le L,
\end{aligned}
\tag{S.3.5}
$$

and at $l = 1$ the destinations are pixel-stream terminals carrying the routing-edge κ,

$$
T_M^{(1)}\big(s_{j,p}^{(1,\mathrm{act})} \to s_{i,\kappa}^{(0)}\big) \;=\; \tfrac{1}{2}\,\pi_{M,\kappa}^{(1)}(i \mid j)\,\mathbf{1}\{z_j^{(1)} > 0\}, \qquad \kappa \in \{+,-\},
\tag{S.3.6}
$$

For this act-only marginal, the cemetery probability is the residual

$$
T_M^{(l)}(s_{j,p} \to \dagger) = 1 - \sum_{s' \in \mathscr{S}^{(l-1)}} T_M^{(l)}(s_{j,p} \to s').
$$

An empty stream has zero ordinary policy mass; its $1/2$ branch share is absorbed even at a live source. A nonempty stream uses the temperature-τ Gibbs policy

$$
\pi_{M,\kappa}^{(l)}(i \mid j) \;\propto\; \big[\, a_{M,i}^{(l-1)}\, W_{M,ij}^{(l,\kappa)} \,\big]^{1/\tau}, \qquad i \in \mathrm{Pred}_j^{(l)},
\tag{S.3.7}
$$

(at $l = 1$, $a_{M,i}^{(0)} W_{M,ij}^{(1,\kappa)}$ is replaced by $[x_i W_{M,ij}^{(1)}]^{\kappa}$; when $\varepsilon > 0$, each continuing κ branch sends its $\frac{1}{2}\varepsilon^{1/\tau}/Z_{j,\kappa}^{(l)}$ share to the cemetery; as in the implementation, the bias node of Section S.1.3 is omitted).

If all scores in a stream are $-\infty$, its ordinary policy is zero and the whole branch is absorbed, in agreement with the outside-option convention. Equations (S.3.5)–(S.3.6) marginalize the *diagnostic* routing process, which retains the original gates, omits the bias action, and renormalizes each remaining branch. Nonzero bias mass therefore changes the kernel from Section S.1.3: for $a = W = b = 1$, $\varepsilon = 0$, the ordinary positive route has probability $1/4$ with bias and $1/2$ without it. Remark S.3.1 eliminates states within one process, without identifying these kernels. Full-trajectory diagnostics retain absorption histories through separate gate, branch, and routing substeps. The displayed act-to-act kernels compute ordinary surviving-path overlap; full overlap requires the refined process rather than its collapsed cemetery row.

S.3.1.3 Marginal Occupation Measures

For each model $M \in \{A,B\}$, the marginal probability that the chain visits state s at layer l is

$$m_M^{(l)}(s) \;=\; \sum_{\zeta:\zeta_l=s} \Pi_M^{(l)}(\zeta), \qquad s \in \mathscr{S}_+^{(l)}. \tag{S.3.8}$$

This satisfies the deterministic boundary condition

$$m_M^{(L)}(s_0) = 1, \qquad m_M^{(L)}(s) = 0 \;\text{ for }\; s \neq s_0, \tag{S.3.9}$$

and the one-step recursion

$$m_M^{(l-1)}(s') \;=\; \sum_{s\in\mathscr{S}_+^{(l)}} m_M^{(l)}(s)\, T_M^{(l)}(s \to s'), \qquad s' \in \mathscr{S}_+^{(l-1)}. \tag{S.3.10}$$

The *Marginal Pass* is the backward propagation (S.3.10). It yields the probability $m_M^{(0)}(s)$ of terminating at pixel s from s_0 and the total absorbed mass $m_M^{(0)}(\dagger)$.

S.3.2 Hellinger Distance from Bhattacharyya Coefficients

Properties. $\mathrm{BC}^{(l)}$ and $\mathrm{H}^{(l)}$ lie in [0,1] for every l, even under support mismatch. If a state is alive in one model and dead in the other, its geometric-mean row vanishes and its mass contributes directly to the Hellinger increment. Since H is a metric and satisfies $\mathrm{H}^2 \le d_{\mathrm{TV}} \le \sqrt{2}\,\mathrm{H}$ and $\mathrm{H}^2 \le \frac{1}{2}\mathrm{KL}$, small Hellinger distance controls TV and small KL controls H as well. *Normalization conventions.* These bounds use half-L^1 total variation $d_{\mathrm{TV}}(\Pi_A,\Pi_B) = \frac{1}{2}\sum_\zeta |\Pi_A(\zeta) - \Pi_B(\zeta)| \in [0,1]$ and KL in nats. For unnormalized total variation $\|\Pi_A - \Pi_B\|_1 = 2d_{\mathrm{TV}}$, adjust the factor of 2.

Alternative divergences. KL and Jeffreys divergence obey an exact Markov chain rule, are unbounded, and become infinite under support mismatch, such as routes opened or closed by randomization. Jensen–Shannon is bounded and finite; its mixture $\frac{1}{2}(\Pi_A + \Pi_B)$ is not Markov on the layered graph, precluding an exact linear-time dynamic program. Wasserstein transport requires a ground cost between neuron-labeled trajectories across layers, signs, and permuted neurons, with no canonical choice or exact local recursion. Hellinger is bounded, finite, and tractable: factorization of the overlap (Lemma S.3.1) gives a dynamic program linear in edges (Algorithm 8.1) and Theorem 8.1's non-negative per-state decomposition. It compares the *undiscounted* law Π: d_t lies outside the kernels, so path weights (α,β) and SG magnitudes $\gamma_j^{(l)}$, entering only through d_t, leave H unchanged. Thus H measures routing independently of carried weight.

Computation. Compute the marginal occupations $m_M^{(l)}$, run the Hellinger pass (Algorithm 8.1), and read off $\mathrm{H}^2(s)$ from (8.5); the total cost is linear in the number of edges.

S.3.3 Hellinger Identities and Proofs

S.3.3.1 Multiplicative Chain Rule

Lemma S.3.1 (Chain rule, `Lean_81`). *For every $l \in \{0, 1, \dots, L\}$, the Bhattacharyya coefficient of the prefix distributions factorizes along trajectories:*

$$\mathrm{BC}\big(\Pi_A^{(l)}, \Pi_B^{(l)}\big) \;=\; \sum_{(\zeta_L,\dots,\zeta_l)} \mathbf{1}\{\zeta_L = s_0\} \prod_{k=l+1}^{L} \widetilde{T}^{(k)}(\zeta_k, \zeta_{k-1}). \tag{S.3.11}$$

Proof. Substitute the Markov factorization $\Pi_M^{(l)}(\zeta) = \mathbf{1}\{\zeta_L = s_0\} \prod_{k=l+1}^{L} T_M^{(k)}(\zeta_k, \zeta_{k-1})$ for $M \in \{A,B\}$ into the definition of BC:

$$\mathrm{BC}\big(\Pi_A^{(l)}, \Pi_B^{(l)}\big) \;=\; \sum_{(\zeta_L,\dots,\zeta_l)} \mathbf{1}\{\zeta_L = s_0\} \sqrt{\prod_{k=l+1}^{L} T_A^{(k)}(\zeta_k, \zeta_{k-1})\, T_B^{(k)}(\zeta_k, \zeta_{k-1})} \tag{S.3.12}$$

$$= \sum_{(\zeta_L,\dots,\zeta_l)} \mathbf{1}\{\zeta_L = s_0\} \prod_{k=l+1}^{L} \widetilde{T}^{(k)}(\zeta_k, \zeta_{k-1}), \tag{S.3.13}$$

using $\sqrt{\prod_k a_k b_k} = \prod_k \sqrt{a_k b_k}$ for non-negative factors. □

Lemma S.3.2 (Monotonicity, `Lean_82`). *Let $\mathrm{BC}^{(l)} := \mathrm{BC}(\Pi_A^{(l)}, \Pi_B^{(l)})$. Then $\mathrm{BC}^{(l)}$ is monotonically non-increasing as l decreases from L to 0:*

$$1 \;=\; \mathrm{BC}^{(L)} \;\geq\; \mathrm{BC}^{(L-1)} \;\geq\; \cdots \;\geq\; \mathrm{BC}^{(0)} \;\geq\; 0. \tag{S.3.14}$$

Equivalently, the squared Hellinger distance $\mathrm{H}^2_{(l)} = 1 - \mathrm{BC}^{(l)}$ is monotonically non-decreasing.

Proof. Define

$$g^{(l)}(s) \;:=\; \sum_{(\zeta_L,\dots,\zeta_l)} \mathbf{1}\{\zeta_L = s_0\} \prod_{k=l+1}^{L} \widetilde{T}^{(k)}(\zeta_k, \zeta_{k-1})\, \mathbf{1}\{\zeta_l = s\}. \tag{S.3.15}$$

From Lemma S.3.1, $\mathrm{BC}^{(l)} = \sum_{s \in \mathscr{S}_+^{(l)}} g^{(l)}(s)$. Grouping the layer-$(l-1)$ sum by $\zeta_l = s$,

$$\mathrm{BC}^{(l-1)} \;=\; \sum_{s \in \mathscr{S}_+^{(l)}} g^{(l)}(s)\, \widetilde{R}^{(l)}(s). \tag{S.3.16}$$

Since $\widetilde{R}^{(l)}(s) \in [0,1]$ and $g^{(l)}(s) \geq 0$, we get $\mathrm{BC}^{(l-1)} \leq \mathrm{BC}^{(l)}$. The base case $\mathrm{BC}^{(L)} = 1$ holds because $\Pi_A^{(L)} = \Pi_B^{(L)} = \delta_{s_0}$ and $\sqrt{1 \cdot 1} = 1$. □

Lemma S.3.3 (Backward pass correctness, Lean_83). *Algorithm 8.1 computes $g^{(l)}$, hence $\mathrm{BC}^{(l)}$, $\mathrm{H}^{(l)}$, and $\mathrm{H}^2(s)$ correctly for every l and s. In particular,*

$$\sum_{s\in\mathscr{S}_+^{(0)}} \mathrm{H}^2(s) \;=\; \mathrm{H}^2_{(0)}. \tag{S.3.17}$$

Proof. Correctness of $g^{(l)}$ by downward induction. At $l = L$, the initialization $g^{(L)}(s) = \mathbf{1}\{s = s_0\}$ matches Lemma S.3.2, since the empty product equals 1. Assuming the stored $g^{(l)}$ matches that definition, iteration l computes

$$g^{(l-1)}(s') \;=\; \sum_{s\in\mathscr{S}_+^{(l)}} g^{(l)}(s)\,\widetilde{T}^{(l)}(s,s'), \tag{S.3.18}$$

which by direct expansion equals

$$\begin{aligned}\sum_{s}\Big(\sum_{(\zeta_L,\dots,\zeta_l)} \mathbf{1}\{\zeta_L = s_0\}\prod_{k=l+1}^{L}\widetilde{T}^{(k)}(\zeta_k,\zeta_{k-1})\,\mathbf{1}\{\zeta_l = s\}\Big)\widetilde{T}^{(l)}(s,s')\\ = \sum_{(\zeta_L,\dots,\zeta_{l-1})} \mathbf{1}\{\zeta_L = s_0\}\prod_{k=l}^{L}\widetilde{T}^{(k)}(\zeta_k,\zeta_{k-1})\,\mathbf{1}\{\zeta_{l-1} = s'\}.\end{aligned} \tag{S.3.19}$$

This equals $g^{(l-1)}(s')$ from Lemma S.3.2, closing the induction.

Correctness of $\mathrm{BC}^{(l)}$. After the loop, the algorithm sets $\mathrm{BC}^{(l)} = \sum_{s'} g^{(l)}(s')$, which by the proof of Lemma S.3.2 equals $\mathrm{BC}\big(\Pi_A^{(l)},\Pi_B^{(l)}\big)$. Then $\mathrm{H}^{(l)} = \sqrt{1-\mathrm{BC}^{(l)}}$ by definition (8.2).

Correctness of $\mathrm{H}^2(\cdot)$. At $l = 0$, the stored value satisfies

$$g^{(0)}(s) = \sum_{\zeta:\zeta_0=s} \sqrt{\Pi_A^{(0)}(\zeta)\Pi_B^{(0)}(\zeta)}, \tag{S.3.20}$$

so substitution into (8.5) gives Theorem 8.1's disagreement values; by (8.6), they sum to $\mathrm{H}^2_{(0)}$. □

Remark S.3.1 (Routing-state elimination and absorption histories, Lean_84). The refined diagnostic process alternates activation and linear-routing states. A dead activation stops at the gate; a live activation chooses its κ branch uniformly, then an empty branch or outside option may absorb at the routing substep. The process retains these substeps and pads the cemetery after absorption. Algorithm 8.1 runs on this expanded graph, with readouts at activation boundaries, as implemented by the separate gate and branch absorption updates.

For a trajectory that remains ordinary, deleting each intermediate routing state is a bijection: its neuron and player are determined by the adjacent activation states, and its branch by whether the player is kept or switched (or by the pixel-stream tag at the input). The act-to-act formulas therefore preserve ordinary surviving-path probabilities and their Bhattacharyya overlap. The same applies after conditioning both laws on survival.

For absorbing trajectories, deletion need not be injective: a gate stop and a later empty-branch stop can become the same act-only trajectory. Thus full-law Bhattacharyya and Hellinger equality is

asserted only when absorption histories are recoverable, for example when no alive routing branch leaks to the cemetery. In general the finite-fibre Cauchy–Schwarz inequality gives

$$\mathrm{BC}_{\text{refined}} \le \mathrm{BC}_{\text{act-only}}, \qquad \mathrm{H}_{\text{refined}} \ge \mathrm{H}_{\text{act-only}}.$$

For example, take one input $x = 1$ and weight $W = 1$. Model A has bias 0 and continues, with its empty negative branch absorbing with probability $1/2$; model B has bias -2 and stops at the gate. Their refined path supports are disjoint, so $\mathrm{BC}_{\text{refined}} = 0$. Collapsing the two absorption substeps instead gives $\mathrm{BC}_{\text{act-only}} = 1/\sqrt{2}$.

S.3.4 Proof of the Per-Pixel Decomposition

The following proves Theorem 8.1.

Proof. Apply the algebraic identity $(\sqrt{a} - \sqrt{b})^2 = a + b - 2\sqrt{ab}$ to each trajectory:

$$\mathrm{H}^2 \;=\; \tfrac{1}{2}\sum_{\zeta}\big(\sqrt{\Pi_A^{(0)}(\zeta)} - \sqrt{\Pi_B^{(0)}(\zeta)}\big)^2 \;=\; \tfrac{1}{2}\sum_{\zeta}\big[\Pi_A^{(0)}(\zeta) + \Pi_B^{(0)}(\zeta) - 2\sqrt{\Pi_A^{(0)}(\zeta)\Pi_B^{(0)}(\zeta)}\big]. \tag{S.3.21}$$

Group the trajectory sum by the terminal pixel $\zeta_0 = s$:

$$\mathrm{H}^2 \;=\; \tfrac{1}{2}\sum_{s\in\mathscr{S}_+^{(0)}}\Big[\sum_{\zeta:\zeta_0=s}\Pi_A^{(0)}(\zeta) + \sum_{\zeta:\zeta_0=s}\Pi_B^{(0)}(\zeta) - 2\sum_{\zeta:\zeta_0=s}\sqrt{\Pi_A^{(0)}(\zeta)\Pi_B^{(0)}(\zeta)}\Big]. \tag{S.3.22}$$

By definition, $\sum_{\zeta:\zeta_0=s}\Pi_M^{(0)}(\zeta) = m_M^{(0)}(s)$ for $M \in \{A,B\}$. Restricting Lemma S.3.1 to trajectories ending at s, with $g^{(l)}$ from Lemma S.3.2, gives $\sum_{\zeta:\zeta_0=s}\sqrt{\Pi_A^{(0)}(\zeta)\Pi_B^{(0)}(\zeta)} = g^{(0)}(s)$. Substituting,

$$\mathrm{H}^2 \;=\; \tfrac{1}{2}\sum_{s}\big[m_A^{(0)}(s) + m_B^{(0)}(s) - 2g^{(0)}(s)\big] \;=\; \sum_{s\in\mathscr{S}_+^{(0)}}\mathrm{H}^2(s), \tag{S.3.23}$$

proving (ii).

For (i), apply Cauchy–Schwarz to the sum defining $g^{(0)}(s)$:

$$g^{(0)}(s) = \sum_{\zeta:\zeta_0=s}\sqrt{\Pi_A^{(0)}(\zeta)\Pi_B^{(0)}(\zeta)} \le \sqrt{\Big(\sum_{\zeta:\zeta_0=s}\Pi_A^{(0)}(\zeta)\Big)\Big(\sum_{\zeta:\zeta_0=s}\Pi_B^{(0)}(\zeta)\Big)} = \sqrt{m_A^{(0)}(s)\,m_B^{(0)}(s)}. \tag{S.3.24}$$

Combined with the AM–GM inequality $\sqrt{m_A^{(0)}(s)m_B^{(0)}(s)} \le \frac{1}{2}(m_A^{(0)}(s) + m_B^{(0)}(s))$, this gives $g^{(0)}(s) \le \frac{1}{2}(m_A^{(0)}(s) + m_B^{(0)}(s))$, so $\mathrm{H}^2(s) \ge 0$. Equality holds iff $\Pi_A^{(0)}(\cdot \mid \zeta_0 = s) = \Pi_B^{(0)}(\cdot \mid \zeta_0 = s)$ *and* $m_A^{(0)}(s) = m_B^{(0)}(s)$.

Item (iii) is immediate from (ii) by separating the cemetery from the ordinary pixels. □

S.3.5 Conditioned-Survival Trajectory Laws

Theorem S.3.4 (Conditioned-survival trajectory law and conditioned Hellinger distance, `Lean_86`). *Let* $\mathsf{Surv} := \{(\zeta_L,\dots,\zeta_0) : \zeta_0 \in \mathscr{S}^{(0)}\}$ *in* $\prod_{l=0}^{L}\mathscr{S}_+^{(l)}$ *denote trajectories ending at an*

ordinary input state, equivalently never entering †. For $M \in \{A,B\}$ define the survival function

$$\psi_M^{(l)}(s) \;:=\; \mathbb{P}_M(\mathsf{Surv} \mid \zeta_l = s), \qquad s \in \mathscr{S}_+^{(l)},\;\; l \in \{0,\dots,L\}. \tag{S.3.25}$$

Then:

(i) The boundary and recursion are

$$\psi_M^{(0)}(s) = 1 \;\; \textit{for } s \in \mathscr{S}^{(0)}, \qquad \psi_M^{(0)}(\dagger) = 0, \tag{S.3.26}$$

and for every $l \geq 1$,

$$\psi_M^{(l)}(s) \;=\; \sum_{s' \in \mathscr{S}_+^{(l-1)}} T_M^{(l)}(s \to s')\, \psi_M^{(l-1)}(s') \;=\; \sum_{s' \in \mathscr{S}^{(l-1)}} T_M^{(l)}(s \to s')\, \psi_M^{(l-1)}(s'). \tag{S.3.27}$$

In particular,

$$Z_M \;:=\; \mathbb{P}_M(\mathsf{Surv}) \;=\; \psi_M^{(L)}(s_0) \;=\; \sum_{s \in \mathscr{S}^{(0)}} m_M^{(0)}(s) \;=\; 1 - m_M^{(0)}(\dagger). \tag{S.3.28}$$

(ii) Assume $Z_M > 0$. For every ordinary state $s \in \mathscr{S}^{(l)}$ with $\psi_M^{(l)}(s) > 0$, define the conditioned kernel on ordinary states by

$$T_{M,\mathrm{surv}}^{(l)}(s \to s') \;:=\; \frac{T_M^{(l)}(s \to s')\, \psi_M^{(l-1)}(s')}{\psi_M^{(l)}(s)}, \qquad s' \in \mathscr{S}^{(l-1)}. \tag{S.3.29}$$

Then $T_{M,\mathrm{surv}}^{(l)}(s \to \cdot)$ is row-stochastic on $\mathscr{S}^{(l-1)}$, and the conditional trajectory distribution $\Pi_{M,\mathrm{surv}} := \Pi_M(\,\cdot \mid \mathsf{Surv}\,)$ factorizes as the layered Markov law

$$\Pi_{M,\mathrm{surv}}(\zeta_L,\dots,\zeta_0) = \mathbf{1}\{\zeta_L = s_0\} \prod_{k=1}^{L} T_{M,\mathrm{surv}}^{(k)}(\zeta_k \to \zeta_{k-1}), \qquad (\zeta_L,\dots,\zeta_0) \in \mathsf{Surv}. \tag{S.3.30}$$

Its layer-l marginals satisfy

$$m_{M,\mathrm{surv}}^{(l)}(s) \;=\; \frac{m_M^{(l)}(s)\, \psi_M^{(l)}(s)}{Z_M}, \qquad s \in \mathscr{S}^{(l)}. \tag{S.3.31}$$

(iii) Define the conditioned geometric-mean kernel

$$\widetilde{T}_{\mathrm{surv}}^{(l)}(s,s') := \sqrt{T_{A,\mathrm{surv}}^{(l)}(s \to s')\, T_{B,\mathrm{surv}}^{(l)}(s \to s')} = \frac{\widetilde{T}^{(l)}(s,s')\, \sqrt{\psi_A^{(l-1)}(s')\, \psi_B^{(l-1)}(s')}}{\sqrt{\psi_A^{(l)}(s)\, \psi_B^{(l)}(s)}}. \tag{S.3.32}$$

The Bhattacharyya coefficient of the conditioned trajectory laws is

$$\mathrm{BC}\big(\Pi_{A,\mathrm{surv}}, \Pi_{B,\mathrm{surv}}\big) \;=\; \sum_{(\zeta_L,\dots,\zeta_0) \in \mathsf{Surv}} \mathbf{1}\{\zeta_L = s_0\} \prod_{k=1}^{L} \widetilde{T}_{\mathrm{surv}}^{(k)}(\zeta_k, \zeta_{k-1}), \tag{S.3.33}$$

so it is computed by the same Hellinger pass (Algorithm 8.1), but on the ordinary-state graph with the conditioned kernel.

(iv) Let

$$\mathrm{BC}_{\mathrm{ord}}^{(0)} \coloneqq \sum_{s\in\mathscr{S}^{(0)}} g^{(0)}(s) = \sum_{(\zeta_L,\dots,\zeta_0)\in\mathsf{Surv}} \sqrt{\Pi_A(\zeta_L,\dots,\zeta_0)\,\Pi_B(\zeta_L,\dots,\zeta_0)}. \tag{S.3.34}$$

Then

$$\mathrm{BC}\big(\Pi_{A,\mathrm{surv}},\Pi_{B,\mathrm{surv}}\big) = \frac{\mathrm{BC}_{\mathrm{ord}}^{(0)}}{\sqrt{Z_A Z_B}}, \tag{S.3.35}$$

and the conditioned-survival Hellinger distance is

$$\mathrm{H}_{\mathrm{surv}}^2 \coloneqq \mathrm{H}^2\big(\Pi_{A,\mathrm{surv}},\Pi_{B,\mathrm{surv}}\big) = 1 - \frac{\mathrm{BC}_{\mathrm{ord}}^{(0)}}{\sqrt{Z_A Z_B}}. \tag{S.3.36}$$

Proof. For (i), every ordinary layer-0 state survives and the cemetery does not, giving (S.3.26). For $l \geq 1$, conditioning on the next step gives

$$\psi_M^{(l)}(s) = \sum_{s'\in\mathscr{S}_+^{(l-1)}} T_M^{(l)}(s\to s')\,\mathbb{P}_M(\mathsf{Surv}\mid \zeta_{l-1}=s') = \sum_{s'\in\mathscr{S}_+^{(l-1)}} T_M^{(l)}(s\to s')\,\psi_M^{(l-1)}(s'). \tag{S.3.37}$$

Since $\psi_M^{(l-1)}(\dagger) = 0$, the cemetery term drops out and (S.3.27) follows. Evaluating at s_0 gives $Z_M = \psi_M^{(L)}(s_0)$. The identities with $m_M^{(0)}$ are just the layer-0 marginal of the event Surv.

For (ii), row-stochasticity follows from (S.3.27):

$$\sum_{s'\in\mathscr{S}^{(l-1)}} T_{M,\mathrm{surv}}^{(l)}(s\to s') = \frac{1}{\psi_M^{(l)}(s)} \sum_{s'\in\mathscr{S}^{(l-1)}} T_M^{(l)}(s\to s')\,\psi_M^{(l-1)}(s') = 1. \tag{S.3.38}$$

Now fix $(\zeta_L,\dots,\zeta_0)\in\mathsf{Surv}$. By definition of conditional probability,

$$\Pi_{M,\mathrm{surv}}(\zeta_L,\dots,\zeta_0) = \frac{\Pi_M(\zeta_L,\dots,\zeta_0)}{Z_M} = \frac{\mathbf{1}\{\zeta_L=s_0\}}{Z_M}\prod_{k=1}^{L} T_M^{(k)}(\zeta_k\to\zeta_{k-1}). \tag{S.3.39}$$

Multiplying the conditioned kernels and telescoping the survival factors gives

$$\prod_{k=1}^{L} T_{M,\mathrm{surv}}^{(k)}(\zeta_k\to\zeta_{k-1}) = \prod_{k=1}^{L} T_M^{(k)}(\zeta_k\to\zeta_{k-1})\frac{\psi_M^{(k-1)}(\zeta_{k-1})}{\psi_M^{(k)}(\zeta_k)} \tag{S.3.40}$$

$$= \frac{\psi_M^{(0)}(\zeta_0)}{\psi_M^{(L)}(s_0)}\prod_{k=1}^{L} T_M^{(k)}(\zeta_k\to\zeta_{k-1}) \tag{S.3.41}$$

$$= \frac{1}{Z_M}\prod_{k=1}^{L} T_M^{(k)}(\zeta_k\to\zeta_{k-1}), \tag{S.3.42}$$

because $\zeta_0\in\mathscr{S}^{(0)}$ implies $\psi_M^{(0)}(\zeta_0)=1$. This is (S.3.30). For the marginals,

$$m_{M,\mathrm{surv}}^{(l)}(s) = \mathbb{P}_M(\zeta_l=s\mid\mathsf{Surv}) = \frac{\mathbb{P}_M(\zeta_l=s,\mathsf{Surv})}{Z_M} = \frac{m_M^{(l)}(s)\,\psi_M^{(l)}(s)}{Z_M}, \tag{S.3.43}$$

which is (S.3.31).

For (iii), substitution of (S.3.29) gives (S.3.32). Lemma S.3.1 applied to (S.3.30) on the ordinary-state graph then gives (S.3.33).

For (iv), expand the Bhattacharyya coefficient of the conditioned laws:

$$\mathrm{BC}\big(\Pi_{A,\mathrm{surv}},\Pi_{B,\mathrm{surv}}\big) = \sum_{(\zeta_L,\dots,\zeta_0)\in\mathsf{Surv}} \sqrt{\frac{\Pi_A(\zeta_L,\dots,\zeta_0)}{Z_A}\,\frac{\Pi_B(\zeta_L,\dots,\zeta_0)}{Z_B}} \tag{S.3.44}$$

$$= \frac{1}{\sqrt{Z_A Z_B}} \sum_{(\zeta_L,\dots,\zeta_0)\in\mathsf{Surv}} \sqrt{\Pi_A(\zeta_L,\dots,\zeta_0)\,\Pi_B(\zeta_L,\dots,\zeta_0)}. \tag{S.3.45}$$

Grouping by the ordinary terminal state $\zeta_0 = s$ and using $g^{(0)}(s)$ as in Theorem 8.1 gives (S.3.34) and (S.3.35). Applying $\mathrm{H}^2 = 1 - \mathrm{BC}$ to the conditioned laws gives (S.3.36). □

Implementation consequence. Theorem S.3.4 gives two equivalent computations:

(a) *Kernel-side conditioning:* replace each row by the reweighted kernel (S.3.29), form the geometric mean (S.3.32), and run the ordinary Hellinger backward pass on $\mathscr{S}^{(L)} \times \cdots \times \mathscr{S}^{(0)}$.

(b) *Post-hoc normalization:* run the original pass once, extract the ordinary overlap $\mathrm{BC}^{(0)}_{\mathrm{ord}} = \sum_{s\in\mathscr{S}^{(0)}} g^{(0)}(s)$ and the survival masses Z_A, Z_B, and apply (S.3.36).

Route (a) exposes the conditioned law; (b) is cheaper for the scalar alone and becomes numerically infeasible in deep networks when either survival probability Z_M, $M \in \{A,B\}$, is too small.

Conditioned disagreement map. Applying Theorem 8.1 to the conditioned kernels $T^{(l)}_{M,\mathrm{surv}}$ yields the conditioned disagreement map

$$\mathrm{H}^2_{\mathrm{surv}}(s) \;:=\; \tfrac{1}{2}\big(m^{(0)}_{A,\mathrm{surv}}(s) + m^{(0)}_{B,\mathrm{surv}}(s)\big) - g^{(0)}_{\mathrm{surv}}(s), \qquad s \in \mathscr{S}^{(0)}, \tag{S.3.46}$$

which is non-negative and satisfies $\sum_{s\in\mathscr{S}^{(0)}} \mathrm{H}^2_{\mathrm{surv}}(s) = \mathrm{H}^2_{\mathrm{surv}}$.

S.3.6 Randomization Bounds

Lemma S.3.5 (Alive-set mismatch, `Lean_87`). *Suppose a dead state sends all its mass to the cemetery. Every state in the alive symmetric difference then has zero geometric-mean mass to ordinary successors: $\sum_{s'\in\mathscr{S}^{(l-1)}} \widetilde{T}^{(l)}(s_i,s') = 0$. If alive states have no cemetery leakage, as for the stated nondegenerate Stopping-Game kernel, its full geometric-mean row is also zero.*

Proof. If i is alive in A and dead in B, then $T^{(l)}_B(s_i,s') = 0$ for every ordinary successor, so each ordinary geometric mean is zero. The argument is symmetric in A,B. Zero cemetery mass in the alive row also annuls the cemetery geometric mean; with leakage, only the ordinary-state conclusion follows. □

Fix a ReLU MLP with hidden widths $n_1, \dots, n_L$, an input x, and start state s_0 at the target output. Let B replace $W^{(l^*)}$ by an i.i.d. matrix from a continuous distribution symmetric about zero, and assume that for each $l \geq l^*$ and every neuron i, $\mathbb{P}_B\big(i \in \mathrm{alive}_B^{(l)} \mid \text{kernels above layer } l\big) \leq 1 - \delta$ for some $\delta \in (0, \frac{1}{2}]$. Then $T_A^{(l)} = T_B^{(l)}$ for $l < l^*$; at $l = l^*$ the row weights differ and the alive sets may differ; for $l > l^*$ the perturbation propagates through the activations. The same reduction applies to convolutional and transformer blocks interpreted as single *Layered Markov Process* (LMP) layers. Write $\eta := 1 - \delta \in \big[\frac{1}{2}, 1\big)$ and $N := L - l^* + 1$.

Theorem S.3.6 (Randomization in the Stopping Game, `Lean_88`). $\mathbb{E}_B\big[\mathrm{BC}_{\mathrm{ord}}^{(0)}(\Pi_A, \Pi_B)\big] \leq \eta^N$. *Hence, for every $z > 0$, the conditioned-survival distance of Theorem S.3.4 satisfies* $\mathbb{E}_B\big[\mathrm{H}_{\mathrm{surv}}^2\, \mathbf{1}\{Z_A Z_B \geq z\}\big] \geq \mathbb{P}_B(Z_A Z_B \geq z) - \eta^N/\sqrt{z}$, *while the plain distance is* $\mathrm{H}^2 = 1 - \mathrm{BC}_{\mathrm{ord}}^{(0)} - g^{(0)}(\dagger)$.

Proof. At a layer $l \geq l^*$, ordinary overlap mass at a state dead in both models moves into the common cemetery, mass on the alive symmetric difference vanishes (Lemma S.3.5), and on states alive in both models the row mass is at most 1 by Cauchy–Schwarz. Hence $\mathrm{BC}_{\mathrm{ord}}^{(l-1)} \leq \sum_{s \in \mathscr{S}^{(l)}} g^{(l)}(s)\, \mathbf{1}\{s \in \mathrm{alive}_B^{(l)}\}$, and, since $g^{(l)}$ is fixed by the kernels above layer l, the conditional bound gives $\mathbb{E}_B[\mathrm{BC}_{\mathrm{ord}}^{(l-1)}] \leq \eta\, \mathbb{E}_B[\mathrm{BC}_{\mathrm{ord}}^{(l)}]$. Iterating from $\mathrm{BC}_{\mathrm{ord}}^{(L)} = 1$ over $l = L, \dots, l^*$ and using that $\mathrm{BC}_{\mathrm{ord}}^{(l)}$ is non-increasing below l^* yields the bound. The $\mathrm{H}_{\mathrm{surv}}$ statement follows from $\mathrm{H}_{\mathrm{surv}}^2 = 1 - \mathrm{BC}_{\mathrm{ord}}^{(0)}/\sqrt{Z_A Z_B} \geq 1 - \mathrm{BC}_{\mathrm{ord}}^{(0)}/\sqrt{z}$ on $\{Z_A Z_B \geq z\}$ (Theorem S.3.4), the plain one from $\mathrm{BC}^{(0)} = \mathrm{BC}_{\mathrm{ord}}^{(0)} + g^{(0)}(\dagger)$. □

Remark S.3.2 (Cemetery floor). Theorem S.3.6 bounds only the ordinary-state overlap. Overlap mass that reaches a neuron dead in both models enters the cemetery in both,

$$g^{(l-1)}(\dagger) = g^{(l)}(\dagger) + \sum_{s \in \mathrm{dead}_A^{(l)} \cap \mathrm{dead}_B^{(l)}} g^{(l)}(s),$$

and is never thinned again, so plain H saturates below one. In a mean-field approximation with uniform dead fraction q under A and alive probability p under B, the ordinary overlap shrinks by $(1-q)p$ per layer while a fraction $q(1-p)$ is absorbed, so $g^{(0)}(\dagger) \to q(1-p)/(1-(1-q)p)$ as $N \to \infty$; for illustrative values $q = 0.2$, $p = \frac{1}{2}$ this gives $\mathrm{H} \approx 0.91$, the same order as the observed 0.94–0.97 (Table 8.3). The conditioned-survival distance removes the cemetery floor exactly, and $\mathbb{E}_B[\mathrm{H}_{\mathrm{surv}}^2 \mid Z_A Z_B \geq z] \to 1$ for every fixed $z > 0$ with $\mathbb{P}_B(Z_A Z_B \geq z)$ bounded away from zero. For the Routing Game the activation-weighted kernel concentrates $g^{(l)}$ on neurons with largest $|aW|$, which rarely coincide with $\mathrm{dead}_A \cap \mathrm{dead}_B$, so H is nearly identical to $\mathrm{H}_{\mathrm{surv}}$ in practice.

Theorem S.3.7 (Randomization in the Routing Game, `Lean_89`). *Under the same setup without the alive-set assumption, take the RG kernel at temperature $\tau > 0$ and assume that for every $l \geq l^*$ each state alive in both models has row mass $\sum_{s'} \widetilde{T}^{(l)}(s, s') \leq \bar{b}$ for some $0 \leq \bar{b} < 1$. Then* $\mathrm{BC}_{\mathrm{ord}}^{(0)}(\Pi_A, \Pi_B) \leq \bar{b}^N$.

Proof. Since $a_{M,i}^{(l-1)} = 0$ for $i \notin \mathrm{alive}_M^{(l-1)}$, the Gibbs policy (S.3.7) sends no mass to dead neurons, so for $1 \leq l < L$ the overlap $g^{(l)}$ vanishes outside $\mathrm{alive}_A^{(l)} \cap \mathrm{alive}_B^{(l)}$, and the start state is alive in both, in the symmetric difference or dead in both. In the latter two cases its ordinary overlap contribution is zero by the ordinary-state part of Lemma S.3.5. Hence $\mathrm{BC}_{\mathrm{ord}}^{(l-1)} \leq \bar{b}\, \mathrm{BC}_{\mathrm{ord}}^{(l)}$ for every $l \geq l^*$, with no alive-set factor (the alive mismatch at layer $l-1$ is already part of the row masses at layer l); iterate as in Theorem S.3.6. The assumption reflects the structure of the RG: each act-to-act row

factors as the source-deadness indicator $\mathbf{1}\{z_j^{(l)} > 0\}$, the constant $\frac{1}{2}$ uniform κ-branch, and the Gibbs policy $\pi_{M,\kappa}^{(l)}(\cdot \mid j) \propto [a_M^{(l-1)}[\cdot\,] W_M^{(l,\kappa)}[\cdot, j]]^{1/\tau}$. At $l = l^*$ the two policies share $a^{(l^*-1)}$, while their weights may differ; above that layer their activation factors may also differ. These changes can reduce overlap, but a continuous randomization law alone does not imply distinct normalized rows almost surely. For example, if each sign branch has just one positive-mass ordinary route, changing its weight without changing its sign leaves that branch's normalized policy unchanged. The strict conditional row-overlap bound $\bar{b} < 1$ is therefore an explicit hypothesis of this theorem; its applicability requires a separate argument for the architecture and randomization law, or an empirical assessment. The equality case of Cauchy–Schwarz explains strict contraction when the completed normalized rows actually differ. □

S.4 Extended Experimental Results and Protocol

This chapter extends both experimental campaigns: the hidden-monotonicity split-method evaluation and the revised game-paper protocol, confidence intervals, validation selections, ablations and cross-dataset results.

S.4.1 Full Evaluation Results for the Split Methods

This section reports all methods evaluated in the split-method setup of Section 7.1.

S.4.1.1 VGG-16 Qualitative Saliency Map Comparison

Figure S.4.1 extends Figure 7.1. SplitCAM at layer 26 again covers classified objects more fully than LayerCAM at that layer. SplitGrad at layer 0 detects too many input edges to explain the decision usefully, especially amid background details such as the rubble in the wood rabbit image. This agrees with the performance peaks of both methods in deeper layers.

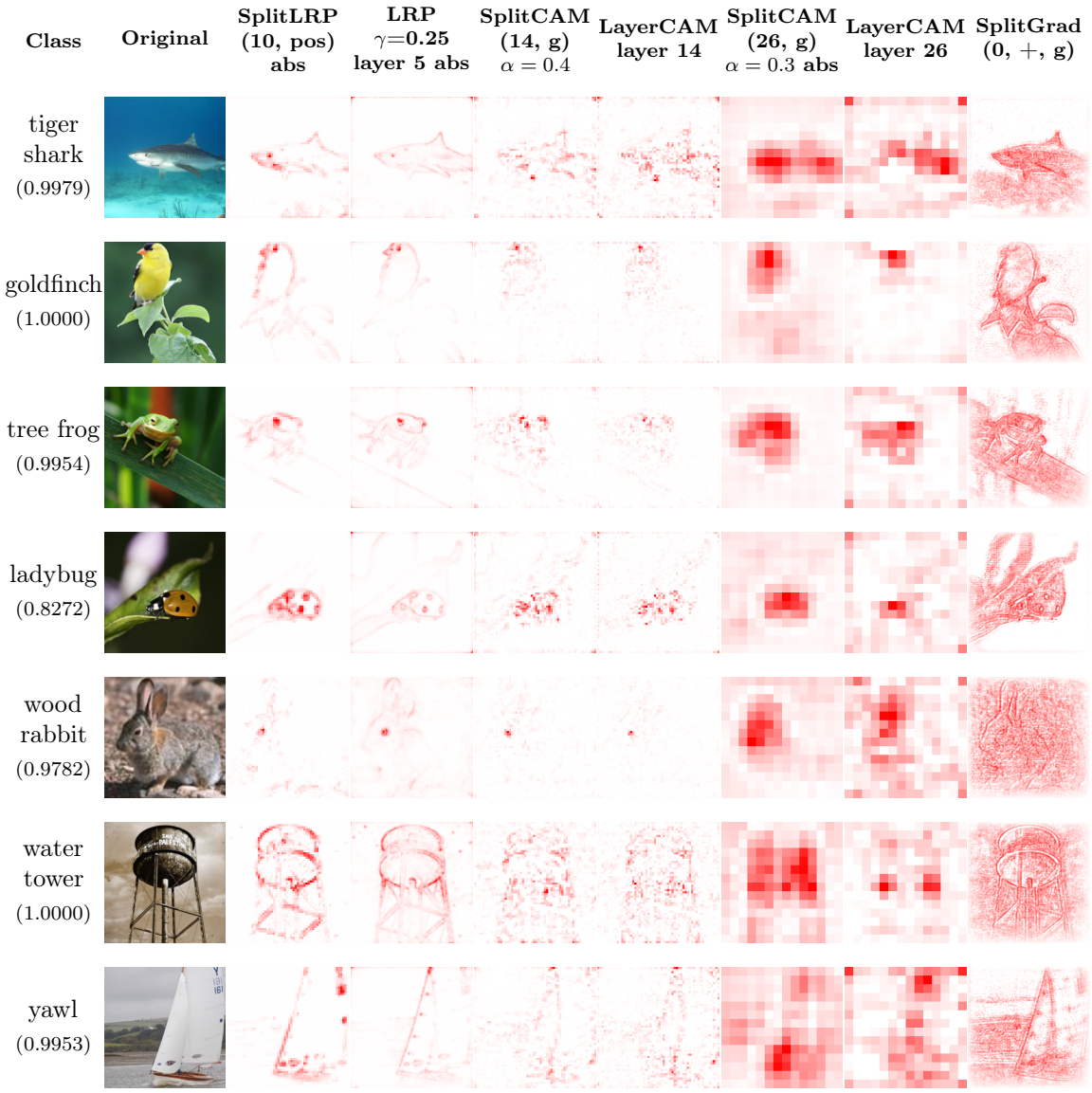


Fig. S.4.1: Saliency map comparison for VGG-16 on images from the custom ImageNet-S test subset. Each row shows the predicted class (with confidence), original image, and saliency maps from different methods. Methods include SplitLRP, classical LRP, SplitCAM variants, LayerCAM, and SplitGrad.

S.4.1.2 Quantitative Analysis on VGG-16

Table S.4.1 reports all VGG-16 split configurations, baselines and classical methods. SplitCAM's best localization scores exceed the listed LayerCAM rows, and its Maximum Sensitivity values are lower; LayerCAM at layer 14 has higher Pixel-Flipping score-drop AUC. Selected-row comparisons also vary the layer or absolute-value convention. The controlled α sweeps (Section S.4.2) expose these trade-offs more directly; neither comparison establishes uniform improvement across metrics.

Tab. S.4.1: Comprehensive interpretability evaluation results (VGG-16; representative configurations, best per method). Abbreviations as in Table 7.1. For each method configuration the best metric values across the considered layer configurations are highlighted in bold with dark blue shading. PF^{drop} is score-drop AUC; n/r denotes a result not reported in the paper, and n/a an inapplicable entry.

Method	Layer	Abs	Sel↓	AL↑	PG↑	PF_5^{drop}↑	PF_{20}^{drop}↑	MaxS↓
SplitCAM Methods								
SplitCAM (sc, α 0.4)	(12, g)	n	4.720	**0.651**	**0.912**	0.213	1.794	**0.389**
SplitCAM (sc, α 0.4)	(14, g)	n	**2.727**	0.563	0.836	**0.300**	**2.404**	1.012
SplitCAM (sc, α 0.4, wta)	(14, g)	n	**2.761**	0.543	0.827	**0.303**	**2.414**	1.056
SplitCAM (sc, α 0.4, wta)	(26, +, g)	y	4.711	**0.635**	**0.938**	0.165	1.590	**0.456**
SplitCAM (sc, α 0.5, wta)	(14, g)	n	**2.766**	0.536	0.823	**0.303**	**2.416**	1.056
SplitCAM (sc, α 0.5, wta)	(26, +, g)	n	4.608	**0.607**	**0.903**	0.185	1.684	**0.566**
SplitGrad Methods								
SplitGrad (α 0.4)	(12, +, g)	n	5.633	**0.613**	**0.873**	0.148	1.461	**0.364**
SplitGrad (α 0.4)	(14, +, g)	y	**3.429**	0.533	0.820	**0.235**	**2.119**	0.851
SplitGrad (α 0.4, wta)	(12, +, g)	n	4.735	**0.635**	**0.883**	0.186	1.756	0.651
SplitGrad (α 0.4, wta)	(14, +, g)	y	**3.287**	0.540	0.818	**0.249**	**2.186**	0.857
SplitGrad (α 0.5, wta)	(12, +, g)	y	**3.379**	**0.551**	0.859	**0.261**	**2.220**	0.869
SplitGrad (α 0.5, wta)	(26, +, g)	y	4.948	0.543	**0.899**	0.162	1.527	**0.617**
SplitLRP Methods								
SplitLRP (sc)	(7, pos)	n	**3.062**	0.092	**0.857**	**0.296**	**2.290**	0.865
SplitLRP (sc)	(19, pos)	n	4.393	**0.600**	0.823	0.166	1.656	**0.291**
SplitLRP (sc, wta)	(7, pos)	n	**3.235**	-0.036	0.836	**0.296**	**2.291**	0.866
SplitLRP (sc, wta)	(26, pos)	y	5.168	**0.588**	**0.871**	0.151	1.473	**0.282**
Classical Methods (No Layer Optimization)								
Deconvolution	0	y	5.435	0.468	0.629	0.204	1.715	0.956
DeepLIFT	0	y	4.447	0.551	0.726	0.281	1.945	1.053
Feature Ablation	0	y	6.029	0.479	0.624	0.123	1.250	0.859
GradientSHAP	0	y	4.808	0.552	0.783	0.261	1.862	1.142
IntGrad	0	y	4.838	0.554	0.797	0.262	1.851	1.069
Occlusion	0	y	5.214	0.499	0.716	0.130	1.217	0.699
GCAM++	30	n	5.955	0.564	0.827	0.130	1.245	0.401
Classical Methods (Layer Optimization)								
Guided Backprop	2	n	**3.276**	**0.632**	0.883	**0.281**	**2.302**	0.571
Guided Backprop	5	n	3.448	0.614	**0.887**	0.240	2.091	**0.375**
Guided GCAM	0	n	**4.301**	**-0.053**	**0.857**	0.161	**1.459**	1.047
Guided GCAM	26	n	5.130	-2.635	0.843	**0.179**	1.415	**0.947**
GCAM	0	n	6.006	**0.356**	**0.638**	0.060	0.899	1.123
GCAM	14	n	**5.555**	0.306	0.580	**0.136**	**1.525**	**0.903**
LCAM	14	n/a	2.792	0.576	0.798	**0.317**	**2.474**	**2.656**
LCAM	2	n/a	**3.128**	**0.580**	**0.855**	0.300	2.191	5.508
LRP $\gamma = 0.25$	0	n	**3.189**	**0.615**	**0.680**	0.218	1.918	0.519
LRP $\gamma = 0.25$	14	n	3.593	0.577	0.611	**0.279**	**2.255**	**0.491**
Baseline Methods								
Baseline: Binary Mask	0	y	6.011	1.000	1.000	0.091	1.054	n/r
Baseline: Original Image	0	y	8.770	-1.699	0.572	0.061	0.619	n/r
Random Baseline	0	y	3.570	0.425	0.458	0.203	1.585	n/r

S.4.1.3 Quantitative Analysis on ResNet-18

Table S.4.2 reports all ResNet-18 split configurations, baselines and classical methods.

Tab. S.4.2: Comprehensive interpretability evaluation results (ResNet-18; representative configurations, best per method). Abbreviations as in Table 7.2. For each method configuration the best metric values across the considered layer configurations are highlighted in bold with dark blue shading. PF^{drop} is score-drop AUC; n/r denotes a result not reported in the paper, and n/a an inapplicable entry.

Method	Layer	Abs	Sel↓	AL↑	PG↑	PF_5^{drop}↑	PF_{20}^{drop}↑	MaxS↓
SplitCAM Methods								
SplitCAM (sc, α 0.3, wta)	(1.0.2, g)	n	**3.503**	0.487	0.747	**0.168**	**1.395**	1.277
SplitCAM (sc, α 0.3, wta)	(4.1.2, g)	n	5.654	**0.673**	**0.972**	0.081	0.880	**0.233**
SplitCAM (sc, α 0.5, wta)	(2.0.1, g)	n	**3.682**	-1.370	0.887	**0.158**	**1.447**	1.078
SplitCAM (sc, α 0.5, wta)	(4.1.1, +, g)	y	6.526	**0.611**	**0.956**	0.066	0.737	**0.187**
SplitGrad Methods								
SplitGrad (α 0.3, wta)	(1.0.2, g)	y	**4.449**	0.513	0.777	**0.149**	**1.266**	0.813
SplitGrad (α 0.3, wta)	(4.1.2, g)	y	5.663	**0.649**	**0.972**	0.082	0.882	**0.219**
SplitGrad (α 0.5, wta)	(3.1.1, g)	y	**3.588**	0.559	0.880	**0.130**	**1.366**	0.859
SplitGrad (α 0.5, wta)	(4.1.2, g)	y	5.690	**0.654**	**0.972**	0.083	0.886	**0.230**
SplitLRP Methods								
SplitLRP (sc, wta)	(conv1, pos)	y	**6.313**	**0.506**	**0.587**	**0.056**	**0.697**	0.755
SplitLRP (sc, wta)	(1.0.2, pos)	y	6.785	0.483	0.567	0.050	0.686	**0.696**
Classical Methods (No Layer Optimization)								
Deconvolution	conv1	y	5.474	0.478	0.666	0.151	1.177	0.618
GradientSHAP	conv1	y	5.269	0.527	0.779	0.166	1.227	1.074
GCAM++	4.1.2	n	6.054	0.556	0.945	0.077	0.838	0.140
IntGrad	conv1	y	5.265	0.528	0.777	0.167	1.226	0.994
Feature Ablation	conv1	n	4.655	0.136	0.684	0.188	1.388	1.077
Occlusion	conv1	n	5.300	0.179	0.726	0.130	1.030	0.785
Deconvolution	conv1	n	3.934	0.484	0.666	0.147	1.203	0.751
Classical Methods (Layer Optimization)								
GCAM	2.0.1	y	**4.003**	0.450	0.678	0.052	**1.024**	0.738
GCAM	4.1.2	n	6.070	**0.554**	**0.945**	**0.076**	0.829	**0.131**
Guided GCAM	4.0.2	n	4.929	**0.742**	0.866	0.119	1.008	0.687
Guided GCAM	4.1.2	n	**3.843**	0.717	**0.883**	**0.129**	**1.096**	**0.612**
LCAM	2.0.2	n	**3.690**	0.549	0.865	**0.144**	**1.428**	0.913
LCAM	4.1.1	n	5.661	**0.664**	**0.971**	0.083	0.890	**0.227**
LRP $\gamma = 0.25$	conv1	y	**3.550**	**0.656**	0.852	**0.213**	**1.737**	0.521
LRP $\gamma = 0.25$	4.1.1	n	5.649	0.630	**0.965**	0.081	0.877	**0.200**
Guided Backprop	conv1	n	**3.517**	**0.621**	0.867	**0.212**	**1.757**	0.459
Guided Backprop	3.1.2	n	3.874	0.582	**0.938**	0.121	1.307	**0.213**
Baseline Methods								
Baseline: Original Image	conv1	y	8.351	-1.699	0.572	0.049	0.435	n/r
Baseline: Binary Mask	conv1	y	5.964	1.000	1.000	0.067	0.739	n/r
Random Baseline	n/a	y	4.071	0.425	0.440	0.147	1.067	n/r

S.4.1.4 Randomization Sanity Check

The model-randomization sanity check (Adebayo et al., 2018) tests dependence on learned parameters versus input patterns surviving model corruption. For each method, maps from trained VGG-16 are compared with maps after Gaussian reinitialization of the last three feedforward layers; parameter-sensitive methods should show a substantial similarity drop. Given the shortcomings of top-down randomization discussed by Binder et al. (2023), this complements the primary trajectory-space diagnostic of Section 8.4. Similarity measures are (i) HOG similarity (Pearson correlation of histograms of oriented gradients), (ii) signed Spearman rank correlation, and (iii) SSIM; lower similarity indicates stronger parameter dependence.

Tab. S.4.3: Randomization sanity check for the split methods on VGG-16: mean similarity between attribution maps of the trained model and of a copy whose last three feedforward layers are randomly reinitialized, across the custom test subset of ImageNet-S. Lower similarity indicates stronger dependence on the learned parameters. Abbreviations as in Table 7.1.

Method	HOG ↓	Spearman ↓	SSIM ↓
Baselines			
Deconvolution	0.985	0.999	0.481
DeepLIFT	0.365	0.274	0.716
GradientSHAP	0.286	0.081	0.571
Input × Gradient	0.313	0.082	0.621
IntGrad	0.303	0.102	0.584
LCAM (layer 14)	0.688	0.123	0.417
LCAM (layer 2)	0.542	0.114	0.475
Saliency	0.188	0.416	0.353
SplitCAM			
SplitCAM (sc, α 0.4) (12, g)	0.736	0.729	0.202
SplitCAM (sc, α 0.4) (14, g)	0.724	0.194	0.174
SplitCAM (sc, α 0.4, wta) (14, g)	0.724	0.389	0.303
SplitCAM (sc, α 0.4, wta) (26, +, g) abs	0.728	0.342	0.319
SplitCAM (sc, α 0.5, wta) (14, g)	0.728	0.096	0.292
SplitCAM (sc, α 0.5, wta) (26, +, g)	0.804	0.186	0.148
SplitGrad			
SplitGrad (α 0.4) (12, +, g)	0.677	0.711	0.174
SplitGrad (α 0.4) (14, +, g) abs	0.704	0.197	0.091
SplitGrad (α 0.4, wta) (12, +, g)	0.706	0.454	0.174
SplitGrad (α 0.4, wta) (14, +, g) abs	0.721	0.211	0.199
SplitGrad (α 0.5, wta) (12, +, g) abs	0.725	0.238	0.206
SplitGrad (α 0.5, wta) (26, +, g) abs	0.795	0.149	0.306
SplitLRP			
SplitLRP (7, pos)	0.997	1.000	0.471
SplitLRP (19, pos)	0.998	1.000	0.562
SplitLRP (wta) (7, pos)	0.998	1.000	0.456
SplitLRP (wta) (26, pos) abs	0.994	0.999	0.446

In Table S.4.3, every reported SplitCAM and SplitGrad configuration has lower SSIM than every listed baseline; HOG and Spearman comparisons are mixed. This shows responses to the tested randomization without isolating stabilization's effect on numerical gradient growth. SplitLRP retains high HOG and Spearman similarity (approximately 0.99 and 1.00), while SSIM falls to 0.45–0.56. Attenuation by the ϵ-LRP rule could explain this behavior; the comparison alone does not identify its cause.

S.4.1.5 Computational Efficiency

Table S.4.4 reports mean attribution times on the custom 50-image ImageNet-S validation subset for ResNet-18 and VGG-16. Splitting times, averaged separately over 20 setups, are 0.033 s and 0.030 s, respectively. SplitCAM and SplitGrad take 0.33–0.44 s per image, versus 0.28–0.31 s for the row labeled Guided Grad-CAM. SplitCAM takes approximately 14–29 times as long as the corresponding ordinary Grad-CAM and LayerCAM rows; the split methods are much faster than the reported Feature Ablation and Occlusion configurations.

The released evaluator splits the model once per configuration and reuses it across images. The per-image timer covers attribution, helper postprocessing, copying and cleanup, excluding image/model loading, initial device transfer, target prediction and splitting. The helper computes maps for all spatial layer chunks from cached activations and gradients before selecting the output; this extra computation is timed. The shifting parameter α changes the backward rule; its runtime effect was not measured separately.

The historical timing artifacts lack hardware, precision and exact configuration IDs. The released VGG-16 `guided_grad_cam` configuration dispatches Guided LayerCAM, whereas the source table labels it Guided Grad-CAM. We retain the reported label and numbers, with this baseline identity unresolved. These timings describe the source implementation and do not bound other implementations' overhead.

Tab. S.4.4: Mean attribution time in seconds per image across the custom 50-image validation subset of ImageNet-S. Model Splitting Time is instead seconds per setup, averaged over 20 procedures and excluded from the attribution times. Timer scope and limits of the historical run metadata are described in Section S.4.1.5.

Method	ResNet-18	VGG-16
Model Splitting Time	0.0331	0.0295
Split Methods		
SplitCAM	0.4368	0.3348
SplitGrad	0.4284	0.3309
SplitLRP	0.5711	0.3204
Classical Methods		
GCAM	0.0216	0.0232
GCAM++	0.0157	0.0155
LCAM	0.0149	0.0124
Guided GCAM	0.3102	0.2808
LRP $\gamma = 0.25$	0.0599	0.0507
DeepLIFT	0.0203	0.0210
IntGrad	0.0070	0.0121
GradientSHAP	0.0084	0.0082
Guided Backprop	0.0253	0.0258
Deconvolution	0.0087	0.0074
Saliency	0.0060	0.0047
Input $\times$ Gradient	0.0055	0.0046
Feature Ablation	231.6631	192.2602
Occlusion	238.2302	201.0051

S.4.2 Stabilization Ablations for the Split Methods

These ablations examine forward/backward stabilization and the max-pooling implementation (Section 3.6).

S.4.2.1 Backward-Pass Stabilization

Table S.4.5 extends Table 7.3 with the full α-ablation for the selected SplitCAM and SplitGrad configurations. At layer 26, changing α from 0.5 to 0.4 improves both localization scores in every configuration; Pixel-Flipping score-drop AUC increases for SplitGrad and decreases for SplitCAM. Layer-14 configurations show smaller or adverse changes in some metrics. Low α often reduces Maximum Sensitivity alongside localization or Pixel-Flipping performance. Because α changes the backward rule, these sweeps conflate numerical stabilization with explanation choice. Convex and winner-takes-it-all max-pooling differ little near $\alpha = 0.5$ and more at lower values; their ranking depends on configuration and metric.

Unresolved source-table discrepancy. For SplitCAM with scaling, $\alpha = 0.4$, winner-takes-it-all pooling and absolute-value configuration $(26, +, g)$, Table S.4.1 reports $\mathrm{PF}_{20}^{\mathrm{drop}} = 1.590$ and MaxS=

0.456, versus 1.5849 and 0.4573 in Table S.4.5. These source-table values remain unchanged: the per-image results needed to resolve their discrepancy are unavailable, leaving its cause unknown.

Tab. S.4.5: Alpha ablation as performed in Section 7.4 across four SplitGrad and SplitCAM configurations of layers 14 and 26, respectively. The notation is similar to Table 7.1.

Method	Metric	α 0.50	α 0.45	α 0.40	α 0.35	α 0.30	α 0.00
SplitGrad (wta) (26, g), abs	PG↑	0.8975	0.9064	0.9470	**0.9558**	0.8887	0.4011
	AL↑	0.5434	0.5476	0.6051	**0.6169**	0.5346	0.4486
	MaxS↓	0.6160	0.6071	0.4399	0.2382	0.3344	**0.0283**
	PF_{20}^{drop} ↑	1.5270	1.5337	**1.5808**	1.4476	1.3171	1.2471
SplitGrad (26, g), abs	PG↑	0.8975	0.9064	**0.9470**	0.9099	0.2827	0.6413
	AL↑	0.5434	0.5476	**0.6037**	0.5216	0.4245	0.4563
	MaxS↓	0.6171	0.6102	0.4426	0.3864	0.0622	**0.0079**
	PF_{20}^{drop} ↑	1.5272	1.5333	**1.5876**	1.3702	0.2431	0.5755
SplitGrad (wta) (14, g), abs	PG↑	0.8198	0.8198	0.8198	**0.8322**	0.8127	0.7898
	AL↑	**0.5403**	0.5403	0.5402	0.5006	0.4936	0.4857
	MaxS↓	0.8579	0.8597	0.8584	0.0875	0.0677	**0.0481**
	PF_{20}^{drop} ↑	2.1887	**2.1894**	2.1869	1.8740	1.8304	1.7684
SplitGrad (14, g), abs	PG↑	0.8198	**0.8198**	0.8180	0.7951	0.8004	0.7968
	AL↑	0.5403	**0.5403**	0.5330	0.5054	0.4926	0.4847
	MaxS↓	0.8618	0.8584	0.8478	0.1103	0.0662	**0.0301**
	PF_{20}^{drop} ↑	**2.1894**	2.1891	2.1185	1.3425	1.2959	1.1927
SplitCAM (sc, wta) (26, +, g), abs	PG↑	0.8675	0.8675	**0.9382**	0.9329	0.6131	0.8551
	AL↑	0.5501	0.5535	0.6347	**0.6627**	0.4033	0.5133
	MaxS↓	0.6906	0.6820	0.4573	0.2645	0.2511	**0.1196**
	PF_{20}^{drop} ↑	1.6067	**1.6085**	1.5849	1.4989	0.8534	1.5071
SplitCAM (sc) (26, +, g), abs	PG↑	0.8940	0.8905	**0.9346**	0.6466	0.8251	0.8604
	AL↑	0.5595	0.5636	**0.6363**	0.4015	0.4977	0.5114
	MaxS↓	0.6738	0.6640	0.4520	0.3729	0.1102	**0.1062**
	PF_{20}^{drop} ↑	1.5902	**1.5978**	1.5906	0.9076	1.2784	1.3605
SplitCAM (sc, wta) (14, g)	PG↑	0.8251	0.8251	**0.8269**	0.7880	0.7809	0.7703
	AL↑	0.5363	0.5363	**0.5435**	0.5338	0.5286	0.5225
	MaxS↓	1.0566	1.0569	1.0489	**0.1620**	0.1625	0.1630
	PF_{20}^{drop} ↑	2.4128	2.4130	**2.4135**	1.9014	1.8613	1.8731
SplitCAM (sc) (14, g)	PG↑	0.8251	0.8251	**0.8428**	0.7756	0.7686	0.7756
	AL↑	0.5363	0.5363	**0.5626**	0.5363	0.5271	0.5208
	MaxS↓	1.0552	1.0593	1.0136	0.1629	0.1585	**0.1555**
	PF_{20}^{drop} ↑	2.4171	**2.4172**	2.4065	1.5032	1.5082	1.5139

S.4.2.2 Forward-Pass Stabilization

We assess forward stabilization for framework-selected SplitLRP layers and manually selected SplitCAM layers. The shifting baseline sets positive-neuron activations to half the original activation and negative-neuron activations to its negative half. Without forward stabilization, late VGG-16 SplitCAM activations reach the order of 10^{18}, with numerical errors of order 10^{9} despite per-layer

correction. Unstabilized SplitLRP produced nearly white maps with small, random, semantically meaningless artifacts; no quantitative metrics were computed.

Tables S.4.7 and S.4.8 report SplitCAM without forward stabilization and with shifting, respectively, using the configurations of Table S.4.5. Table S.4.6 compares SplitLRP shifting and scaling. Scaling improves all reported metrics for both layer-7 configurations. At later layers, shifting gives higher Attribution Localization and Pixel-Flipping score-drop AUC; scaling lowers Maximum Sensitivity. Pointing Game rankings reverse between configurations, supporting a configuration-dependent trade-off.

Tab. S.4.6: Scale (sc) vs. shift (sh) forward-pass stabilization for SplitLRP methods on VGG-16.

Method	Type	Sel↓	AL↑	PG↑	PF_5^{drop}↑	PF_{20}^{drop}↑	MaxS↓
SplitLRP (wta), 7	sc	**3.235**	**-0.036**	**0.836**	**0.296**	**2.291**	**0.866**
	sh	3.619	-0.610	0.698	0.260	2.070	0.914
SplitLRP (wta), 26, abs	sc	**5.168**	0.588	0.871	0.151	1.473	**0.282**
	sh	5.168	**0.628**	**0.877**	**0.161**	**1.478**	0.468
SplitLRP, 7	sc	**3.235**	**-0.036**	**0.836**	**0.296**	**2.291**	**0.866**
	sh	3.619	-0.695	0.698	0.260	2.070	0.914
SplitLRP, 19	sc	5.168	0.588	**0.871**	0.151	1.473	**0.282**
	sh	**4.955**	**0.629**	0.853	**0.197**	**1.692**	0.507

Tab. S.4.7: Alpha ablation for scale normalization across four SplitCAM configurations of layers 14 and 26. Here, in the forward pass *no* stabilization is applied. Notation and structure are similar to Table S.4.5. Light blue entries marked with * indicate better performance of the method compared to the default variant with scale stabilization in the forward pass.

Method	**Metric**	α 0.50	α 0.45	α 0.40	α 0.35	α 0.30	α 0.00
SplitCAM (sc, wta) (26, +, g), abs	PG↑	0.7686	0.7686	0.8632	**0.8712**	0.3441	0.5453
	AL↑	0.5374	0.5412	0.5943	**0.6118**	0.3595	0.4569
	MaxS↓	0.6578*	0.6507*	0.4841	0.2873	0.1918*	**0.0135***
	PF_{20}^{drop} ↑	1.2744	1.2859	**1.3209**	1.1792	0.4430	0.6527
SplitCAM (sc) (26, +, g), abs	PG↑	0.7565	0.7565	**0.8451**	0.3783	0.4507	0.4970
	AL↑	0.5346	0.5384	**0.5897**	0.3663	0.4420	0.4545
	MaxS↓	0.6604*	0.6512*	0.4985	0.3077*	0.0279*	**0.0043***
	PF_{20}^{drop} ↑	1.2680	1.2856	**1.3295**	0.5912	0.4906	0.5563
SplitCAM (sc, wta) (14, g)	PG↑	**0.8491***	0.8491*	0.8491*	0.7867	0.7847*	0.7686
	AL↑	0.5324	0.5321	**0.5406**	0.5306	0.5261	0.5200
	MaxS↓	1.0834	1.0814	1.0885	**0.2271**	0.2309	0.2384
	PF_{20}^{drop} ↑	2.4454*	2.4482*	**2.4486***	1.9588*	1.9193*	1.9349*
SplitCAM (sc) (14, g)	PG↑	0.7746	0.7767	**0.7847**	0.5855	0.5594	0.5573
	AL↑	0.4003	0.4144	**0.5517**	0.5300	0.5209	0.5137
	MaxS↓	1.5733	1.5627	1.5443	**1.2661**	1.3939	1.4272
	PF_{20}^{drop} ↑	2.1298	**2.1384**	2.1371	1.6669*	1.6793*	1.6740*

Tab. S.4.8: Alpha ablation for scaling normalization across four SplitCAM configurations of layers 14 and 26. Here, in the forward pass shift normalization is applied. Notation and structure are similar to Table S.4.5. Light blue entries marked with * indicate better performance of the method compared to the default variant with scale stabilization in the forward pass.

Method	Metric	α 0.50	α 0.45	α 0.40	α 0.35	α 0.30	α 0.00
SplitCAM (sc, wta) (26, +, g), abs	PG↑	0.8330	0.8149	0.7968	**0.9235**	0.6036	0.8592*
	AL↑	0.5242	0.5203	0.5147	**0.6549**	0.3968	0.5093
	MaxS↓	0.6180*	0.6182*	0.6509	0.3104	0.2791	**0.1665**
	PF_{20}^{drop} ↑	**1.6804***	1.6646*	1.5065	1.4962	0.8896*	1.5405*
SplitCAM (sc) (26, +, g), abs	PG↑	0.8330	0.8149	0.7988	0.5654	0.8431*	**0.8652***
	AL↑	**0.5242**	0.5203	0.5076	0.3736	0.4949	0.5094
	MaxS↓	0.6129*	0.6168*	0.6494	0.3820	0.1686	**0.1664**
	PF_{20}^{drop} ↑	**1.6804***	1.6646*	1.5105	0.8714	1.4289*	1.4765*
SplitCAM (sc, wta) (14, g)	PG↑	**0.8310***	0.8310*	0.8310*	0.7887*	0.7827*	0.7726*
	AL↑	0.5312	0.5312	**0.5376**	0.5315	0.5264	0.5203
	MaxS↓	1.0562*	1.0521*	1.0649	**0.1633**	0.1641	0.1642
	PF_{20}^{drop} ↑	**2.4412***	2.4412*	2.4400*	1.9306*	1.8953*	1.9059*
SplitCAM (sc) (14, g)	PG↑	0.8310*	0.8310*	**0.8431***	0.7746	0.7706*	0.7807*
	AL↑	0.5311	0.5312	**0.5580**	0.5337	0.5245	0.5183
	MaxS↓	1.0554	1.0568*	1.0198	0.1641	0.1587	**0.1570**
	PF_{20}^{drop} ↑	**2.4412***	2.4411*	2.4318*	1.5187*	1.5222*	1.5329*

S.4.3 Implementation and Evaluation Details

Section A.2 specifies data splits, targets, metrics, configuration selection and statistical tests.

Backbone and dataset provenance. All game-based experiments use the pretrained checkpoints shipped with the evaluation release of torchvision: VGG-16 uses `VGG16_Weights.IMAGENET1K_V1` (https://download.pytorch.org/models/vgg16-397923af.pth; recipe https://github.com/pytorch/vision/tree/main/references/classification#alexnet-and-vgg), ResNet-50 uses `ResNet50_Weights.IMAGENET1K_V2` (https://download.pytorch.org/models/resnet50-11ad3fa6.pth; recipe https://github.com/pytorch/vision/issues/3995#issuecomment-1013906621), and ViT-B/16 uses `ViT_B_16_Weights.IMAGENET1K_V1` (https://download.pytorch.org/models/vit_b_16-c867db91.pth; recipe https://github.com/pytorch/vision/tree/main/references/classification#vit_b_16). These ImageNet-1K classifiers were trained on ImageNet / ILSVRC (Deng et al., 2009; Russakovsky et al., 2015) and scored with Quantus (Hedström et al., 2023).

Licenses. ImageNet images (Deng et al., 2009) are used under the ImageNet terms of access (non-commercial research and educational use)[1]. The ImageNet-S mask release (Gao et al., 2023) does not state a separate mask license[2]; its images are ImageNet images and fall under the same terms. The Pascal VOC 2012 images are subject to the Flickr terms of use, as stated by the dataset providers[3]. The checkpoints are distributed with torchvision and were trained on ImageNet-1K. Libraries: PyTorch

[1] https://www.image-net.org/download-images.php.
[2] Authors' release: https://github.com/LUSSeg/ImageNet-S.
[3] https://www.robots.ox.ac.uk/~vgg/projects/pascal/VOC/voc2012/, Database Rights.

and torchvision (BSD-3-Clause), Quantus (GPL-3.0), zennit (GPL-3.0/LGPL-3.0), lxt, captum and scikit-image (BSD-3-Clause), pytorch-grad-cam (MIT); the reference code of TIBAV and GAE (MIT) is included with its license notice.

S.4.3.1 Methods

Gradient-based block. **Gradient** is raw input-gradient saliency (Simonyan et al., 2014): the maximum absolute gradient over color channels. **SmGrad** averages noisy gradient maps (SmoothGrad) (Smilkov et al., 2017). **IntGrad** uses Integrated Gradients (Sundararajan et al., 2017) with a black-image baseline. **DeepLIFT** propagates activation differences from a reference input using the Rescale rule (Shrikumar et al., 2017) (CNNs only). **DAVE** is the distribution-aware ViT gradient decomposition of Wróbel et al. (2026); our implementation additionally freezes the LayerNorm mean, whereas published DAVE differentiates through mean-centering. **SG**, our SG, belongs here because its occupation measure recovers the gradient (Theorem S.1.6).

LRP / relevance-propagation block. **LRP-ε** is stabilized Layer-wise Relevance Propagation (Bach et al., 2015); **LRP-γ** amplifies positive weights (Montavon et al., 2019). **AttnLRP** propagates transformer relevance through softmax, the bilinear attention product and the Value path (Achtibat et al., 2024). **RG** is our RG; **RG+Eq** adds equivariant Reynolds averaging, mainly to suppress ViT tokenization-grid artifacts. **TIBAV** (Chefer et al., 2021b), **GAE** (Chefer et al., 2021a) and Attention Rollout (**AR**) are transformer attention-map baselines.

CAM block. **GCAM**, **GCAM++** and **LCAM** denote Grad-CAM (Selvaraju et al., 2017), Grad-CAM++ (Chattopadhay et al., 2018) and LayerCAM (Jiang et al., 2021), respectively. LayerCAM weights class-activation evidence per location. All three use the CNNs' last convolutional block and bilinear upsampling.

Perturbation block. **LIME** is Local Interpretable Model-agnostic Explanations (Ribeiro et al., 2016); **KernelSHAP** estimates SHAP values by model-agnostic weighted regression (Lundberg and Lee, 2017). Both perturb SLIC superpixels (Achanta et al., 2012) and fit local surrogates, distinguishing them from the backward-propagation methods.

S.4.3.2 Declared validation grids

Table S.4.9 lists every method's grid. Functionally duplicate rows (e.g. different σ^2 when $\lambda = \lambda_{\text{sm}} = 0$) are evaluated once, yielding 2,021 configurations. On ViT-B/16, activation-side λ is fixed to 0, keeping GELU gates conditioned (Section 5.4). At attention–Value products, RG applies $\alpha\beta$-LRP-ε to the composite attention-product/Value-projection map (Theorem S.2.3; Section 5.5.3). Baselines without grids use their reference implementations' settings.

Tab. S.4.9: Per-method validation grids (all architectures unless stated). Symbols: Section S.4.4.

Method	Grid
Gradient	single config
SmGrad	N=50, noise level $\sigma/(x_{\max}-x_{\min}) \in \{0.1, 0.15, 0.2\}$, mean aggregation (3 configs)
IntGrad	black-image baseline, $N_{\text{steps}} \in \{20, 50, 100, 300\}$ (4 configs)
DeepLIFT	Rescale rule, single config (CNNs only)
DAVE	$N \in \{25, 50\} \times \theta_{\max} \in \{0, 10, 20\} \times s_{\max} \in \{0, 0.05, 0.1\} \times t_{\max} \in \{0, 0.25, 0.5\}$, horizontal flips with probability 0.5, averaged effective map multiplied by the input (54 configs; ViT-B/16 only)
SG	$\lambda \in \{-5, -2, -1, -0.3, 0\} \times \sigma^2 \in \{1, 2, 5, 10\}\times$ noisy gate $\tilde{\Phi} \in \{\text{off}, \text{on}\}$ (40 configs; VGG-16 only)
LRP-ε	$\varepsilon \in \{0.01, 0.05, 0.1, 0.25, 0.5, 1, 2\}$ (7 configs)
LRP-γ	$(\varepsilon, \gamma) \in \{0.1, 0.25, 0.5\} \times \{0.1, 0.25, 0.5, 1, 2\}$; ViT-B/16: γ on all Linear layers (15 configs); CNNs: both zennit's EpsilonGammaBox composite ($z^{\mathscr{B}}$ input layer, γ on convolutions, ε on dense layers) and γ on all layers (30 configs)
AttnLRP	γ-composite of Achtibat et al. (2024): reference setting $(\gamma_{\text{conv}}, \gamma_{\text{lin}}, \varepsilon_{\text{attn}})$=$(0.25, 0.05, 10^{-6})$ + univariate sweeps $\gamma_{\text{lin}} \in \{0.01, 0.025, 0.1, 0.25\}$, $\gamma_{\text{conv}} \in \{0.1, 0.5, 1\}$, $\varepsilon_{\text{attn}} \in \{0.01, 0.05, 0.25\}$ (11 configs; ViT-B/16 only)
TIBAV	single config (ViT-B/16 only)
GAE	single config (ViT-B/16 only)
AR	residual weight $w_r \in \{0, 0.25, 0.5, 0.75\}$ (4 configs; ViT-B/16 only)
RG	$\tau \in \{0.3, 0.4, \ldots, 1.0\}$, (α,β)=(2,1), $\sigma^2 \in \{1, 2, 5, 10\}$, $\varepsilon \in \{0.1, 0.25, 0.5, 1\}$; ViT-B/16: λ=0, $\lambda_{\text{sm}} \in \{-50, -10, -5, -2, -1, 0\}$, composite Value rule of Theorem S.2.3 (672 configs); CNNs: $\lambda \in \{-5, -2, -1, -0.3, 0\}$ (544 configs)
RG+Eq	$\tau \in \{0.3, 0.4, \ldots, 1.0\} \times \lambda_{\text{sm}} \in \{0, -10\}$, σ^2=2, ε=0.5, N=50, composite Value rule (16 configs; ViT-B/16 only)
GCAM, GCAM++, LCAM	last convolutional block (VGG-16: `features.29`, ResNet-50: `layer4`), single config (CNNs only)
LIME	80 SLIC superpixels, N=1000, ridge surrogate, exponential kernel of width 0.25 on the cosine distance of the binary superpixel vectors, removed superpixels filled with their mean color; fixed reference row
KernelSHAP	80 SLIC superpixels, N=$2M$+2048 for M superpixels, removed superpixels filled with the image mean; fixed reference row
Anchor	$\alpha\beta$-LRP anchor $(\alpha,\beta,\varepsilon,\tau)$=(2,1,0.5,1) of RG and RG+Eq, reported as the theorem-driven reference row (not selected)

S.4.3.3 Compute resources

Evaluation is inference-only; repeated backward passes across validation/test grids dominate its cost, including equivariant-Reynolds and noise averaging. Runtime was measured on one NVIDIA A100-SXM4-40GB at batch size 1. Sweeps used several GPUs of that type, with up to two processes per GPU. The full pipeline (validation, test, robustness, Pascal VOC, ablations and randomization diagnostics) required about 450 GPU-hours.

S.4.3.4 Code and correctness tests

The code (https://github.com/JayPiZimmermann/Playing_The_Network_Backward_Official_Repository) contains unit tests that pin the implemented backward rules to the statements of the thesis and supplement. Two of them concern the attention block.

`tests/test_attention_ab_recovery.py` runs the RG hooks at $\tau = 1$ on a small attention block with signed inputs, a Value bias and $\varepsilon > 0$, in float64, and compares the result with an independent implementation of $\alpha\beta$-LRP-ε on the composite map $X \mapsto A(XW_v^\top + b_v)$, i.e. the target rule (5.21) of Theorem S.2.3, for a single head and for a multi-head block in the layout of `nn.MultiheadAttention`; it also asserts that the result differs from the sequential variant of §5.5.1, which the released code does not offer (`av_rule` accepts only `'merged'`). `tests/test_hellinger_merged_attention.py` enumerates every backward trajectory of a small attention block under two models, at $\tau \in \{0.5, 1, 2\}$, and checks the per-input masses, the Bhattacharyya mass and the cemetery masses of the Hellinger pass (Section 8.3) against this brute-force reference; a pre-norm residual block built from `nn.MultiheadAttention` must give the same distance as the same block with its attention matrix supplied as a constant.

S.4.4 Additional Results

One pipeline generates the validation, test and VOC tables from the declared grids and selection rules of Section A.2.

Configuration symbols. The configuration columns of all tables use the following parameters. RG, RG+Eq and SG (Table 4.1): temperature τ, activation-side risk aversion λ, attention-logit risk aversion λ_{sm}, ADF input variance σ^2, LRP stabilizer ε, stream weights (α,β) (default (2,1)), noisy-observation gate $\tilde{\Phi}$ ("gate on/off"), and the number N of transformed inputs averaged by RG+Eq. SmoothGrad: N noisy samples with noise standard deviation σ relative to the input range (σ/range). Integrated Gradients: N_{steps} integration steps from a black-image baseline. DAVE: N transformed inputs with rotations up to $\theta_{\max}$ degrees, shifts up to $s_{\max}$ of the image extent, and variance-preserving Gaussian noise of interpolation weight up to $t_{\max}$. LRP-ε and LRP-γ: stabilizer ε and positive-weight boost γ; on the CNNs, LRP-γ is zennit's composite with the $z^{\mathscr{B}}$ rule at the input layer. AttnLRP: γ-rule parameters γ_{conv} and γ_{lin} of the convolutional and linear layers and the stabilizer $\varepsilon_{\text{attn}}$ of the attention layers. AR (attention rollout): residual weight w_r. LIME and KernelSHAP: N perturbation samples ($2M$+2048 for M superpixels) on 80 SLIC superpixels; LIME fits a ridge surrogate. DeepLIFT: Rescale rule. Grad-CAM, Grad-CAM++ and LayerCAM: the target layer (torchvision module name). Row labels in parentheses give the selection criterion (`loc`, `faith`); single-configuration methods have no criterion.

S.4.4.1 Validation selections

Tables S.4.10–S.4.12 give each method's `loc`- and `faith`-selected configurations on the 50-image validation split (Section A.2.3), with both rank-sums. Fixed configurations have no selection comparison; their rank-sums are n/a.

Tab. S.4.10: ViT-B/16: validation-selected configurations per method and criterion (`loc`: minimal rank-sum over AL, PG, TK; `faith`: over PF_5, PF_{20}, Sel; rank-sums over the configurations of one method, lower is better; the number of configurations is given after the method name). Symbols: Section S.4.4.

Method	Criterion	Selected configuration	Loc. rank-sum	Faith. rank-sum
Gradient (1)	fixed	fixed	n/a	n/a
SmGrad (3)	loc	N=50,σ/range=0.2	3.5	3
	faith	N=50,σ/range=0.2	3.5	3
IntGrad (4)	loc	N_{steps}=100, black baseline	3.5	4
	faith	N_{steps}=100, black baseline	3.5	4
LRP-ε (7)	loc	ε=0.25	5.5	7
	faith	ε=0.1	6.5	3
LRP-γ (15)	loc	ε=0.1,γ=2	6	25
	faith	ε=0.1,γ=0.1	32	12
AttnLRP (11)	loc	γ_{conv}=0.25,γ_{lin}=0.05,$\varepsilon_{\text{attn}}$=0.25	3	29
	faith	γ_{conv}=0.25,γ_{lin}=0.05,$\varepsilon_{\text{attn}}$=0.01	14	7
DAVE (54)	loc	N=50,$\theta_{\max}$=0,$s_{\max}$=0,$t_{\max}$=0.5	14	6
	faith	N=50,$\theta_{\max}$=0,$s_{\max}$=0,$t_{\max}$=0.5	14	6
TIBAV (1)	fixed	fixed	n/a	n/a
GAE (1)	fixed	fixed	n/a	n/a
AR (4)	loc	w_r=0.75	3.5	3
	faith	w_r=0.75	3.5	3
LIME (1)	fixed	N=1000, 80 superpixels, ridge	n/a	n/a
KernelSHAP (1)	fixed	N=2M+2048, 80 superpixels	n/a	n/a
RG (672)	loc	τ=0.4,λ_{sm}=$-$10,σ^2=5,ε=0.1	47	788
	faith	τ=0.7,λ_{sm}=$-$5,σ^2=5,ε=0.1	1156	22
	anchor	τ=1	1420	344
RG+Eq (16)	loc	τ=0.5,λ_{sm}=$-$10,σ^2=2,N=50	6.5	29
	faith	τ=0.7,λ_{sm}=$-$10,σ^2=2,N=50	14	11
	anchor	τ=1,N=50	47	31

Tab. S.4.11: VGG-16: validation-selected configurations per method and criterion, as in Table S.4.10.

Method	Criterion	Selected configuration	Loc. rank-sum	Faith. rank-sum
Gradient (1)	fixed	fixed	n/a	n/a
SmGrad (3)	loc	N=50,σ/range=0.1	3	3
	faith	N=50,σ/range=0.1	3	3
IntGrad (4)	loc	N_{steps}=20, black baseline	6	3
	faith	N_{steps}=20, black baseline	6	3
DeepLIFT (1)	fixed	Rescale	n/a	n/a
LRP-ε (7)	loc	ε=0.5	6	9
	faith	ε=1	6	4
LRP-γ (30)	loc	ε=0.5,γ=0.25, $z^{\mathscr{B}}/\gamma/\varepsilon$ composite	13	43
	faith	ε=0.25,γ=0.1	36	5
GCAM (1)	fixed	`features.29`	n/a	n/a
GCAM++ (1)	fixed	`features.29`	n/a	n/a
LCAM (1)	fixed	`features.29`	n/a	n/a
LIME (1)	fixed	N=1000, 80 superpixels, ridge	n/a	n/a
KernelSHAP (1)	fixed	N=2M+2048, 80 superpixels	n/a	n/a
SG (40)	loc	λ=-1,σ^2=5, gate on	37	10
	faith	λ=-0.3,σ^2=1, gate off	49.5	6
RG (544)	loc	τ=0.3,ε=0.1	78	194
	faith	τ=0.8,ε=0.1	173.5	5
	anchor	τ=1	339	60

Tab. S.4.12: ResNet-50: validation-selected configurations per method and criterion, as in Table S.4.10.

Method	Criterion	Selected configuration	Loc. rank-sum	Faith. rank-sum
Gradient (1)	fixed	fixed	n/a	n/a
SmGrad (3)	loc	N=50,σ/range=0.15	4	5
	faith	N=50,σ/range=0.1	8	4
IntGrad (4)	loc	N_{steps}=100, black baseline	3	6
	faith	N_{steps}=50, black baseline	7.5	6
DeepLIFT (1)	fixed	Rescale	n/a	n/a
LRP-ε (7)	loc	ε=0.5	5.5	9
	faith	ε=2	8	4
LRP-γ (30)	loc	ε=0.5,γ=2, $z^{\mathscr{B}}/\gamma/\varepsilon$ composite	12.5	33
	faith	ε=0.5,γ=0.5, $z^{\mathscr{B}}/\gamma/\varepsilon$ composite	36	22.5
GCAM (1)	fixed	`layer4`	n/a	n/a
GCAM++ (1)	fixed	`layer4`	n/a	n/a
LCAM (1)	fixed	`layer4`	n/a	n/a
LIME (1)	fixed	N=1000, 80 superpixels, ridge	n/a	n/a
KernelSHAP (1)	fixed	N=2M+2048, 80 superpixels	n/a	n/a
RG (544)	loc	τ=0.9,ε=1	19	224
	faith	τ=1,λ=−1,σ^2=1,ε=0.25	466	38
	anchor	τ=1	212.5	140

S.4.4.2 Test results with confidence intervals

Tables S.4.13–S.4.15 report all 565-image test rows (each method's `loc`/`faith` selections, anchors and fixed configurations), with 95% bootstrap confidence intervals and the tests of Section A.2.4. Bold/dark shading marks each column's best row; ≈/light shading marks rows without a significant difference from it. Per-image runtimes use the single-GPU benchmark of Section S.4.3.3.

Tab. S.4.13: ViT-B/16, ImageNet-S test split: all rows with 95% bootstrap confidence intervals. Metrics as in Table 7.4; configuration symbols and row labels: Section S.4.4.

Method	Config	AL↑	PG↑	TK↑	PF_5↓	PF_{20}↓	Sel↓	MaxS↓	AvgS↓
Gradient	fixed	0.411 [0.389,0.432]	0.361 [0.322,0.402]	0.386 [0.359,0.413]	8.45 [8.33,8.56]	8.08 [7.94,8.22]	6.033 [5.905,6.154]	1.290 [1.233,1.351]	0.922 [0.893,0.953]
SmGrad (loc/faith)	N=50,σ/range=0.2	0.517 [0.498,0.536]	0.706 [0.667,0.743]	0.675 [0.654,0.694]	8.10 [7.96,8.24]	7.02 [6.83,7.20]	5.029 [4.894,5.163]	0.597 [0.569,0.630]	0.468 [0.451,0.485]
IntGrad (loc/faith)	N_{steps}=100,black baseline	0.458 [0.437,0.479]	0.513 [0.473,0.554]	0.498 [0.472,0.524]	8.37 [8.24,8.48]	7.93 [7.78,8.08]	5.712 [5.577,5.842]	1.090 [1.041,1.140]	0.825 [0.796,0.855]
LRP-ε (loc)	ε=0.25	0.592 [0.575,0.610]	0.834 [0.804,0.864]	0.746 [0.728,0.764]	7.97 [7.81,8.11]	6.88 [6.69,7.07]	4.859 [4.733,4.983]	0.486 [0.468,0.505]	0.384 [0.372,0.395]
LRP-ε (faith)	ε=0.1	0.582 [0.564,0.599]	0.834 [0.804,0.864]	0.748 [0.730,0.765]	7.84 [7.68,7.99]	6.76 [6.57,6.96]	4.890 [4.763,5.015]	0.623 [0.597,0.650]	0.498 [0.482,0.515]
LRP-γ (loc)	ε=0.1,γ=2	0.644 [0.628,0.660]	0.873 [0.844,0.899]	0.816≈ [0.800,0.832]	7.93 [7.78,8.08]	6.64 [6.44,6.83]	4.451≈ [4.324,4.572]	0.300 [0.287,0.314]	0.218 [0.210,0.226]

continued on next page

Method	Config	AL↑	PG↑	TK↑	PF_5↓	PF_{20}↓	Sel↓	MaxS↓	AvgS↓
LRP-γ (faith)	ε=0.1,γ=0.1	0.614 [0.597,0.630]	0.844 [0.814,0.873]	0.779 [0.763,0.795]	7.73 [7.56,7.89]	6.40 [6.20,6.60]	4.603 [4.478,4.724]	0.394 [0.378,0.410]	0.309 [0.299,0.319]
AttnLRP (loc)	γ_{conv}=0.25, γ_{lin}=0.05, $\varepsilon_{\text{attn}}$=0.25	0.644 [0.628,0.660]	0.855 [0.825,0.883]	0.809 [0.794,0.825]	7.77 [7.61,7.93]	6.43 [6.23,6.62]	4.582 [4.452,4.706]	0.456 [0.430,0.484]	0.342 [0.324,0.364]
AttnLRP (faith)	γ_{conv}=0.25, γ_{lin}=0.05, $\varepsilon_{\text{attn}}$=0.01	0.629 [0.613,0.645]	0.850 [0.819,0.878]	0.798 [0.783,0.814]	**7.64** [7.47,7.81]	**6.12** [5.92,6.32]	**4.441** [4.314,4.563]	0.416 [0.395,0.438]	0.317 [0.304,0.331]
DAVE (loc/faith)	N=50,$\theta_{\max}$=0, $s_{\max}$=0,$t_{\max}$=0.5	0.491 [0.472,0.510]	0.717 [0.678,0.752]	0.612 [0.591,0.633]	8.12 [7.98,8.25]	7.37 [7.19,7.54]	5.386 [5.256,5.514]	0.781 [0.755,0.810]	0.643 [0.626,0.661]
TIBAV	fixed	0.615 [0.597,0.634]	0.878 [0.851,0.904]	0.782 [0.759,0.805]	8.13 [8.01,8.24]	7.60 [7.46,7.74]	4.906 [4.774,5.033]	1.062 [1.001,1.126]	0.512 [0.486,0.540]
GAE	fixed	0.533 [0.514,0.551]	0.743 [0.708,0.779]	0.649 [0.623,0.677]	8.13 [8.01,8.25]	7.62 [7.47,7.76]	5.098 [4.961,5.231]	0.525 [0.494,0.558]	0.368 [0.346,0.391]
AR (loc/faith)	w_r=0.75	0.447 [0.428,0.467]	0.565 [0.522,0.605]	0.537 [0.510,0.564]	8.44 [8.34,8.53]	8.06 [7.94,8.17]	5.254 [5.128,5.374]	0.208 [0.195,0.222]	0.122≈ [0.116,0.128]
LIME	N=1000,80 superpixels,ridge	0.535 [0.515,0.553]	0.897 [0.871,0.922]	0.686 [0.656,0.716]	8.36 [8.25,8.46]	7.68 [7.54,7.82]	5.376 [5.242,5.505]	0.937 [0.905,0.972]	0.668 [0.647,0.689]
KernelSHAP	N=2M+2048,80 superpixels	0.565 [0.548,0.583]	**0.961** [0.945,0.977]	0.823≈ [0.799,0.848]	8.29 [8.18,8.40]	7.50 [7.35,7.65]	5.140 [5.012,5.261]	0.897 [0.868,0.928]	0.650 [0.632,0.667]
RG anchor	τ=1	0.509 [0.490,0.528]	0.724 [0.685,0.759]	0.699 [0.678,0.720]	8.19 [8.05,8.31]	6.97 [6.78,7.15]	4.854 [4.732,4.972]	**0.166** [0.161,0.172]	0.120≈ [0.117,0.124]
RG (loc)	τ=0.4,λ_{sm}=−10, σ^2=5,ε=0.1	**0.711** [0.692,0.730]	0.825 [0.793,0.855]	0.788 [0.768,0.808]	8.27 [8.14,8.39]	7.48 [7.31,7.64]	5.109 [4.987,5.226]	0.729 [0.685,0.777]	0.494 [0.471,0.518]
RG (faith)	τ=0.7,λ_{sm}=−5, σ^2=5,ε=0.1	0.549 [0.531,0.567]	0.770 [0.736,0.804]	0.741 [0.721,0.760]	8.11 [7.97,8.24]	6.78 [6.59,6.97]	4.808 [4.686,4.925]	0.257 [0.245,0.268]	0.174 [0.168,0.179]
RG+Eq anchor	τ=1,N=50	0.520 [0.501,0.539]	0.761 [0.726,0.796]	0.752 [0.729,0.775]	8.41 [8.29,8.52]	7.72 [7.56,7.87]	4.864 [4.742,4.981]	0.170≈ [0.165,0.175]	**0.115** [0.112,0.118]
RG+Eq (loc)	τ=0.5,λ_{sm}=−10, σ^2=2,N=50	0.698 [0.680,0.717]	0.855 [0.827,0.883]	**0.836** [0.815,0.857]	8.39 [8.28,8.50]	7.80 [7.65,7.94]	5.010 [4.889,5.127]	0.441 [0.421,0.463]	0.301 [0.290,0.313]
RG+Eq (faith)	τ=0.7,λ_{sm}=−10, σ^2=2,N=50	0.654 [0.636,0.672]	0.812 [0.781,0.844]	0.820 [0.799,0.841]	8.39 [8.28,8.51]	7.71 [7.56,7.86]	4.856 [4.735,4.974]	0.310 [0.297,0.325]	0.199 [0.194,0.206]

Tab. S.4.14: VGG-16, ImageNet-S test split: all rows with 95% bootstrap confidence intervals, as in Table S.4.13.

Method	Config	AL↑	PG↑	TK↑	PF_5↓	PF_{20}↓	Sel↓	MaxS↓	AvgS↓
Gradient	fixed	0.522 [0.501,0.542]	0.738 [0.701,0.773]	0.688 [0.665,0.711]	14.47 [13.95,14.99]	10.96 [10.55,11.37]	6.470 [6.212,6.731]	1.212 [1.162,1.269]	0.947 [0.917,0.980]
SmGrad (loc/faith)	N=50,σ/range=0.1	0.533 [0.515,0.552]	0.782 [0.747,0.816]	0.723 [0.705,0.741]	14.12 [13.60,14.65]	9.24 [8.85,9.63]	5.768 [5.526,6.007]	0.641 [0.625,0.659]	0.523 [0.515,0.531]
IntGrad (loc/faith)	N_{steps}=20,black baseline	0.550 [0.530,0.571]	0.770 [0.735,0.804]	0.706 [0.683,0.730]	15.39 [14.86,15.95]	11.89 [11.46,12.32]	6.438 [6.174,6.694]	1.132 [1.089,1.179]	0.917 [0.890,0.945]
DeepLIFT	Rescale	0.542 [0.523,0.562]	0.701 [0.664,0.738]	0.654 [0.631,0.677]	13.05 [12.59,13.52]	9.32 [8.96,9.69]	6.049 [5.806,6.291]	1.193 [1.145,1.249]	0.948 [0.919,0.985]
LRP-ε (loc)	ε=0.5	0.613 [0.595,0.632]	0.832 [0.800,0.862]	0.745 [0.725,0.764]	12.63 [12.18,13.10]	8.61 [8.26,8.96]	5.685 [5.458,5.908]	1.173 [1.127,1.223]	0.918 [0.893,0.947]
LRP-ε (faith)	ε=1	0.639 [0.621,0.657]	0.809 [0.777,0.841]	0.744 [0.725,0.762]	**12.43** [11.98,12.89]	8.17 [7.83,8.50]	5.489 [5.270,5.704]	1.100 [1.059,1.143]	0.840 [0.820,0.861]
LRP-γ (loc)	ε=0.5,γ=0.25, $z^{\mathscr{B}}/\gamma/\varepsilon$ composite	0.601 [0.584,0.617]	0.692 [0.655,0.729]	0.704 [0.687,0.721]	14.83 [14.28,15.39]	10.07 [9.68,10.46]	**5.010** [4.799,5.219]	0.515 [0.489,0.542]	0.368 [0.353,0.384]
LRP-γ (faith)	ε=0.25,γ=0.1	0.615 [0.598,0.633]	0.793 [0.759,0.827]	0.698 [0.681,0.716]	12.66 [12.18,13.13]	**7.89** [7.56,8.23]	5.078≈ [4.867,5.285]	0.702 [0.675,0.733]	0.561 [0.543,0.580]
GCAM	`features.29`	0.557 [0.538,0.576]	0.830 [0.798,0.860]	**0.821** [0.798,0.844]	16.36 [15.79,16.93]	12.47 [12.01,12.93]	5.640 [5.407,5.870]	0.430 [0.412,0.449]	0.334 [0.320,0.349]

continued on next page

Method	Config	AL↑	PG↑	TK↑	PF_5↓	PF_{20}↓	Sel↓	MaxS↓	AvgS↓
GCAM++	`features.29`	0.551 [0.533,0.569]	0.823 [0.791,0.855]	0.814≈ [0.792,0.835]	16.68 [16.09,17.27]	12.87 [12.40,13.35]	5.700 [5.462,5.938]	0.314 [0.301,0.327]	0.243 [0.233,0.253]
LCAM	`features.29`	0.544 [0.526,0.562]	0.720 [0.683,0.756]	0.747 [0.725,0.768]	15.98 [15.42,16.54]	11.48 [11.05,11.91]	5.341 [5.113,5.565]	0.434 [0.416,0.452]	0.330 [0.319,0.341]
LIME	N=1000,80 superpixels,ridge	0.603 [0.585,0.622]	**0.972** [0.958,0.984]	0.814≈ [0.790,0.837]	16.66 [16.09,17.25]	12.62 [12.17,13.06]	5.785 [5.551,6.015]	0.848 [0.816,0.882]	0.621 [0.599,0.644]
KernelSHAP	N=2M+2048,80 superpixels	0.559 [0.541,0.577]	0.954≈ [0.936,0.970]	0.820≈ [0.795,0.844]	16.68 [16.10,17.27]	12.82 [12.37,13.26]	6.140 [5.912,6.366]	0.994 [0.955,1.035]	0.736 [0.712,0.761]
SG (loc)	λ=−1,σ^2=5,gate on	0.597 [0.577,0.616]	0.770 [0.735,0.804]	0.770 [0.747,0.793]	17.28 [16.69,17.87]	13.32 [12.85,13.79]	6.217 [5.969,6.459]	**0.278** [0.267,0.290]	**0.196** [0.188,0.204]
SG (faith)	λ=−0.3,σ^2=1,gate off	0.598 [0.577,0.618]	0.821 [0.789,0.851]	0.788≈ [0.764,0.812]	17.08 [16.50,17.67]	12.98 [12.53,13.44]	6.278 [6.025,6.524]	0.543 [0.516,0.571]	0.423 [0.401,0.446]
RG anchor	τ=1	0.619 [0.601,0.637]	0.759 [0.724,0.795]	0.691 [0.673,0.709]	14.27 [13.74,14.80]	9.49 [9.10,9.89]	5.179 [4.968,5.388]	0.551 [0.526,0.578]	0.398 [0.385,0.412]
RG (loc)	τ=0.3,ε=0.1	**0.685** [0.665,0.705]	0.736 [0.699,0.772]	0.673 [0.654,0.692]	14.48 [13.95,15.02]	9.64 [9.25,10.03]	5.632 [5.411,5.845]	1.349 [1.273,1.430]	0.879 [0.842,0.918]
RG (faith)	τ=0.8,ε=0.1	0.640 [0.623,0.658]	0.712 [0.673,0.749]	0.688 [0.670,0.706]	14.05 [13.53,14.58]	9.07 [8.70,9.46]	5.117 [4.906,5.325]	0.655 [0.621,0.690]	0.466 [0.448,0.485]

Tab. S.4.15: ResNet-50, ImageNet-S test split: all rows with 95% bootstrap confidence intervals, as in Table S.4.13.

Method	Config	AL↑	PG↑	TK↑	PF_5↓	PF_{20}↓	Sel↓	MaxS↓	AvgS↓
Gradient	fixed	0.481 [0.459,0.504]	0.556 [0.515,0.596]	0.568 [0.540,0.595]	6.33 [6.22,6.44]	5.91 [5.78,6.04]	5.027 [4.900,5.151]	1.448 [1.365,1.535]	1.072 [1.019,1.130]
SmGrad (loc)	N=50, σ/range=0.15	0.549 [0.530,0.569]	0.789 [0.756,0.821]	0.738 [0.719,0.755]	6.05 [5.93,6.17]	5.17 [5.02,5.31]	4.429 [4.303,4.554]	0.735 [0.718,0.753]	0.600 [0.593,0.609]
SmGrad (faith)	N=50,σ/range=0.1	0.544 [0.525,0.564]	0.773 [0.740,0.807]	0.735 [0.715,0.753]	6.06 [5.94,6.19]	5.19 [5.04,5.34]	4.489 [4.361,4.612]	0.726 [0.706,0.748]	0.586 [0.577,0.597]
IntGrad (loc)	N_{steps}=100,black baseline	0.510 [0.488,0.532]	0.635 [0.596,0.676]	0.600 [0.574,0.626]	6.27 [6.16,6.38]	5.83 [5.69,5.96]	4.949 [4.820,5.072]	1.203 [1.152,1.257]	0.947 [0.917,0.979]
IntGrad (faith)	N_{steps}=50,black baseline	0.507 [0.484,0.529]	0.604 [0.563,0.642]	0.595 [0.568,0.622]	6.28 [6.16,6.39]	5.82 [5.68,5.95]	4.952 [4.822,5.075]	1.207 [1.157,1.261]	0.951 [0.922,0.983]
DeepLIFT	Rescale	0.543 [0.523,0.564]	0.727 [0.690,0.765]	0.653 [0.629,0.677]	5.97 [5.85,6.10]	5.29 [5.15,5.43]	4.515 [4.390,4.637]	1.273 [1.203,1.351]	1.000 [0.955,1.052]
LRP-ε (loc)	ε=0.5	0.603 [0.584,0.622]	0.775 [0.740,0.809]	0.714 [0.694,0.734]	5.94 [5.81,6.06]	5.14 [4.99,5.28]	4.318 [4.200,4.433]	1.246 [1.203,1.292]	0.982 [0.960,1.006]
LRP-ε (faith)	ε=2	0.637 [0.618,0.656]	0.699 [0.662,0.736]	0.683 [0.663,0.702]	5.94 [5.81,6.06]	4.91 [4.76,5.05]	4.280 [4.164,4.394]	1.262 [1.224,1.301]	0.967 [0.948,0.986]
LRP-γ (loc)	ε=0.5,γ=2,$z^{\mathscr{B}}/\gamma/\varepsilon$ composite	0.711≈ [0.695,0.726]	0.842 [0.812,0.873]	0.822≈ [0.807,0.836]	5.69≈ [5.55,5.83]	4.53 [4.38,4.68]	3.935≈ [3.814,4.053]	0.633 [0.601,0.666]	0.486 [0.464,0.507]
LRP-γ (faith)	ε=0.5,γ=0.5, $z^{\mathscr{B}}/\gamma/\varepsilon$ composite	0.700 [0.684,0.716]	0.834 [0.802,0.864]	0.796 [0.781,0.812]	**5.65** [5.51,5.79]	**4.43** [4.28,4.59]	**3.931** [3.809,4.050]	0.745 [0.714,0.777]	0.584 [0.562,0.605]
GCAM	`layer4`	0.664 [0.645,0.683]	0.873 [0.844,0.899]	0.850≈ [0.830,0.870]	6.42 [6.32,6.52]	6.15 [6.03,6.26]	4.344 [4.216,4.469]	0.292 [0.274,0.312]	0.207 [0.195,0.220]
GCAM++	`layer4`	0.627 [0.607,0.646]	0.834 [0.804,0.864]	0.836 [0.815,0.857]	6.43 [6.34,6.52]	6.18 [6.07,6.29]	4.483 [4.356,4.609]	0.294 [0.278,0.310]	0.207 [0.197,0.219]
LCAM	`layer4`	0.649 [0.630,0.668]	0.869 [0.841,0.897]	**0.851** [0.831,0.871]	6.44 [6.34,6.53]	6.18 [6.07,6.29]	4.437 [4.311,4.559]	**0.274** [0.260,0.288]	**0.194** [0.184,0.204]
LIME	N=1000,80 superpixels,ridge	0.553 [0.533,0.572]	0.926≈ [0.903,0.947]	0.716 [0.687,0.744]	6.37 [6.26,6.46]	5.99 [5.88,6.11]	4.759 [4.630,4.881]	0.916 [0.891,0.941]	0.672 [0.655,0.690]
KernelSHAP	N=2M+2048,80 superpixels	0.558 [0.540,0.575]	**0.965** [0.949,0.979]	0.818≈ [0.793,0.841]	6.26 [6.16,6.36]	5.81 [5.69,5.92]	4.727 [4.601,4.849]	0.924 [0.896,0.954]	0.688 [0.670,0.708]
RG anchor	τ=1	0.693 [0.676,0.710]	0.752 [0.717,0.788]	0.756 [0.738,0.774]	5.84 [5.70,5.97]	4.71 [4.56,4.86]	4.017 [3.898,4.133]	0.708 [0.651,0.775]	0.508 [0.481,0.541]
RG (loc)	τ=0.9,ε=1	**0.719** [0.701,0.737]	0.763 [0.727,0.796]	0.760 [0.741,0.778]	5.85 [5.72,5.98]	4.70 [4.55,4.85]	4.047 [3.929,4.163]	0.798 [0.749,0.855]	0.560 [0.535,0.586]

continued on next page

Method	Config	AL↑	PG↑	TK↑	PF_5↓	PF_{20}↓	Sel↓	MaxS↓	AvgS↓
RG (faith)	τ=1,λ=−1,σ^2=1, ε=0.25	0.674 [0.656,0.691]	0.743 [0.708,0.779]	0.738 [0.719,0.756]	5.88 [5.74,6.01]	4.74 [4.59,4.89]	4.030 [3.911,4.144]	0.685 [0.629,0.755]	0.489 [0.460,0.525]

S.4.4.3 Trade-off sweeps

Table S.4.17 sweeps τ and λ_{sm} individually around the `loc`-selected ViT-B/16 RG configuration. Test evaluation follows selection and never feeds back into it.

S.4.4.4 Cross-Dataset Evaluation: Pascal VOC 2012

This protocol underlies the ViT-B/16 cross-dataset summary in Section 7.5 (all rows: Table S.4.16). It tests transfer of ImageNet-S-selected RG/RG+Eq configurations without re-tuning and compares baselines selected by the same per-method procedure on the new pool.

Dataset. We use Pascal VOC 2012 (Everingham et al., 2010) segmentation annotations[4]: the trainval pool of 2,913 images with pixel-accurate `SegmentationClass` masks across 20 foreground classes plus background. Each image is expanded to one (image, foreground-class) sample per distinct VOC class label present in its mask, yielding 3,422 total samples after dropping the `person` class (no faithful ImageNet-1K target exists for it) and discarding sliver masks whose pixel area on the *original* (pre-resize) annotation is below 64 pixels. Each (image, class) sample carries the same input image and an explanation target equal to the most semantically central ImageNet-1K index for that VOC class (e.g. `horse` → N02389026 SORREL; `tvmonitor` → N03782006 MONITOR); the curated mapping is part of the released code.

Per-image disjoint val/test split. The trainval pool is partitioned randomly with seed 42 at the *image* level (not the (image, class) level) into a 1:5 val/test split: the trainval stems are shuffled deterministically and the first 1/6 is assigned to val. All (image, class) samples for a given image stay in the same partition, preventing the same image from appearing in both tuning and test sets. As on ImageNet-S, samples whose object mask lies entirely outside the 224×224 center crop are excluded after the split (18 val and 105 test samples). The resulting partitions:

	Images	(image, class) samples
val (tuning)	460	556
test (eval)	2,293	2,743

Pixel masks for localization. *Attribution Localization* (AL), *Pointing Game* (PG), and Top-k Intersection use each image–class sample's binary VOC2012 segmentation mask, rewarding only foreground pixels rather than background within a bounding box. NEAREST resizing and 224×224 center-cropping match the input pipeline (`Resize(256)` → `CenterCrop(224)`); Quantus (Hedström et al., 2023) receives this mask unchanged alongside the attribution map.

[4]Release statistics: https://www.robots.ox.ac.uk/~vgg/projects/pascal/VOC/voc2012/dbstats.html.

Tuning protocol. The protocol mirrors the ImageNet-S protocol of Section A.2, with one deliberate asymmetry: RG and RG+Eq do *not* get a fresh grid search on VOC.

- **Baselines.** Every row of the per-method ViT-B/16 grid of Table S.4.9 is evaluated on the 556-sample val pool, and the `loc` and `faith` configurations are selected per method with the criteria of Section A.2.3. The selected rows are re-evaluated on the 2,743-sample test pool.
- **RG / RG+Eq (transfer test).** The anchor and the `loc`- and `faith`-selected ImageNet-S configurations are evaluated on the VOC test pool unchanged, without any VOC-side re-tuning.

VOC evaluates localization and faithfulness only, saving compute (no MaxS/AvgS or runtime column). Baselines are selected on VOC; game configurations remain fixed from ImageNet-S. Per-metric tests (Section A.2.4) count significant wins, losses and undetected differences; the last category does not establish equivalence.

Tab. S.4.16: ViT-B/16 on the Pascal VOC 2012 test pool (2,743 image–class records from 2,293 images): all rows with 95% confidence intervals from image-cluster bootstrap resampling. Means are weighted over records. RG / RG+Eq use the ImageNet-S-selected configurations without re-tuning; baselines are selected on the VOC val pool. Only localization and faithfulness metrics were evaluated; the unmeasured MaxS and AvgS columns are omitted. Metrics as in Table 7.4; configuration symbols and row labels: Section S.4.4.

Method	Config	AL↑	PG↑	TK↑	PF_5↓	PF_{20}↓	Sel↓
Gradient	fixed	0.295 [0.286,0.304]	0.360 [0.342,0.378]	0.355 [0.343,0.368]	3.45 [3.33,3.56]	3.17 [3.05,3.28]	2.316 [2.234,2.397]
SmGrad (loc)	N=50,σ/range=0.1	0.344 [0.334,0.354]	0.557 [0.538,0.576]	0.511 [0.498,0.523]	3.25 [3.13,3.36]	2.70 [2.60,2.81]	1.940 [1.871,2.011]
SmGrad (faith)	N=50, σ/range=0.15	0.347 [0.337,0.356]	0.555 [0.536,0.574]	0.511 [0.499,0.523]	3.24 [3.12,3.35]	2.67 [2.57,2.77]	1.907 [1.840,1.977]
IntGrad (loc)	N_{steps}=300,black baseline	0.292 [0.283,0.302]	0.388 [0.369,0.406]	0.358 [0.346,0.370]	3.43 [3.32,3.55]	3.21 [3.10,3.32]	2.278 [2.198,2.356]
IntGrad (faith)	N_{steps}=100,black baseline	0.292 [0.283,0.302]	0.389 [0.371,0.407]	0.358 [0.346,0.370]	3.44 [3.32,3.55]	3.21 [3.10,3.32]	2.279 [2.199,2.358]
LRP-ε (loc/faith)	ε=0.1	0.378 [0.368,0.388]	0.604 [0.585,0.623]	0.518 [0.506,0.531]	3.07 [2.96,3.18]	2.63 [2.54,2.73]	1.923 [1.856,1.990]
LRP-γ (loc/faith)	ε=0.1,γ=1	0.403 [0.393,0.414]	0.591 [0.572,0.610]	0.545 [0.532,0.558]	2.99 [2.88,3.10]	2.37 [2.27,2.46]	1.694≈ [1.634,1.756]
AttnLRP (loc/faith)	γ_{conv}=0.25, γ_{lin}=0.05, $\varepsilon_{\text{attn}}$=0.05	0.404 [0.393,0.414]	0.602 [0.583,0.620]	0.542 [0.529,0.554]	**2.91** [2.80,3.01]	**2.31** [2.22,2.40]	**1.690** [1.630,1.751]
DAVE (loc)	N=50,$\theta_{\max}$=0, $s_{\max}$=0,$t_{\max}$=0.25	0.334 [0.324,0.343]	0.547 [0.527,0.566]	0.456 [0.444,0.468]	3.37 [3.26,3.49]	3.01 [2.91,3.12]	2.172 [2.097,2.247]
DAVE (faith)	N=50,$\theta_{\max}$=0, $s_{\max}$=0,$t_{\max}$=0.5	0.332 [0.322,0.342]	0.535 [0.515,0.554]	0.454 [0.443,0.466]	3.37 [3.26,3.48]	2.98 [2.88,3.09]	2.153 [2.079,2.227]
TIBAV	fixed	0.413 [0.402,0.424]	0.729 [0.713,0.747]	**0.575** [0.561,0.590]	3.04 [2.93,3.15]	2.67 [2.57,2.77]	1.731 [1.665,1.798]
GAE	fixed	0.355 [0.345,0.365]	0.712 [0.695,0.729]	0.548 [0.535,0.562]	3.00 [2.89,3.11]	2.63 [2.52,2.74]	1.754 [1.685,1.824]
AR (loc/faith)	w_r=0.75	0.272 [0.263,0.281]	0.471 [0.452,0.490]	0.411 [0.397,0.425]	3.57 [3.45,3.68]	3.41 [3.30,3.51]	2.146 [2.075,2.216]
LIME	N=1000,80 superpixels,ridge	0.351 [0.341,0.361]	0.785≈ [0.768,0.800]	0.523 [0.508,0.538]	3.45 [3.33,3.56]	3.21 [3.10,3.32]	2.121 [2.051,2.194]
KernelSHAP	N=2M+2048,80 superpixels	0.341 [0.332,0.351]	**0.793** [0.777,0.809]	0.561≈ [0.546,0.576]	3.36 [3.25,3.48]	3.03 [2.93,3.14]	2.077 [2.009,2.147]
RG anchor	τ=1	0.323 [0.313,0.333]	0.517 [0.498,0.536]	0.489 [0.475,0.502]	3.30 [3.19,3.41]	2.67 [2.58,2.77]	1.980 [1.914,2.047]

continued on next page

Method	Config	AL↑	PG↑	TK↑	PF_5↓	PF_{20}↓	Sel↓
RG (loc)	τ=0.4,λ_{sm}=−10, σ^2=5,ε=0.1	**0.459** [0.447,0.472]	0.562 [0.543,0.582]	0.512 [0.498,0.527]	3.44 [3.33,3.56]	3.06 [2.96,3.17]	2.152 [2.083,2.222]
RG (faith)	τ=0.7,λ_{sm}=−5, σ^2=5,ε=0.1	0.351 [0.341,0.361]	0.538 [0.519,0.557]	0.508 [0.495,0.522]	3.27 [3.16,3.38]	2.63 [2.53,2.72]	1.965 [1.899,2.032]
RG+Eq anchor	τ=1,N=50	0.329 [0.319,0.339]	0.574 [0.554,0.593]	0.534 [0.519,0.549]	3.43 [3.31,3.54]	3.06 [2.96,3.17]	1.988 [1.922,2.056]
RG+Eq (loc)	τ=0.5,λ_{sm}=−10, σ^2=2,N=50	0.456 [0.443,0.468]	0.599 [0.579,0.619]	0.571≈ [0.555,0.586]	3.45 [3.34,3.57]	3.14 [3.03,3.25]	2.085 [2.016,2.154]
RG+Eq (faith)	τ=0.7,λ_{sm}=−10, σ^2=2,N=50	0.424 [0.412,0.436]	0.591 [0.572,0.610]	0.562≈ [0.547,0.578]	3.44 [3.32,3.55]	3.08 [2.98,3.19]	2.004 [1.938,2.071]

S.4.4.5 One-factor ablations

Tables S.4.17–S.4.19 vary one parameter over its declared grid around each validation-selected RG, RG+Eq and SG configuration (selected value in bold), fixing the others. Localization and faithfulness are evaluated on the 565-image test split without feeding back into selection. On ViT-B/16, "+Eq" applies RG+Eq at the selected RG parameters. Validation-inadmissible configurations (Section A.2.3) are excluded.

Tab. S.4.17: ViT-B/16, ImageNet-S test split: one-factor ablations around the selected configurations. Center: the selected configuration (criterion in parentheses) around which the parameter in the Factor column is varied over the Value column, all other parameters fixed; bold: the center's own value. Symbols: Section S.4.4.

Center	Factor	Value	AL↑	PG↑	TK↑	PF_5↓	PF_{20}↓	Sel↓
RG (faith)	τ	0.3	0.705	0.802	0.731	8.34	7.60	5.195
		0.4	0.688	0.827	0.773	8.26	7.41	5.095
		0.5	0.665	0.821	0.793	8.22	7.20	4.956
		0.6	0.613	0.814	0.774	8.13	6.88	4.820
		0.7	0.549	0.770	0.741	8.11	6.78	4.808
		0.8	0.507	0.742	0.712	8.14	6.85	4.881
		0.9	0.482	0.697	0.683	8.21	7.02	4.983
		1	0.467	0.648	0.648	8.29	7.25	5.115
RG (faith)	λ_{sm}	-50	0.467	0.556	0.541	8.51	8.13	5.928
		-10	0.556	0.777	0.763	8.16	6.97	4.876
		-5	0.549	0.770	0.741	8.11	6.78	4.808
		-2	0.547	0.743	0.720	8.11	6.81	4.793
		-1	0.546	0.733	0.713	8.11	6.83	4.794
		0	0.546	0.727	0.706	8.13	6.87	4.804
RG (faith)	σ^2	1	0.558	0.738	0.703	8.16	6.90	4.790
		2	0.555	0.743	0.719	8.13	6.81	4.783
		5	0.549	0.770	0.741	8.11	6.78	4.808
		10	0.545	0.765	0.749	8.12	6.82	4.833
RG (faith)	ε	**0.1**	0.549	0.770	0.741	8.11	6.78	4.808
		0.25	0.595	0.804	0.778	8.16	6.95	4.846
		0.5	0.621	0.809	0.792	8.24	7.21	4.944
		1	0.630	0.812	0.794	8.29	7.43	5.051
RG (faith)	+Eq (N=50)	on	0.557	0.800	0.791	8.38	7.69	4.816

Center	Factor	Value	AL↑	PG↑	TK↑	PF_5↓	PF_{20}↓	Sel↓
RG (loc)	τ	0.3	0.723	0.816	0.746	8.33	7.64	5.201
		0.4	0.711	0.825	0.788	8.27	7.48	5.109
		0.5	0.691	0.830	0.812	8.23	7.31	4.969
		0.6	0.632	0.827	0.801	8.16	7.00	4.847
		0.7	0.556	0.777	0.763	8.16	6.97	4.876
		0.8	0.506	0.743	0.715	8.22	7.13	5.013
		0.9	0.477	0.683	0.673	8.29	7.28	5.143
		1	0.461	0.628	0.629	8.36	7.42	5.247
RG (loc)	λ_{sm}	-50	0.520	0.543	0.528	8.52	7.99	5.745
		-10	0.711	0.825	0.788	8.27	7.48	5.109
		-5	0.688	0.827	0.773	8.26	7.41	5.095
		-2	0.676	0.827	0.764	8.26	7.39	5.086
		-1	0.673	0.818	0.761	8.26	7.38	5.080
		0	0.671	0.818	0.758	8.26	7.38	5.076
RG (loc)	σ^2	1	0.707	0.823	0.769	8.23	7.35	4.986
		2	0.712	0.823	0.780	8.24	7.39	5.032
		5	0.711	0.825	0.788	8.27	7.48	5.109
		10	0.705	0.837	0.787	8.31	7.58	5.168
RG (loc)	ε	**0.1**	0.711	0.825	0.788	8.27	7.48	5.109
		0.25	0.709	0.842	0.780	8.29	7.55	5.155
		0.5	0.708	0.828	0.776	8.30	7.57	5.171
		1	0.708	0.823	0.773	8.31	7.59	5.181
RG (loc)	+Eq (N=50)	on	0.711	0.846	0.842	8.38	7.82	5.077
RG+Eq (faith)	τ	0.3	0.721	0.805	0.786	8.37	7.75	5.129
		0.4	0.712	0.832	0.822	8.39	7.81	5.049
		0.5	0.698	0.855	0.836	8.39	7.80	5.010
		0.6	0.681	0.846	0.833	8.39	7.77	4.943
		0.7	0.654	0.812	0.820	8.39	7.71	4.856
		0.8	0.610	0.805	0.803	8.38	7.70	4.814
		0.9	0.567	0.796	0.787	8.41	7.71	4.825
		1	0.534	0.777	0.772	8.42	7.74	4.876
RG+Eq (faith)	λ_{sm}	**-10**	0.654	0.812	0.820	8.39	7.71	4.856
		0	0.613	0.814	0.815	8.37	7.73	4.914
RG+Eq (loc)	τ	0.3	0.721	0.805	0.786	8.37	7.75	5.129
		0.4	0.712	0.832	0.822	8.39	7.81	5.049
		0.5	0.698	0.855	0.836	8.39	7.80	5.010
		0.6	0.681	0.846	0.833	8.39	7.77	4.943
		0.7	0.654	0.812	0.820	8.39	7.71	4.856
		0.8	0.610	0.805	0.803	8.38	7.70	4.814
		0.9	0.567	0.796	0.787	8.41	7.71	4.825
		1	0.534	0.777	0.772	8.42	7.74	4.876
RG+Eq (loc)	λ_{sm}	**-10**	0.698	0.855	0.836	8.39	7.80	5.010
		0	0.652	0.848	0.828	8.40	7.85	5.073

Tab. S.4.18: VGG-16, ImageNet-S test split: one-factor ablations around the selected configurations, as in Table S.4.17.

Center	Factor	Value	AL↑	PG↑	TK↑	PF_5↓	PF_{20}↓	Sel↓
RG (faith)	τ	0.3	0.685	0.736	0.673	14.48	9.64	5.632
		0.4	0.645	0.657	0.641	14.40	9.36	5.574
		0.5	0.626	0.653	0.628	14.27	9.14	5.451
		0.6	0.633	0.651	0.646	14.07	9.00	5.258
		0.7	0.643	0.671	0.674	14.03	8.98	5.142
		0.8	0.640	0.712	0.688	14.05	9.07	5.117
		0.9	0.630	0.742	0.693	14.05	9.17	5.126
		1	0.618	0.749	0.695	14.06	9.25	5.137
RG (faith)	λ	-5	0.518	0.458	0.536	17.60	13.82	7.362
		-2	0.623	0.604	0.631	15.90	11.58	5.852
		-1	0.636	0.676	0.661	15.03	10.35	5.404
		-0.3	0.640	0.694	0.676	14.54	9.70	5.231
		0	0.640	0.712	0.688	14.05	9.07	5.117
RG (faith)	ε	**0.1**	0.640	0.712	0.688	14.05	9.07	5.117
		0.25	0.639	0.692	0.684	14.18	9.23	5.146
		0.5	0.638	0.690	0.680	14.34	9.46	5.183
		1	0.639	0.690	0.676	14.63	9.79	5.249
RG (loc)	τ	**0.3**	0.685	0.736	0.673	14.48	9.64	5.632
		0.4	0.645	0.657	0.641	14.40	9.36	5.574
		0.5	0.626	0.653	0.628	14.27	9.14	5.451
		0.6	0.633	0.651	0.646	14.07	9.00	5.258
		0.7	0.643	0.671	0.674	14.03	8.98	5.142
		0.8	0.640	0.712	0.688	14.05	9.07	5.117
		0.9	0.630	0.742	0.693	14.05	9.17	5.126
		1	0.618	0.749	0.695	14.06	9.25	5.137
RG (loc)	λ	-5	0.561	0.543	0.555	17.05	13.21	6.898
		-2	0.657	0.696	0.634	15.31	10.80	5.885
		-1	0.674	0.735	0.656	14.85	10.13	5.731
		-0.3	0.680	0.727	0.666	14.66	9.87	5.675
		0	0.685	0.736	0.673	14.48	9.64	5.632
RG (loc)	ε	**0.1**	0.685	0.736	0.673	14.48	9.64	5.632
		0.25	0.691	0.747	0.673	14.66	9.91	5.659
		0.5	0.697	0.742	0.672	14.87	10.19	5.705
		1	0.702	0.742	0.671	15.13	10.55	5.767
SG (faith)	λ	-2	0.614	0.710	0.636	18.17	14.83	6.941
		-1	0.609	0.750	0.697	17.91	13.87	6.517
		-0.3	0.598	0.821	0.788	17.08	12.98	6.278
		0	0.547	0.781	0.764	17.63	14.35	6.789
SG (faith)	σ^2	**1**	0.598	0.821	0.788	17.08	12.98	6.278
		2	0.601	0.811	0.765	17.30	13.07	6.323
		5	0.604	0.798	0.731	17.66	13.50	6.417
		10	0.607	0.763	0.706	17.86	13.77	6.500
SG (faith)	gate $\tilde{\Phi}$	**off**	0.598	0.821	0.788	17.08	12.98	6.278

Center	Factor	Value	AL↑	PG↑	TK↑	PF_5↓	PF_{20}↓	Sel↓
		on	0.568	0.823	0.805	17.30	13.74	6.428
SG (loc)	λ	-5	0.568	0.550	0.558	18.17	14.31	7.509
		-2	0.620	0.703	0.681	17.84	13.63	6.404
		-1	0.597	0.770	0.770	17.28	13.32	6.217
		-0.3	0.566	0.830	0.809	17.33	13.75	6.452
		0	0.554	0.827	0.802	17.55	14.16	6.618
SG (loc)	σ^2	1	0.596	0.793	0.766	17.32	13.35	6.246
		2	0.597	0.786	0.766	17.31	13.34	6.232
		5	0.597	0.770	0.770	17.28	13.32	6.217
		10	0.595	0.772	0.776	17.25	13.30	6.206
SG (loc)	gate $\tilde{\Phi}$	off	0.614	0.696	0.628	18.19	15.00	7.050
		on	0.597	0.770	0.770	17.28	13.32	6.217

Tab. S.4.19: ResNet-50, ImageNet-S test split: one-factor ablations around the selected configurations, as in Table S.4.17.

Center	Factor	Value	AL↑	PG↑	TK↑	PF_5↓	PF_{20}↓	Sel↓
RG (faith)	τ	0.3	0.688	0.727	0.684	6.10	5.10	4.357
		0.4	0.676	0.712	0.692	6.08	5.03	4.340
		0.5	0.677	0.720	0.706	6.06	5.00	4.308
		0.6	0.673	0.704	0.711	5.99	4.96	4.270
		0.7	0.667	0.680	0.707	5.95	4.89	4.219
		0.8	0.665	0.688	0.711	5.89	4.85	4.134
		0.9	0.674	0.720	0.728	5.88	4.79	4.075
		1	0.674	0.743	0.738	5.88	4.74	4.030
RG (faith)	λ	-5	0.667	0.740	0.730	5.90	4.81	4.034
		-2	0.672	0.740	0.736	5.89	4.75	4.034
		-1	0.674	0.743	0.738	5.88	4.74	4.030
		-0.3	0.674	0.747	0.739	5.87	4.75	4.026
		0	0.675	0.747	0.740	5.86	4.74	4.023
RG (faith)	σ^2	**1**	0.674	0.743	0.738	5.88	4.74	4.030
		2	0.673	0.742	0.737	5.87	4.74	4.032
		5	0.671	0.736	0.736	5.88	4.75	4.031
		10	0.670	0.747	0.734	5.89	4.76	4.031
RG (faith)	ε	0.1	0.658	0.745	0.722	5.90	4.79	4.049
		0.25	0.674	0.743	0.738	5.88	4.74	4.030
		0.5	0.692	0.747	0.754	5.86	4.72	4.023
		1	0.712	0.766	0.761	5.85	4.72	4.038
RG (loc)	τ	0.3	0.733	0.761	0.719	6.03	5.04	4.272
		0.4	0.717	0.758	0.718	6.02	4.96	4.243
		0.5	0.709	0.740	0.720	5.98	4.92	4.216
		0.6	0.710	0.733	0.728	5.93	4.87	4.172
		0.7	0.716	0.745	0.741	5.91	4.80	4.125
		0.8	0.720	0.761	0.752	5.87	4.72	4.078
		0.9	0.719	0.763	0.760	5.85	4.70	4.047

Center	Factor	Value	AL↑	PG↑	TK↑	PF_5↓	PF_{20}↓	Sel↓
		1	0.713	0.770	0.763	5.85	4.70	4.035
RG (loc)	λ	-5	0.708	0.765	0.745	5.89	4.80	4.069
		-2	0.716	0.773	0.755	5.87	4.74	4.055
		-1	0.718	0.777	0.757	5.86	4.72	4.054
		-0.3	0.719	0.768	0.758	5.86	4.72	4.050
		0	0.719	0.763	0.760	5.85	4.70	4.047
RG (loc)	ε	0.1	0.656	0.712	0.712	5.89	4.84	4.106
		0.25	0.674	0.717	0.730	5.87	4.77	4.067
		0.5	0.696	0.736	0.749	5.84	4.70	4.042
		1	0.719	0.763	0.760	5.85	4.70	4.047

S.4.4.6 Pointing game at matched granularity

Quantus Pointing Game counts a hit if any maximum-attribution pixel lies in the mask. A piecewise-constant maximum spans hundreds to thousands of pixels for superpixel and 14×14 attention maps, versus usually one for pixel-level maps. Table S.4.20 reports the median maximum-set size, the expected hit rate of a uniformly drawn maximum pixel (PG_{1px}), and Pointing Game after average-pooling all maps to a common 16×16-pixel grid (PG_{16px}). Single-pixel sampling removes most of LIME's and KernelSHAP's advantage. At common granularity, leaders move closer: LRP-γ and AttnLRP lead on ViT-B/16; localization-selected SG and faithfulness-selected RG are not significantly below Grad-CAM++, the best VGG-16 baseline; Grad-CAM, LayerCAM and LRP-γ lead on ResNet-50 (McNemar tests against the best baseline).

Tab. S.4.20: Pointing game under the Quantus tie rule (PG, as in the main tables) and at matched granularity, ImageNet-S test split. max. set: median number of pixels attaining the map maximum; PG_{1px}: expected hit rate of one pixel drawn uniformly from that set; PG_{16px}: PG after average-pooling the map to 16×16-pixel cells. Rows: the `loc`-selected or fixed configuration of every method, plus the `faith` rows of our games. **Bold**: best per model and column.

Model	Method	PG	max. set (px)	PG_{1px}	PG_{16px}
ViT-B/16	Gradient	0.361	1	0.361	0.382
	SmGrad	0.706	1	0.706	0.835
	IntGrad	0.513	1	0.513	0.543
	LRP-ε	0.834	1	0.834	0.904
	LRP-γ	0.873	1	**0.873**	**0.968**
	AttnLRP	0.855	1	0.855	0.926
	DAVE	0.717	1	0.717	0.781
	TIBAV	0.878	256	0.784	0.878
	GAE	0.743	256	0.670	0.743
	AR	0.565	256	0.516	0.565
	LIME	0.897	1104	0.693	0.786
	KernelSHAP	**0.961**	1031	0.833	0.903
	RG (faith)	0.770	1	0.770	0.899
	RG	0.825	1	0.825	0.888
	RG+Eq (faith)	0.812	1	0.812	0.919
	RG+Eq	0.855	1	0.855	0.919
VGG-16	Gradient	0.738	1	0.738	0.812
	SmGrad	0.782	1	0.782	0.915
	IntGrad	0.770	1	0.770	0.860
	DeepLIFT	0.701	1	0.701	0.784
	LRP-ε	0.832	1	**0.832**	0.906
	LRP-γ	0.692	1	0.692	0.908
	GCAM	0.830	1	0.823	0.920
	GCAM++	0.823	1	0.815	**0.924**
	LCAM	0.720	1	0.715	0.834
	LIME	**0.972**	1103	0.826	0.899
	KernelSHAP	0.954	1011	0.826	0.894
	SG (faith)	0.821	1	0.821	0.883
	SG	0.770	1	0.770	0.908
	RG (faith)	0.712	1	0.712	0.915
	RG	0.736	1	0.736	0.873
ResNet-50	Gradient	0.556	1	0.556	0.630
	SmGrad	0.789	1	0.789	0.919
	IntGrad	0.635	1	0.635	0.743
	DeepLIFT	0.727	1	0.727	0.793
	LRP-ε	0.775	1	0.775	0.880
	LRP-γ	0.842	1	0.842	0.958
	GCAM	0.873	1	**0.862**	**0.959**
	GCAM++	0.834	1	0.827	0.945
	LCAM	0.869	1	0.860	0.958
	LIME	0.926	1058	0.729	0.844
	KernelSHAP	**0.965**	1014	0.825	0.912
	RG (faith)	0.743	1	0.743	0.903
	RG	0.763	1	0.763	0.912

S.4.4.7 Gradient-fallback attention

By default, RG conditions on realized attention and routes relevance only through Values, with zero Q, K gradient, as in CP-LRP (Ali et al., 2022) (Section 5.5.3). Gradient fallback uses pure autograd through attention, recovering zennit-style ε-LRP. Table S.4.21 compares both rules at the $\alpha\beta$-LRP anchor and the two validation-selected RG configurations on 565 test images. Softmax risk λ_{sm} acts on Value routing and is inactive under fallback. Fallback lowers all three localization metrics in every configuration (AL and TK significantly throughout) and worsens every PF_5, PF_{20} and Sel cell, significantly in four of nine.

Tab. S.4.21: ViT-B/16, RG: attention rule ablation on the ImageNet-S test split (565 images), at the anchor and at the loc- and faith-selected configurations. Each cell shows the gradient fallback on top and the composite Value rule of Theorem S.2.3 (gray) below; configuration symbols: Section S.4.4; †: paired difference significant (paired bootstrap / McNemar for PG, Holm-corrected over the table, $p < 0.05$).

Center	AL↑	PG↑	TK↑	PF_5↓	PF_{20}↓	Sel↓
anchor (τ=1)	0.504†	0.722	0.694†	8.19	6.98†	4.877†
	0.509	0.724	0.699	8.19	6.97	4.854
loc (τ=0.4,λ_{sm}=−10,σ^2=5,ε=0.1)	0.624†	0.743†	0.672†	8.31	7.50	5.364†
	0.711	0.825	0.788	8.27	7.48	5.109
faith (τ=0.7,λ_{sm}=−5,σ^2=5,ε=0.1)	0.543†	0.729†	0.706†	8.13	6.87†	4.816
	0.549	0.770	0.741	8.11	6.78	4.808

S.4.4.8 Qualitative comparison

Figure S.4.2 compares attribution maps for six ImageNet-S examples on ViT-B/16. Each column shows a method at its `loc`-selected configuration (Section A.2.3), or at its fixed configuration if the method has no tunable parameters (Gradient, TIBAV): RG with τ=0.4,λ_{sm}=−10,σ^2=5,ε=0.1 and RG+Eq with τ=0.5,λ_{sm}=−10,σ^2=2,N=50, both with the merged attention–Value rule, i.e. the composite rule of Theorem S.2.3, which routes through the conditioned attention block as one map $X \mapsto O$; SmoothGrad with N=50,σ/range=0.2; Integrated Gradients with N_{steps}=100 and a black baseline; DAVE with N=50,$\theta_{\max}$=0,$s_{\max}$=0,$t_{\max}$=0.5; LRP-γ with ε=0.1,γ=2; and AttnLRP with γ_{conv}=0.25,γ_{lin}=0.05,$\varepsilon_{\text{attn}}$=0.25. As in the evaluation, every map explains the ImageNet-S label, which is the top-1 prediction for all images except the kuvasz (predicted: English setter). The first image is from the test split, the other five are from the 50-image validation split used for the selection. Heatmaps show the raw evaluator-side maps, before the absolute value and max-abs normalization of Section A.2.2 (the 14×14 TIBAV map is upsampled by nearest neighbor, as for the metrics): red marks positive and blue negative attribution; each sign is divided by its 99th percentile over the image and clipped, so the top 1% of pixels saturate and concentrated maps stay legible in print; the same display transform is used for every method.

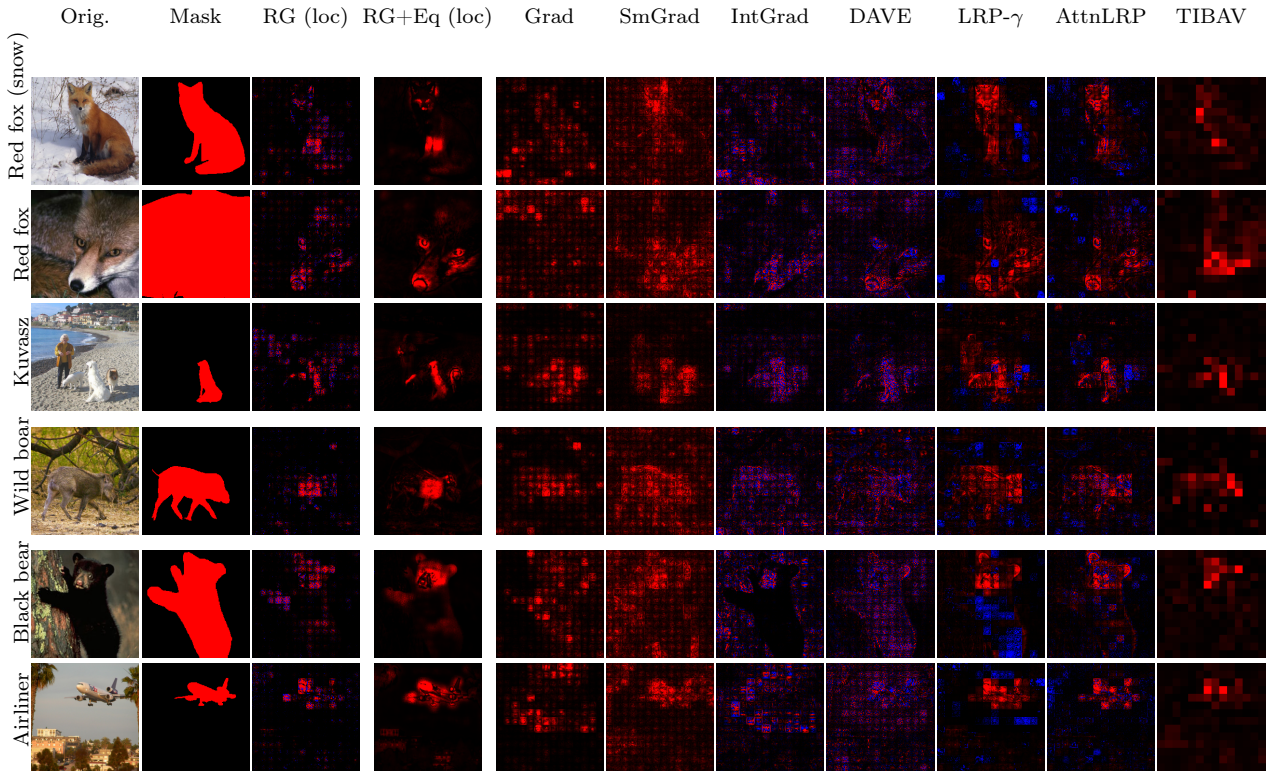


Fig. S.4.2: Dense qualitative comparison on six ImageNet-S examples (ViT-B/16). Columns: Original; ImageNet-S pixel Mask; RG and RG+Eq (merged attention–Value rule); Gradient, SmoothGrad, Integrated Gradients, DAVE, LRP-γ, AttnLRP and TIBAV. Every method is shown at its `loc`-selected configuration, Gradient and TIBAV at their fixed configurations (values in the text). Red: positive, blue: negative attribution, each sign divided by its 99th percentile over the image and clipped.

S.4.5 Numerical Diagnostic of the Transformer DC Construction

The deterministic `scripts/transformer_dc_demo.py` tests Section 5.3's construction on a three-token, width-three Transformer: one causal head, two post-residual LayerNorms, a width-four exact Gaussian-GELU MLP, mean pooling and two signed logits. An independent ordinary PyTorch computation uses identical parameters. The diagnostic runs on CPU in float64, without recentering, clipping or reference-activation substitution.

With random seed 23, the sweep combines $\varepsilon \in \{1, 10^{-2}, 10^{-5}\}$, weight scales $\{0.01, 0.1, 0.5\}$, and input scales $\{0.1, 1\}$. Two additional cases add common input-stream offsets of 1 and 10 to the $\varepsilon = 1$, weight-scale-0.01, input-scale-0.1 case. All twenty cases, including failures, are retained in `reports/evidence/transformer-dc/numeric.json`, together with the environment, source hash, per-stage reconstruction error, and stream magnitudes. The command is:

```
python scripts/transformer_dc_demo.py \
    --output RESULT.json
```

The benign case reconstructs logits within 2.78×10^{-17}; fifteen of twenty cases exceed 10^{-8} final maximum absolute error. Every tested $\varepsilon = 10^{-5}$ case loses output accuracy, with streams reaching 5.95×10^{111}. Separate primitive and zero-variance LayerNorm checks pass. These finite checks diagnose

implementation, without proving convexity or numerical stability. Catastrophic cancellation affects floating-point evaluation of even this small construction; a stable practical implementation remains open.

S.4.6 Formal Verification Details

Both papers share a finite, varying-width Lean MLP definition: an identity output or an affine layer followed by a smaller network, evaluated with ordinary or paired activations. Local difference preservation suffices algebraically for paired activations; induction proves complete-network recovery. This definition supports ReLU, mixing and Softplus, with a separate affine readout for ungated target logits.

The Transformer development defines signed projections, actual masked Query–Key products, coupled softmax, attention–Value products, input-dependent LayerNorm, residual additions and the feedforward subnetwork. Finite block lists allow arbitrary depth, including zero, an optional final LayerNorm and signed affine readout. Compilation to an explicit expression language equals the ordinary evaluator. Recursive two-stream evaluation proves Theorem 5.5: exact reconstruction and convex, monotone, non-negative components on non-negative doubled inputs. The positive/negative raw-input lift gives global convexity, without asserting raw-input monotonicity. The proofs derive structural properties from the closed-form softmax and normalization constructions, with positive ε and nonempty allowed-key sets. They use exact real arithmetic; Section S.4.5 diagnoses direct evaluation's limits.

A forward trace records actual gates. Each sensitivity row maps an input perturbation linearly to its scalar output change; paired rows describe partial sensitivities to both input streams. Their half-difference is the derivative along the balanced lift $(x/2, -x/2)$. At strict gate comparisons, the recursive Jacobian equals the complete network's Fréchet derivative.

The layer-wise shifted backward algorithm equals its closed formula for arbitrary widths and shifts. Source terms are derivatives of explicit Q-networks with the original prefix, one minimum and a bias-free linear absolute-weight tail. Checked identities cover shift-coefficient suffix products, sensitivity mass, separate cross-head partials and original-gradient reconstruction. Shifted algorithmic sensitivities remain distinct from derivatives through input-dependent forward re-centering; classical derivatives at gate ties are not asserted.

The game developments define finite sign-labeled paths, probabilities, discounts and occupation sums. Under the stated policies, discounted stopping occupations recover the actual MLP gradient; routing occupations recover the full $\alpha\beta$ recurrence from network contributions. The smooth case includes the full Softplus derivative. Attention uses a fixed matrix and the complete composite Value projection, including its bias; backward recovery alone asserts neither Query/Key differentiation nor global equilibrium. Generic finite layered-game saddle bounds and Bellman uniqueness are checked separately for actual randomized policies.

The trajectory module checks finite path normalization, dynamic-programming identities, geometric-mean overlap, terminal-state disagreement and positive survival conditioning. Hellinger probability laws remain distinct from discounted occupation measures. The regularization module checks the finite Gibbs variational maximum, attaining policy, binary stop/continue optimizer and actual Softplus derivatives.

The audit exposed a missing 1/2 in the four-partial backward invariant, a shift-product index error, an omitted weight matrix, and overclaims about differentiability, multiplicative convexity, re-centering

and nonzero combined gradients. Corrections and counterexamples are in reports/mathematical-corrections.md; nonzero raw sensitivity mass need not yield a nonzero balanced head gradient.

The source inventory lists statements outside the checked scope. Proofs and corrections certify neither Python implementations nor experiments. Independently, the table adapter regenerates thirteen outputs from the revised game's published summaries and checks them against the thesis. Diagnostic and hidden-monotonicity tables lacking raw summaries retain source provenance. Reproduce mathematical checks with `lake build` in `formalization/` and its explicit axiom audit; the README gives complete commands.

List of References

Achanta, R., A. Shaji, K. Smith, A. Lucchi, P. Fua, S. Süsstrunk (2012): SLIC Superpixels Compared to State-of-the-Art Superpixel Methods. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 34 (11), 2274–2282. DOI: 10.1109/TPAMI.2012.120. URL: https://doi.org/10.1109/TPAMI.2012.120 (cit. on p. 54).

Achtibat, R., S. M. V. Hatefi, M. Dreyer, A. Jain, T. Wiegand, S. Lapuschkin, W. Samek (2024): AttnLRP: Attention-Aware Layer-Wise Relevance Propagation for Transformers. *Proceedings of the 41st International Conference on Machine Learning*. Vol. 235. Proceedings of Machine Learning Research. PMLR, 135–168. URL: https://proceedings.mlr.press/v235/achtibat24a.html (cit. on pp. 54, 55).

Adebayo, J., J. Gilmer, M. Muelly, I. Goodfellow, M. Hardt, B. Kim (2018): Sanity Checks for Saliency Maps. *Advances in Neural Information Processing Systems*. Vol. 31. URL: https://proceedings.neurips.cc/paper/2018/hash/294a8ed24b1ad22ec2e7efea049b8737-Abstract.html (cit. on p. 48).

Ali, A., T. Schnake, O. Eberle, G. Montavon, K.-R. Müller, L. Wolf (2022): XAI for Transformers: Better Explanations through Conservative Propagation. *Proceedings of the 39th International Conference on Machine Learning (ICML)*. Vol. 162. Proceedings of Machine Learning Research. PMLR, 435–451. URL: https://proceedings.mlr.press/v162/ali22a.html (cit. on p. 70).

Anders, C. J., D. Neumann, W. Samek, K.-R. Müller, S. Lapuschkin (2026): Software for dataset-wide XAI: From local explanations to global insights with Zennit, CoRelAy, and ViRelAy. *PLOS One* 21 (1), e0336683. DOI: 10.1371/journal.pone.0336683. arXiv: 2106.13200. URL: https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0336683 (cit. on p. 19).

Arora, R., A. Basu, P. Mianjy, A. Mukherjee (2018): Understanding Deep Neural Networks with Rectified Linear Units. *International Conference on Learning Representations*. arXiv: 1611.01491 [cs.LG]. URL: https://openreview.net/forum?id=B1J_rgWRW (cit. on p. 2).

Auer, P., N. Cesa-Bianchi, P. Fischer (2002): Finite-time Analysis of the Multiarmed Bandit Problem. *Machine Learning* 47 (2-3), 235–256. ISSN: 1573-0565. DOI: 10.1023/a:1013689704352. URL: http://dx.doi.org/10.1023/A:1013689704352 (cit. on p. 22).

Bach, S., A. Binder, G. Montavon, F. Klauschen, K.-R. Müller, W. Samek (2015): On Pixel-Wise Explanations for Non-Linear Classifier Decisions by Layer-Wise Relevance Propagation. *PLOS ONE* 10 (7), e0130140. DOI: 10.1371/journal.pone.0130140. URL: https://doi.org/10.1371/journal.pone.0130140 (cit. on pp. 18, 19, 54).

Binder, A., L. Weber, S. Lapuschkin, G. Montavon, K.-R. Müller, W. Samek (2023): Shortcomings of Top-Down Randomization-Based Sanity Checks for Evaluations of Deep Neural Network Explanations. *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 16143–16152. DOI: 10.1109/cvpr52729.2023.01549. URL: http://dx.doi.org/10.1109/CVPR52729.2023.01549 (cit. on p. 48).

Boyd, S., L. Vandenberghe (2004): *Convex Optimization*. Cambridge University Press. ISBN: 9780511804441. DOI: 10.1017/cbo9780511804441. URL: http://dx.doi.org/10.1017/CBO9780511804441 (cit. on pp. 12, 24, 28).

Chattopadhay, A., A. Sarkar, P. Howlader, V. N. Balasubramanian (2018): Grad-CAM++: Generalized Gradient-Based Visual Explanations for Deep Convolutional Networks. *2018 IEEE Winter Conference on Applications of Computer Vision (WACV)*. IEEE, 839–847. DOI: 10.1109/WACV.2018.00097. URL: https://doi.org/10.1109/WACV.2018.00097 (cit. on p. 54).

Chefer, H., S. Gur, L. Wolf (2021a): Generic Attention-Model Explainability for Interpreting Bi-Modal and Encoder-Decoder Transformers. *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 397–406. DOI: 10.1109/ICCV48922.2021.00045. URL: https://openaccess.thecvf.com/content/ICCV2021/html/Chefer_Generic_Attention-Model_Explainability_for_Interpreting_Bi-Modal_and_Encoder-Decoder_Transformers_ICCV_2021_paper.html (cit. on p. 54).

Chefer, H., S. Gur, L. Wolf (2021b): Transformer Interpretability Beyond Attention Visualization. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 782–791. DOI: 10.1109/CVPR46437.2021.00084. URL: https://openaccess.thecvf.com/content/CVPR2021/html/Chefer_Transformer_Interpretability_Beyond_Attention_Visualization_CVPR_2021_paper.html (cit. on p. 54).

Deng, J., W. Dong, R. Socher, L.-J. Li, K. Li, L. Fei-Fei (2009): ImageNet: A Large-Scale Hierarchical Image Database. *2009 IEEE Conference on Computer Vision and Pattern Recognition*. IEEE, 248–255. DOI: 10.1109/CVPR.2009.5206848. URL: https://doi.org/10.1109/CVPR.2009.5206848 (cit. on p. 53).

Dombrowski, A.-K., C. J. Anders, K.-R. Müller, P. Kessel (2022): Towards robust explanations for deep neural networks. *Pattern Recognition* 121, 108194. ISSN: 0031-3203. DOI: 10.1016/j.patcog.2021.108194. URL: http://dx.doi.org/10.1016/j.patcog.2021.108194 (cit. on p. 26).

Everingham, M., L. Van Gool, C. K. I. Williams, J. Winn, A. Zisserman (2010): The PASCAL Visual Object Classes (VOC) Challenge. *International Journal of Computer Vision* 88 (2), 303–338. DOI: 10.1007/s11263-009-0275-4. URL: https://doi.org/10.1007/s11263-009-0275-4 (cit. on p. 62).

Fei, Y., Z. Yang, Y. Chen, Z. Wang, Q. Xie (2020): Risk-Sensitive Reinforcement Learning: Near-Optimal Risk-Sample Tradeoff in Regret. *NeurIPS*. URL: https://papers.neurips.cc/paper_files/paper/2020/hash/fdc42b6b0ee16a2f866281508ef56730-Abstract.html (cit. on p. 22).

Gao, S., Z.-Y. Li, M.-H. Yang, M.-M. Cheng, J. Han, P. Torr (2023): Large-Scale Unsupervised Semantic Segmentation. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 45 (6), 7457–7476. DOI: 10.1109/TPAMI.2022.3218275. arXiv: 2106.03149 [cs.CV]. URL: https://doi.org/10.1109/TPAMI.2022.3218275 (cit. on p. 53).

Gast, J., S. Roth (2018): Lightweight Probabilistic Deep Networks. *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*. IEEE, 3369–3378. DOI: 10.1109/cvpr.2018.00355. URL: http://dx.doi.org/10.1109/cvpr.2018.00355 (cit. on p. 22).

Gaubert, S., Y. Vlassopoulos (2025): *Relu and softplus neural nets as zero-sum turn-based games*. DOI: 10.48550/arXiv.2512.20582. arXiv: 2512.20582 [cs.LG]. URL: https://arxiv.org/abs/2512.20582 (cit. on pp. 4, 24).

Hedström, A., L. Weber, D. Krakowczyk, D. Bareeva, F. Motzkus, W. Samek, S. Lapuschkin, M. M.-C. Höhne (2023): Quantus: An Explainable AI Toolkit for Responsible Evaluation of Neural Network Explanations and Beyond. *Journal of Machine Learning Research* 24 (34), 1–11. URL: https://jmlr.org/papers/v24/22-0142.html (cit. on pp. 53, 62).

Hendrycks, D., K. Gimpel (2016): Gaussian Error Linear Units (GELUs). *arXiv preprint arXiv:1606.08415*. DOI: 10.48550/arXiv.1606.08415. URL: https://arxiv.org/abs/1606.08415 (cit. on p. 24).

Jiang, P.-T., C.-B. Zhang, Q. Hou, M.-M. Cheng, Y. Wei (2021): LayerCAM: Exploring Hierarchical Class Activation Maps for Localization. *IEEE Transactions on Image Processing* 30, 5875–5888. DOI: 10.1109/TIP.2021.3089943. URL: https://doi.org/10.1109/TIP.2021.3089943 (cit. on p. 54).

Lundberg, S. M., S.-I. Lee (2017): A Unified Approach to Interpreting Model Predictions. *Advances in Neural Information Processing Systems*. Ed. by I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, R. Garnett. Vol. 30. Curran Associates, Inc. URL: https://proceedings.neurips.cc/paper_files/paper/2017/hash/8a20a8621978632d76c43dfd28b67767-Abstract.html (cit. on p. 54).

Montavon, G., A. Binder, S. Lapuschkin, W. Samek, K.-R. Müller (2019): Layer-Wise Relevance Propagation: An Overview. In: *Explainable AI: Interpreting, Explaining and Visualizing Deep Learning*. Ed. by W. Samek, G. Montavon, A. Vedaldi, L. K. Hansen, K.-R. Müller. Cham: Springer International Publishing, 193–209. ISBN: 978-3-030-28954-6. DOI: 10.1007/978-3-030-28954-6_10. URL: https://doi.org/10.1007/978-3-030-28954-6_10 (cit. on p. 54).

Ribeiro, M. T., S. Singh, C. Guestrin (2016): "Why Should I Trust You?": Explaining the Predictions of Any Classifier. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. KDD '16. San Francisco, California, USA: Association for Computing Machinery, 1135–1144. ISBN: 9781450342322. DOI: 10.1145/2939672.2939778. URL: https://doi.org/10.1145/2939672.2939778 (cit. on p. 54).

Russakovsky, O., J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein, A. C. Berg, L. Fei-Fei (2015): ImageNet Large Scale Visual Recognition Challenge. *International Journal of Computer Vision* 115 (3), 211–252. ISSN: 1573-1405. DOI: 10.1007/s11263-015-0816-y. URL: http://dx.doi.org/10.1007/s11263-015-0816-y (cit. on p. 53).

Selvaraju, R. R., M. Cogswell, A. Das, R. Vedantam, D. Parikh, D. Batra (2017): Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization. *2017 IEEE International Conference on Computer Vision (ICCV)*, 618–626. DOI: 10.1109/ICCV.2017.74. URL: https://openaccess.thecvf.com/content_iccv_2017/html/Selvaraju_Grad-CAM_Visual_Explanations_ICCV_2017_paper.html (cit. on p. 54).

Shrikumar, A., P. Greenside, A. Kundaje (2017): Learning Important Features Through Propagating Activation Differences. *Proceedings of the 34th International Conference on Machine Learning*. Ed. by D. Precup, Y. W. Teh. Vol. 70. Proceedings of Machine Learning Research. PMLR, 3145–3153. URL: https://proceedings.mlr.press/v70/shrikumar17a.html (cit. on p. 54).

Simonyan, K., A. Vedaldi, A. Zisserman (2014): Deep Inside Convolutional Networks: Visualising Image Classification Models and Saliency Maps. *2nd International Conference on Learning Representations, ICLR 2014, Banff, AB, Canada, April 14-16, 2014, Workshop Track Proceedings*. Ed. by Y. Bengio, Y. LeCun. URL: http://arxiv.org/abs/1312.6034 (cit. on p. 54).

Smilkov, D., N. Thorat, B. Kim, F. Viégas, M. Wattenberg (2017): *SmoothGrad: removing noise by adding noise*. DOI: 10.48550/arXiv.1706.03825. arXiv: 1706.03825 [cs.LG]. URL: https://arxiv.org/abs/1706.03825 (cit. on p. 54).

Sundararajan, M., A. Taly, Q. Yan (2017): Axiomatic Attribution for Deep Networks. *Proceedings of the 34th International Conference on Machine Learning*. Ed. by D. Precup, Y. W. Teh. Vol. 70. Proceedings of Machine Learning Research. PMLR, 3319–3328. URL: https://proceedings.mlr.press/v70/sundararajan17a.html (cit. on p. 54).

Wróbel, A., S. Gairola, J. Tabor, B. Schiele, B. M. Zieliński, D. D. Rymarczyk (2026): DAVE: Distribution-Aware Attribution via ViT Gradient Decomposition. *Proceedings of the 43rd International Conference on Machine Learning*. Vol. 306. Proceedings of Machine Learning Research. PMLR, 135468–135511. arXiv: 2602.06613 [cs.CV]. URL: https://proceedings.mlr.press/v306/wrobel26a.html (cit. on p. 54).

Zimmermann, J. P., G. Loho (2026): Hidden Monotonicity: Explaining Deep Neural Networks via their DC Decomposition. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. Highlight, 24107–24117. arXiv: 2601.07700 [cs.CV]. URL: https://openaccess.thecvf.com/content/CVPR2026/html/Zimmermann_Hidden_Monotonicity_Explaining_Deep_Neural_Networks_via_their_DC_Decomposition_CVPR_2026_paper.html (cit. on p. 2).